TUM School of Management
Technische Universität München

Balanced and Staggered Routing in Autonomous Mobility-on-Demand Systems

Antonio Coppola

Vollständiger Abdruck der von der TUM School of Management der Technischen Universität München zur Erlangung des akademischen Grades eines

Doktor der Wirtschafts- und Sozialwissenschaften (Dr. rer. pol.)

genehmigten Dissertation.

Vorsitz : Prof. Dr. Martin Grunow

Prüfende der Dissertation:

1. Prof. Dr. Maximilian Schiffer
2. Prof. Dr. Claudia Archetti

Die Dissertation wurde am 18.04.2026 bei der Technischen Universität München eingereicht und durch die TUM School of Management angenommen.

Abstract

Autonomous mobility-on-demand (AMoD) systems redefine urban ride-hailing by replacing decentralized driver decisions with centrally coordinated fleets of self-driving vehicles. Centralized control enables system-level optimization of routing and departure decisions, offering the potential to enhance vehicle utilization, reduce operating costs, and alleviate congestion compared to traditional ride-hailing services. However, realizing these gains requires routing algorithms that coordinate large-scale fleets across space and time under demand uncertainty and evolving traffic conditions, all while remaining computationally tractable for real-world urban networks.

This thesis develops a unified optimization framework for the spatiotemporal coordination of AMoD operations, structured around two complementary mechanisms: balanced and staggered routing. The framework builds on a discretized adaptation of Vickrey's bottleneck model, capturing the essential dynamics of congestion propagation and vehicle interactions. The analysis progresses from offline planning with perfect information to real-time control under uncertainty, incrementally increasing both modeling fidelity and operational realism.

The first part of the thesis establishes the methodological foundation by introducing and analyzing staggered routing as a mechanism for temporal coordination along fixed routes within AMoD systems. It formulates the problem as a mixed-integer linear program and develops a scalable matheuristic to address the associated computational challenges. A first case study on the Manhattan network demonstrates that temporal coordination substantially reduces traffic delays and improves network efficiency, proving the potential of departure time optimization for congestion management.

Building on these insights, the second part integrates balanced and staggered routing into a unified framework for jointly optimizing vehicle routes and departure times. To address the resulting combinatorial complexity at a city-wide scale, it develops a metaheuristic based on large neighborhood search. A second case study conducted on the Manhattan network reveals that spatial load balancing and temporal staggering serve complementary functions: route balancing alleviates pressure on critical corridors, while departure-time control smooths residual peak demand. Their joint application yields system-wide delay reductions that closely approximate the combined potential of both mechanisms when applied individually. These benefits persist in mixed-traffic settings, where coordinating only a fraction of vehicles yields network-wide congestion reductions that also improve traffic conditions for the uncontrolled share.

The final part addresses real-time AMoD operations under stochastic and evolving demand, where the controller makes sequential decisions with limited foresight. The thesis resolves this challenge by leveraging high-quality offline solutions to inform real-time control through an optimization-augmented learning pipeline. By integrating an optimization layer directly into the learning architecture, the framework couples prediction with decision-making, enabling proactive control within complex, constrained combinatorial spaces. Numerical experiments show that the framework recovers a substantial share of offline coordination gains when deployed online, offering a scalable pathway toward efficient autonomous urban mobility.

Acknowledgments

The journey that led me to writing this thesis has been about four years long. During this (long!) time, countless beautiful things have happened to me, and summarizing them all in a few pages would be a truly heroic task. I can only thank a small subset of the many people who come to mind while writing these lines and who have been part of this journey. If you, dear reader, do not find your name in the following paragraphs, yet we have shared emotions and a true connection, please know that I thank you as well.

To start off, I would like to thank my family. *Mamma* and *Papà*, you taught me the value of education and the importance of building a path of my own. *Nonna*, you have been an example of unwavering dedication and hard work. *Dario* and *Giorgia*, I am proud of the people you are becoming and deeply grateful for the special bond we share. This thesis is especially dedicated to you. And, very importantly, *Janis*, my lovely dog, you are a constant source of joy for all of us.

I would also like to thank my friends from Napoli — *Davide*, *Manu*, *Giorgio*, *Ciro*, *Naji*, and especially *Francesco*. Thank you, Fra. Our bond runs deep, and above all, I cherish the incredible holidays we have spent together, lost in the wild. Those days always replenish me in full, giving me fresh energy to pursue new goals and projects. You are a true inspiration, with your incredible humor, vibrant energy, ambition, and love for life.

When I left Napoli to come to Munich, I knew I was leaving my family — but I didn't know I was about to find a new one. I am deeply grateful to my people here in Munich, with whom I have shared countless memories: trips, concerts, art events, brunches, dinners, hikes, heartbreaks, tears, and laughter. Thank you, *Pia*, *Nicola*, *Tom*, *Anna*, *Davide*, *Saba*, *Teo*, *Francisco*, *Sofia*, *Ankur*, *Ali*, and *Alessandro*. You made Munich feel like home. During my time in Canada, you were what I truly missed, and I feel blessed to have you in my life.

I would like to thank my fantastic colleagues — especially *Chris*, *Caro*, *Christina*, *Breno*, *Banu*, *Shidi*, *Georgina*, *Paul*, *Leonard*, and *Heiko*. When I joined the group, I had so much to learn, both professionally and personally, and you helped me through this journey. You encouraged self-reflection, supported me, and made everyday life at work genuinely enjoyable. I am thankful to all my co-authors, and in particular to *Gerhard*, who guided me at the very beginning of my research path. He taught me how to think critically, write clean code, challenge ideas constructively, and navigate a research environment — always with a fun and laid-back attitude.

Finally, I am deeply thankful to my PhD supervisor, *Max*, for the opportunity he gave me. Joining your group and immersing myself in such a stimulating academic environment has profoundly shaped my approach to life. Thank you for the time spent reading my papers, for patiently correcting my *Dienstreisenanträge*, for the moments shared inside and outside the office, for the chance to spend my research stay in Montréal, and for your guidance both on a personal and a professional level. Most importantly, thank you for giving me the chance to build a life here in Munich and to experience everything that has shaped this unforgettable life chapter.

Ringraziamenti

Il percorso che mi ha portato a scrivere questa tesi è durato circa quattro anni. In questo (lungo!) periodo ho vissuto innumerevoli meravigliose esperienze, e riassumerle tutte in poche pagine sarebbe davvero un'impresa eroica. Posso solo ringraziare una piccola parte delle tante persone che mi vengono in mente mentre scrivo queste righe e che hanno fatto parte di questo viaggio. Se tu, caro lettore, non dovessi trovare il tuo nome nei paragrafi seguenti, ma dovessimo aver condiviso emozioni e una vera connessione, sappi che ringrazio anche te.

Per iniziare, desidero ringraziare la mia famiglia. *Mamma* e *Papà*, mi avete insegnato il valore dell'istruzione e l'importanza di costruire un percorso tutto mio. *Nonna*, sei sempre stata un esempio di dedizione incrollabile e di impegno instancabile. *Dario* e *Giorgia*, sono orgoglioso delle persone che state diventando e profondamente grato per il legame speciale che condividiamo. Questa tesi è dedicata soprattutto a voi. E *Janis*, il mio amato cane, sei una fonte costante di gioia per tutti noi.

Vorrei ringraziare anche i miei amici di Napoli — *Davide*, *Manu*, *Giorgio*, *Ciro*, *Naji* e soprattutto *Francesco*. Grazie, Fra. Il nostro legame è profondo e, più di ogni altra cosa, custodisco le incredibili vacanze che abbiamo trascorso insieme negli anni, persi nella natura. Quei giorni mi ricaricano completamente e mi danno nuova energia per perseguire sogni e obiettivi. Sei una vera fonte di ispirazione, con il tuo incredibile senso dell'umorismo, la tua energia vitale, la tua ambizione e il tuo amore per la vita.

Quando ho lasciato Napoli per venire a Monaco, sapevo che stavo lasciando la mia famiglia, ma non sapevo che ne avrei trovata un'altra. Sono profondamente grato alle persone che ho incontrato qui e con cui ho condiviso ricordi indimenticabili: viaggi, concerti, mostre, brunch, cene, escursioni, cuori spezzati, lacrime e risate. Grazie, *Pia*, *Nicola*, *Tom*, *Anna*, *Davide*, *Saba*, *Teo*, *Francisco*, *Sofia*, *Ankur*, *Ali* e *Alessandro*. Avete reso Monaco casa. Durante il mio periodo in Canada, siete stati ciò che mi è mancato davvero, e mi sento fortunato ad avervi nella mia vita.

Vorrei ringraziare anche i miei fantastici colleghi, in particolare *Chris*, *Caro*, *Christina*, *Breno*, *Banu*, *Shidi*, *Georgina*, *Paul*, *Leonard* e *Heiko*. Quando sono entrato nel gruppo, non immaginavo neanche quanto avessi da imparare, sia sul piano professionale sia su quello personale. Grazie per avermi accompagnato lungo questo percorso, per avermi incoraggiato a riflettere su me stesso, per aver sostenuto e reso la vita quotidiana al lavoro genuinamente piacevole. Sono grato a tutti i miei coautori e, in particolare, a *Gerhard*, che mi ha guidato all'inizio del mio percorso di ricerca. Mi ha insegnato a pensare in modo critico, a scrivere codice, a mettere in discussione le idee in modo costruttivo e a muovermi in un ambiente di ricerca, sempre con un'attitudine divertente e rilassata.

Infine, desidero ringraziare il mio supervisore di dottorato, *Max*, per l'opportunità che mi ha dato. Entrare nel tuo gruppo e immergermi in un ambiente accademico così stimolante ha profondamente influenzato il mio approccio alla vita. Grazie per il tempo dedicato alla lettura dei miei articoli, per aver corretto con pazienza le mie *Dienstreisenanträge*, per i momenti condivisi dentro e fuori dall'ufficio, per la possibilità di trascorrere il mio periodo di ricerca a Montréal e per la tua guida, sia personale sia professionale. Soprattutto, grazie per avermi dato la possibilità di costruire una vita qui a Monaco e di vivere tutto ciò che ha reso indimenticabile questo capitolo della mia vita.

A Dario e Giorgia

Contents

List of Figures

List of Tables

Acronyms

AMoD	Autonomous mobility-on-demand
CO	Combinatorial optimization
COaML	Combinatorial optimization-augmented machine learning
CP	Constraint programming
CTM	Cell transmission model
DAG	Directed acyclic graph
DSO	Dynamic system optimum
DTA	Dynamic traffic assignment
DUE	Dynamic user equilibrium
DUO	Dynamic user optimum
FIFO	First-in-first-out
GNN	Graph neural network
HC	High-congestion
J3	Three-machine job shop scheduling problem
KPI	Key performance indicator
KSPwLO	k-shortest paths with limited overlap
LC	Low-congestion
LNS	Large neighborhood search
MAE	Mean absolute error
MDP	Markov decision process
MFD	Macroscopic fundamental diagram
MILP	Mixed integer linear program
ML	Machine learning
MLP	Multilayer perceptrons
MPC	Model predictive control
NWT	No-wait constraints
RCPSP	Resource-constrained project scheduling

RDUO	Reactive dynamic user optimum
RL	Reinforcement learning
RMSE	Root mean square error
SGD	Stochastic gradient descent
SO	System optimum
SPO	Smart predict–then–optimize
SRDTC	Simultaneous route and departure time choice
UPT	Unit processing times

1 Introduction

Over the last decade, the rapid proliferation of ride-hailing platforms – digitally coordinated intermediaries matching passenger demand with a decentralized pool of drivers (Wang & Yang, 2019) – has fundamentally restructured the global urban transportation landscape. Since the emergence of major app-based services, adoption has surged, with annual trips tripling to approximately 16.5 billion between 2016 and 2019 (Heineke et al., 2021). In major metropolitan hubs such as New York City and San Francisco, app-based mobility volumes now exceed traditional taxi trips by more than an order of magnitude on a typical weekday. This dominance is even more pronounced in high-density urban areas such as Manhattan, where ride-hailing services account for over 70% of all for-hire vehicle activity (Lam et al., 2021).

The widespread adoption of ride-hailing services is driven by the seamless, digitized, door-to-door mobility they offer. Leveraging ubiquitous mobile infrastructure, they enable rapid matching of requests with available vehicles across geographically distributed fleets, thereby reducing expected passenger wait times relative to traditional street-hailing or telephone-based dispatch systems (Rayle et al., 2016). Beyond these operational strengths, ride-hailing platforms have established new benchmarks for convenience and transparency through upfront fare estimates, real-time vehicle tracking, and bilateral rating systems that minimize information asymmetry and enhance perceived safety (Brown & LaValle, 2021).

However, these service advantages reflect only the user-facing dimension of the technology. Beyond improving individual travel experiences, ride-hailing platforms generate system-wide effects through the aggregation of millions of platform-mediated decisions, including trip requests, dynamic pricing, vehicle dispatching, and routing (Ward et al., 2021). As these actions collectively reshape traffic flows and influence the allocation of scarce urban road space, studying ride-hailing requires a dual perspective that balances individual service optimization with broader network-level implications for urban mobility.

1.1 Negative externalities of ride-hailing systems

Despite their high level of service, ride-hailing platforms have introduced substantial system-level costs, with escalating traffic congestion emerging as a primary source of environmental and economic harm (Levy et al., 2010). Empirical evidence underscores the scale of this impact: in San Francisco, app-based services accounted for an estimated 51% of the growth in vehicle hours of delay and a 55% decline in average traffic speeds between 2010 and 2016 (Erhardt et al., 2019). Multi-city analyses corroborate these findings, indicating that ride-hailing entry prolonged the duration of congested conditions by approximately 4.5% across major U.S. metropolitan areas (Diao et al., 2021).

These adverse impacts arise from several mutually reinforcing mechanisms. First, app-based mobility often increases total vehicle volumes by inducing additional travel demand rather than substituting for private car use. Survey evidence from U.S. cities indicates that approximately 49% of trips booked via app-based platforms either displace more sustainable modes, such as walking, cycling, or public transit, or represent induced travel that would not have occurred without these services (Clewlow & Mishra, 2017). Second, app-based mobility generates substantial volumes of empty vehicle travel, as drivers cruise or reposition in anticipation of future requests. Off-duty mileage can account for up to 50% of total ride-hailing vehicle travel, markedly amplifying congestion and urban emissions (Henao & Marshall, 2019). Third, the decentralized operational architecture of these services further exacerbates congestion. Without system-wide coordination, individual drivers make independent decisions regarding vehicle positioning, cruising locations, and route choices to maximize personal earnings (Ashkrof et al., 2024). This myopic, profit-driven behavior leads to frequent clustering in high-demand corridors and systematically overloads physical bottlenecks, especially during peak hours (Dong et al., 2024).

Together, induced demand, empty-vehicle circulation, and decentralized decision making generate a misalignment between individual incentives and system-level performance. In response, cities have attempted to internalize these externalities through regulatory and pricing-based instruments, with mixed results. For example, congestion surcharges in New York City reduced ride-hailing volumes by approximately 11% but failed to produce measurable improvements in traffic conditions, suggesting that pricing alone may be insufficient to correct coordination failures in platform-based mobility (Li & Vignon, 2024). Combined with persistent technical, political, and equity challenges associated with congestion pricing, these findings motivate complementary congestion-mitigation strategies centered on coordinated, fleet-level operational control (Ke & Qian, 2023).

1.2 Autonomous mobility as a coordinated alternative

In recent years, autonomous mobility-on-demand (AMoD) systems — centrally managed fleets of self-driving vehicles providing on-demand transportation (Pavone, 2015) — have rapidly transitioned from localized pilot projects to large-scale commercial deployment. By mid-2025, robotaxi services in China surpassed 11 million cumulative trips, while U.S. operators delivered over 250,000 rides per week across major metropolitan areas. With leading firms now expanding into broader European and Asian markets, the sector is projected to reach $40 billion by 2030 (Yazawa et al., 2025).

Beyond commercial scaling, this technological maturation enables a structural response to the persistent misalignment between individual behavior and system-level performance inherent in conventional ride-hailing. By centralizing dispatching, routing, and repositioning, AMoD systems internalize the network externalities that decentralized, driver-based platforms cannot directly control. In conventional ride-hailing markets, drivers independently choose where and when to operate, typically gravitating toward perceived high-demand areas. While such decisions are individually rational, their aggregate effect often leads to spatial imbalances in vehicle supply and localized congestion that degrade overall system performance. In principle, this unified control promises higher vehicle utilization, reduced congestion, and improved network efficiency at scale. The emerging consensus in the literature suggests that the magnitude of these benefits depends critically on the quality of the underlying operational policies. When jointly optimized, centralized operations can reduce empty mileage by up to 40% and total travel time by up to 35% in dense metropolitan settings (cf. Hörl, 2020; Hyland & Mahmassani, 2020; Spieser et al., 2014; Wollenstein-Betech et al., 2022). Conversely, weak coordination can increase empty mileage by up to 15%, eroding the intrinsic advantages of centralization (Alotaibi et al., 2023). Congestion mitigation is therefore not a guaranteed byproduct of automation, but a function of deliberate operational control.

Automation not only has the potential to strengthen spatial coordination but also expands the operational decision space by enabling control over fleet departure timing. In human-driven systems, routing can be steered to some extent through navigation guidance and incentives, even if compliance is imperfect. Fine-grained departure timing is even more elusive, as it lacks robust mechanisms to incentivize driver compliance and ultimately hinges on humans' readiness to implement system suggestions. This variability renders reliable fleet-wide temporal coordination infeasible in practice. AMoD systems circumvent these limitations by replacing human discretion with automated execution, allowing both spatial and temporal decisions to be implemented deterministically.

Despite these expanded capabilities, how to effectively realize the potential of departure-time optimization remains an open research question. On the one hand, precise control

over departure timing enables platforms to align vehicle entries with network capacity, smoothing demand peaks and mitigating transient congestion. On the other hand, poorly coordinated departures risk injecting vehicles into already saturated segments, potentially intensifying peak-hour bottlenecks beyond levels observed in decentralized systems. Furthermore, the extent to which coordinated departure timing can complement and amplify spatial optimization remains largely unexplored. Addressing this spatial–temporal interplay is therefore essential to unlocking the full potential of centralized coordination and achieving system-level efficiency gains in urban mobility.

1.3 Balanced and staggered routing

While the potential for centralized coordination is clear, translating this capability into implementable fleet control requires a framework that addresses the interdependent spatial and temporal dimensions of urban congestion. This thesis proposes an optimization paradigm built on two core coordination principles: *balanced routing* and *staggered routing*. Balanced routing serves as the *spatial* coordination mechanism, distributing AMoD flows across the network topology to prevent the formation of localized bottlenecks. Rather than assigning all vehicles to shortest-distance routes, which often leads to oversaturation of critical links, balanced routing redirects traffic toward underutilized segments to harmonize link utilization throughout the network. Staggered routing provides the complementary *temporal* coordination mechanism by treating departure time as a controllable decision variable. Instead of dispatching vehicles immediately upon request, staggering strategically defers departures within admissible service windows. By buffering vehicle entries while still guaranteeing timely arrivals, the system smooths transient demand peaks to prevent network oversaturation.

Implementing such high-dimensional coordination is inherently challenging, as the joint routing–scheduling action space grows exponentially with the number of trip requests, network scale, and the degree of temporal flexibility. These decisions are dynamically coupled through traffic propagation: a deferred departure at one location alters downstream conditions over time, requiring a framework that captures complex temporal interdependencies. Consequently, any viable solution must overcome the combinatorial growth that renders naive optimization impractical for real-time deployment. Moreover, the system must remain robust to demand uncertainty and stochastic traffic fluctuations, necessitating an anticipatory control logic that adapts to evolving conditions while maintaining system-level coordination.

This thesis addresses these technical and operational challenges by developing a scalable mathematical framework for the joint spatiotemporal control of AMoD traffic in large-

scale, complex networks. It establishes the modeling foundations, theoretical insights, and algorithmic machinery required for proactive fleet management, capable of navigating both combinatorial complexity and environmental uncertainty.

1.4 Thesis scope and contributions

The overarching aim of this thesis is to establish the modeling and algorithmic foundations of *balanced* and *staggered* routing for AMoD systems. The central premise is that the deliberate *spatiotemporal coordination* of vehicle movements, enabled by centralized fleet control, can substantially reduce network inefficiencies while preserving the convenience of on-demand mobility. To this end, the thesis develops a new class of optimization models that capture the complex interplay between routing decisions, departure-time choices, and network load. These models are designed to be expressive enough to reflect non-linear traffic dynamics yet structured enough to admit scalable solution methods for large-scale urban environments. A key objective throughout the thesis is to bridge the gap between theoretical coordination mechanisms and computationally viable control strategies that can operate at metropolitan scale.

Within this scope, the thesis provides three core contributions. First, it formalizes staggered routing as a novel operational control mechanism. To the best of our knowledge, this work provides the first systematic formulation of departure-time staggering as a controllable decision variable in fleet routing. By treating departure-time assignment as a controllable decision variable, the system can strategically buffer vehicle entries to smooth demand peaks. The thesis establishes the NP-hardness of the staggered routing problem and develops a tailored matheuristic that overcomes the scalability limits of direct mixed-integer formulations, enabling the solution of instances beyond the reach of exact methods. It evaluates the methodology through a case study of the Manhattan road network, which serves to appraise the efficacy of the proposed algorithms and quantify the extent to which temporal staggering alone can reduce systemic congestion in a high-density urban environment.

Second, it develops an integrated framework that unifies balanced and staggered routing within a single control paradigm. This constitutes the first optimization framework that simultaneously captures spatial routing and temporal departure-time coordination within a unified formulation. By jointly optimizing spatial and temporal decisions, the framework captures spatiotemporal interactions that are overlooked when routing and departure times are treated in isolation. Methodologically, we propose a scalable large neighborhood search (LNS) metaheuristic tailored to solve the associated optimization problem. The algorithm exploits the problem structure through adaptive neighborhood

exploration, enabling the computation of high-quality solutions for large-scale instances that would otherwise be computationally intractable. Solution quality is benchmarked on the Manhattan network, and system-wide effects are evaluated in both fully autonomous scenarios and mixed-traffic settings where the AMoD fleet interacts with unmanaged background demand.

Third, it formulates the online balanced and staggered routing problem as a Markov decision process (MDP) with demand revealed sequentially over time. This represents one of the first attempts to combine spatiotemporal fleet coordination with sequential decision-making under uncertainty in AMoD systems. To solve its instances, it develops an algorithm that reformulates the decision process as a shortest-path problem embedded within an optimization-augmented machine learning pipeline. By leveraging offline expert solutions to guide online control, the proposed architecture enables near-real-time decision making despite stochastic demand and partial observability. Numerical experiments demonstrate that optimization-augmented methods recover a substantial share of the theoretical efficiency gains found in offline settings, yielding a responsive and principled control logic for time-varying traffic environments.

The scope is focused on the *operational* control of AMoD systems. We consider a single operator managing a homogeneous fleet on a fixed road network, treating strategic aspects, such as infrastructure investment, pricing, and long-term demand evolution, as exogenous variables. Within this operational scope, the thesis advances the state of the art by developing novel coordination mechanisms and scalable algorithms designed for deployment on city-scale transportation networks.

1.5 Structure of the thesis

The thesis is organized to progressively increase both modeling fidelity and control complexity, moving from isolated coordination mechanisms to fully integrated, real-time decision-making under uncertainty. Table 1.1 summarizes the routing components and demand assumptions addressed in each methodological chapter.

Chapter 2 reviews the relevant literature on AMoD systems and dynamic traffic as-

Table 1.1: Routing components and demand assumptions across chapters.

	Routing Components		**Demand Modeling**	
	Balancing	**Staggering**	**Deterministic**	**Stochastic**
Chapter 3		✓	✓	
Chapter 4	✓	✓	✓	
Chapter 5	✓	✓		✓

signment, identifying gaps in existing fleet management research regarding the lack of integrated spatiotemporal control. Furthermore, it surveys recent advances in optimization-augmented machine learning, providing the conceptual groundwork for bridging high-level coordination with real-time execution.

Chapter 3 isolates the temporal dimension by formalizing the *staggered routing problem* under fixed spatial paths. It introduces a traffic model inspired by bottleneck dynamics, analyzes the computational complexity of departure-time control, and develops scalable solution methods. A case study on the Manhattan network demonstrates the effectiveness of temporal buffering in mitigating peak congestion.

Chapter 4 extends coordination to the spatial domain by integrating balanced routing with departure-time control. It proposes a scalable LNS metaheuristic for joint spatiotemporal optimization and quantifies the synergies between balancing and staggering. Numerical results show that integrated coordination yields greater network stability than decoupled approaches.

Chapter 5 generalizes the framework to dynamic and stochastic environments. It introduces an optimization-augmented learning pipeline that reformulates the assignment problem as a structured shortest-path task. By implementing forward-looking decisions in real-time, the approach maintains high-quality spatiotemporal coordination under demand uncertainty and evolving traffic conditions.

Chapter 6 synthesizes the main findings, discusses their implications for urban mobility systems, and outlines directions for future research.

2 Literature Review

This chapter reviews the scientific literature underlying the modeling and control of congestion in urban transportation systems, with a particular focus on AMoD operations. The reviewed body of work spans multiple research traditions, including classical traffic assignment and congestion theory, demand-management and incentive mechanisms, and recent advances in optimization-augmented learning-based theory and applications.

To organize this extensive literature, the chapter is structured around three main themes. First, Section 2.1 reviews traffic assignment and congestion modeling, covering user-equilibrium and system-optimal formulations, dynamic traffic assignment, and behavioral and technological mechanisms designed to bridge the efficiency gap between decentralized and coordinated traffic states. These models provide the analytical foundations for traffic-sensitive routing and control in networked transportation systems. Second, Section 2.2 surveys the literature on AMoD systems, organizing existing work by modeling paradigms and decision problems. This section reviews strategic planning and system design, real-time operational control of dispatching, routing, and rebalancing, as well as the incentive-oriented analyses. Third, Section 2.3 surveys the methodological approaches to online routing and motivates combinatorial optimization-augmented machine learning (COaML) as a natural framework for problems that require real-time decisions over large, structured action spaces under uncertainty. Finally, Section 2.4 synthesizes these strands to position the thesis at their intersection. It identifies the development of balanced and staggered routing as a necessary advancement for AMoD systems to act as proactive instruments for congestion management in dynamic transportation networks.

2.1 Traffic assignment and congestion modeling

Traffic assignment has been studied extensively over several decades and constitutes a central research area in transportation science. At its core, it examines how vehicular traffic flows and distributes across a transportation network. Here, we briefly summarize the concepts most relevant to our work and refer the reader to Patriksson (2015), Peeta and Ziliaskopoulos (2001), Sheffi (1985), and Wang et al. (2018) for comprehensive treatments of static and dynamic traffic assignment theory. Section 2.1.1 traces the evolution

from static to dynamic traffic assignment (DTA) and introduces the core components of dynamic models. Focusing on DTA, Section 2.1.2 surveys dynamic network loading formulations, while Section 2.1.3 reviews traveler choice models and behavioral principles. Section 2.1.4 then discusses mechanisms designed to bridge the gap between user-optimal and system-optimal traffic states.

2.1.1 Static and dynamic traffic assignment

Early economic analyses conceptualized congestion as a technological externality arising from the use of shared infrastructure, establishing marginal-cost pricing, whereby users are charged for the incremental delay they impose on the rest of the network, as the efficiency benchmark (Pigou, 1920). In networked transportation systems, these insights led to the formulation of user equilibrium, formalized by Wardrop's first principle, under which no traveler can unilaterally reduce their perceived travel cost by changing routes or departure times (Wardrop, 1952). These foundational contributions gave rise to traffic assignment theory, which studies how vehicular flows distribute over a transportation network in response to travel costs.

Classical static traffic assignment models cast user equilibrium as a fixed-point problem over feasible flow patterns, typically formulated using variational inequalities or complementarity systems. These formulations provide a rigorous analytical framework for establishing the existence and uniqueness of equilibria, as well as for sensitivity analysis, and remain central to congestion pricing and transportation planning applications (Patriksson, 2015; Yang & Meng, 1998). However, static models abstract away from the temporal evolution of traffic and are therefore limited in their ability to represent peak spreading, queue formation, and the transient nature of congestion.

In contrast, DTA models address these limitations by explicitly modeling the temporal propagation of traffic across a network (Peeta & Ziliaskopoulos, 2001). A typical DTA formulation decomposes the system into two coupled components: a *network loading model*, which maps departure rates to time-dependent link states and travel times through a dynamic loading operator, and a *travelers' choice model*, which specifies user actions and the behavioral principles governing them (Wang et al., 2018).

2.1.2 Dynamic network loading models

The loading model describes how traffic flows propagate through the network and how congestion emerges from interactions among vehicles. Common loading operators range from simpler bottleneck models, which capture essential queueing dynamics at capacity-constrained infrastructure (Li et al., 2020; Vickrey, 1969), to more complex cell trans-

mission model (CTM) formulations that preserve key traffic phenomena such as queue spillback and intersection dynamics (Adacher & Tiriolo, 2018; Daganzo, 1994).

A fundamental trade-off between behavioral realism and computational tractability underlies dynamic congestion modeling. While physically grounded loading models based on queueing and kinematic-wave traffic flow are conceptually general, they typically yield non-smooth and non-monotone delay functions that hinder algorithmic convergence and limit scalability in large urban networks (Ziliaskopoulos et al., 2004). To improve tractability in network-scale applications, researchers have therefore proposed macroscopic and mesoscopic approximations. Macroscopic fundamental diagram (MFD) models, for instance, aggregate traffic dynamics at the regional level and offer substantial computational advantages over link-level DTA, albeit at the cost of reduced spatial resolution and potential inconsistencies between discrete and continuum representations (Aghamohammadi & Laval, 2020).

2.1.3 Choice models and behavioral principles

The choice model specifies the set of decision dimensions available to travelers and governs how they respond to experienced or anticipated congestion. The literature commonly classifies DTA models according to the decisions they allow. Among these, three broad categories are particularly relevant here: (i) pure departure-time choice models (e.g., Hendrickson & Kocur, 1981; Lindsey et al., 2012; Vickrey, 1969), (ii) pure route choice models (e.g., Carey & Watling, 2012; Jalota et al., 2023; Levin, 2017; Mounce & Carey, 2011), and (iii) simultaneous route and departure time choice (SRDTC) models (e.g., Han et al., 2013; Huang & Lam, 2002; Jauffred & Bernstein, 1996; Ran et al., 1996; Ziliaskopoulos & Rao, 1999).

Across these categories, traveler behavior is typically governed by one of three guiding principles. Under the dynamic user optimum (DUO) principle, individuals minimize their own perceived travel costs given prevailing congestion (e.g., Friesz et al., 2011; Nie, 2010; Ukkusuri et al., 2012). In contrast, the dynamic system optimum (DSO) principle seeks to minimize total system-wide travel cost by coordinating route and departure-time decisions across travelers (e.g., Carey & Watling, 2012; Long et al., 2018). Between these two extremes lies the *dynamic fleet optimal* principle, which considers centrally coordinated routing for a subset of vehicles—such as a managed fleet—interacting with decentralized traffic (e.g., Battifarano & Qian, 2023; Ke & Qian, 2023). The discrepancy between DUO and DSO thus reflects the classic efficiency loss induced by decentralized decision-making in congested networks.

2.1.4 Bridging user-optimal and system-optimal traffic states

A substantial body of work investigates how to reduce the efficiency gap between equilibrium and system-optimal traffic patterns (Morandi, 2024). A first class of interventions relies on external control and incentive mechanisms that trade off efficiency, fairness, and implementability. Congestion pricing and marginal-cost tolling constitute the most established approaches. By charging travelers for the marginal congestion externalities they impose on others, these schemes internalize external costs and can, under ideal conditions, decentralize the system-optimal solution (Heller et al., 2019). More practically oriented second- and third-best pricing schemes (e.g., facility-based tolls, cordon and zonal pricing, distance-based charges, and dynamic tolling) extend this framework to account for real-world constraints (De Palma & Lindsey, 2011; De Palma et al., 2005; Lindsey & Verhoef, 2001; Stopher, 2004; Yang & Huang, 2005).

Beyond pricing, a range of complementary instruments seeks to improve efficiency by reshaping incentives or constraining feasible choices. Tradable credit schemes regulate network access through market-based mechanisms in which travelers exchange mobility credits, often achieving efficiency gains without net revenue transfers (Miralinaghi & Peeta, 2016, 2019; Shirmohammadi & Yin, 2016; Shirmohammadi et al., 2013; Wang et al., 2012; Yang & Wang, 2011). Stackelberg routing models introduce partial centralized control by assigning routes to a compliant fraction of users while anticipating the selfish response of the remaining traffic, with extensions accounting for tolls and imperfect information (Bhaskar et al., 2014; Bonifaci et al., 2010; Korilis et al., 2002; Krichene et al., 2014; Swamy, 2012). Other regulatory and infrastructural measures include dedicated and managed lanes (Chen et al., 2016; Seilabi et al., 2020; Song et al., 2015), as well as demand management, route guidance, signal control, and access restriction policies that influence user behavior through information provision and operational constraints (Hu & Mahmassani, 1997; Makridis et al., 2024; Menelaou et al., 2021).

Beyond incentive-based or regulatory approaches, a complementary stream focuses on coordinated traffic control enabled by intelligent transportation systems, connectivity, and automation (Papageorgiou & Kotsialos, 2003; Sichitiu & Kihl, 2008; Speranza, 2018). Social and coordinated routing frameworks show that even partial compliance with centrally issued guidance can yield substantial system-level benefits (Djavadian et al., 2014; Lujak et al., 2015; van Essen et al., 2019). In contrast, empirical evidence suggests that uncoordinated real-time navigation tends to converge to inefficient equilibrium patterns (Klein et al., 2018). These limitations motivate a shift from advisory guidance to settings in which a central controller directly determines routing and departure-time decisions. Autonomous and connected vehicles facilitate the coordination of routes and departure times for a managed subset of traffic, allowing system- or fleet-optimal policies to oper-

ate alongside decentralized user behavior (Qian et al., 2021). This perspective naturally extends traffic assignment and coordinated routing principles to the operational control of AMoD systems, where centralized decision-making is intrinsic and can be exploited to actively shape spatiotemporal traffic patterns.

2.2 Modeling, planning, and control of AMoD systems

This section reviews the literature on AMoD systems, organized by modeling foundations and by the main classes of decision problems addressed. Section 2.2.1 summarizes the dominant modeling paradigms used to represent AMoD operations and discusses their respective strengths and limitations. Section 2.2.2 structures the literature by decision time scale and control perspective, covering strategic planning and system design, operational control, and incentive- and policy-oriented analyses.

2.2.1 Modeling foundations for AMoD systems

The literature on AMoD systems has converged around three main modeling paradigms that represent fleet operations at different levels of abstraction: queueing-theoretic models, agent-based simulation, and network flow formulations. Rather than competing approaches, these paradigms are complementary. Each emphasizes a different trade-off between analytical tractability, behavioral and network fidelity, and computational scalability, and each serves a distinct role in the analysis, design, and control of AMoD systems.

A first stream develops analytical representations of AMoD fleets using queueing-theoretic models. By modeling vehicles as circulating in closed networks, these formulations enable compact representations of system dynamics and tractable analysis of stability, throughput, and steady-state performance. Within this framework, the literature has produced fundamental insights into fleet sizing, utilization, spatial flow balance, and rebalancing requirements, and in several cases has enabled the synthesis of optimal or asymptotically optimal control policies derived from steady-state or fluid-limit optimization (Braverman et al., 2019; George, 2012; Iglesias et al., 2019; Pavone et al., 2012; Rossi et al., 2018). These models trade scalability for analytical tractability. They admit rigorous performance guarantees and closed-form insights, making them well-suited for long-run analysis and strategic planning. However, this structure limits their applicability to large, detailed urban networks, as well as their ability to capture transient dynamics and fine-grained operational decisions in congested settings.

A complementary stream relies on agent-based simulation to study AMoD systems under realistic demand, network, and behavioral assumptions. By explicitly modeling individual travelers, vehicles, and infrastructure components as interacting agents, these

approaches capture heterogeneity, network dynamics, and feedback effects that are difficult to represent analytically. Early city-scale simulation studies focused on fleet replacement and sizing, assessing whether shared autonomous fleets could substitute private car travel at acceptable service levels (Bischoff & Maciejewski, 2016; Boesch et al., 2016). Subsequent work expanded the scope to broader system impacts, including congestion, induced demand, and interactions with public transport, while highlighting the strong sensitivity of outcomes to dispatching and rebalancing policies (Basu et al., 2018; Fagnant & Kockelman, 2018; Hörl et al., 2019; Maciejewski & Bischoff, 2018; Oh et al., 2020). This line of research has been supported by the development of general-purpose simulation platforms—such as MATSim (Axhausen et al., 2016), SimMobility (Adnan et al., 2016), and microscopic traffic simulators like SUMO (Krajzewicz, 2010)—as well as dedicated AMoD testbeds like AMoDeus (Ruch et al., 2018). While agent-based simulation plays a central role in feasibility assessment and policy evaluation, it typically relies on exogenously specified control policies and offers limited support for optimization-driven decision making.

A third modeling paradigm adopts network flow approximations to represent AMoD. Vehicle movements are modeled as continuous flows over transportation networks or regions, typically under steady-state or slowly varying conditions. This abstraction scales naturally to city-sized networks and admits efficient optimization, making it well suited for strategic analysis of system-optimal routing and rebalancing under aggregate congestion effects (Levin, 2017; Rossi et al., 2018; Salazar et al., 2020). Extensions incorporate intermodal settings by integrating public transport and background traffic flows (Salazar et al., 2019; Wollenstein-Betech et al., 2022). Flow-based formulations are also widely used in planning-oriented studies, such as fleet sizing and network design, where aggregate balance conditions provide insight into utilization and infrastructure needs (Spieser et al., 2014; Vazifeh et al., 2018).

2.2.2 Problem classes and perspectives in AMoD systems

The problem classes addressed in the AMoD literature span four tightly connected streams that differ primarily by *decision time scale* and *decision levers*: (i) strategic planning and system design, (ii) operational control, (iii) market mechanisms and incentives, and (iv) impact and policy evaluation. Across streams, a consistent message emerges: the welfare promise of AMoD hinges on both *long-horizon capacity and infrastructure choices* and *short-horizon, scalable coordination* that explicitly accounts for congestion, uncertainty, and energy constraints. At the same time, realized system outcomes depend critically on pricing, regulation, and traveler behavior, which mediate the interaction between centrally coordinated fleets and the broader urban mobility ecosystem.

Strategic planning and system design

A first stream studies long-term *strategic planning and system design* problems in AMoD systems, including fleet sizing, infrastructure requirements, and steady-state dynamics.

Fleet sizing and capacity planning constitute well-established topics in this literature (e.g., Cáp & Alonso-Mora, 2018; Fagnant & Kockelman, 2018; Spieser et al., 2014; Vazifeh et al., 2018). Using network-based abstractions, assignment models, and large-scale scenario analyses, these studies consistently show that shared, centrally coordinated AMoD fleets can meet urban mobility demand with substantially fewer vehicles than private car systems. The reductions stem from exploiting temporal and spatial complementarities in trip demand and from eliminating idle time inherent to private vehicle ownership.

A closely related line of work formulates joint planning problems that couple fleet sizing with charging infrastructure siting, charging rates, electricity prices, and grid constraints (e.g., Estandia et al., 2021; Iacobucci et al., 2018; Luke et al., 2021). A central insight is that the cost and feasibility of electric AMoD depend jointly on mobility demand patterns and grid-side operational limits, implying that transportation planning and power-system design cannot be decoupled at long horizons.

Strategic models address congestion and long-run system behavior by coupling AMoD operations with network-level traffic representations. Instead of tracking individual vehicle trajectories, these formulations represent fleet movements as aggregate flows embedded in traffic assignment or equilibrium models, where congestion emerges through volume-delay functions (e.g., US Bur. Public Roads, 1964) or capacity constraints. This abstraction enables the study of service–congestion trade-offs, the structural role of rebalancing flows, and system-optimal operating points at the city scale (e.g., Rossi et al., 2018; Solovey et al., 2019; Wollenstein-Betech et al., 2020). Complementary steady-state and asymptotic analyses rely on queueing, fluid, or mean-field approximations to characterize limiting performance in large-scale systems, yielding bounds and structural insights into stability, utilization, and congestion regimes under idealized coordination (e.g., Braverman et al., 2019; Iglesias et al., 2019).

Operational control of AMoD systems

A second stream studies *operational control* of AMoD systems, focusing on short-horizon coordination of AMoD fleets under constraints such as stochastic demand, limited vehicle availability, and network capacity. Unlike strategic or design-oriented models, this literature confronts the full complexity of real-time decision making, in which vehicle actions must be computed repeatedly, reliably, and at scale.

One class of works focuses on *dispatching and matching*, encompassing vehicle–request assignment as well as acceptance or rejection decisions. A first line of contributions casts

dispatching as a *sequential stochastic control* problem and evaluates reactive or diversion-aware assignment policies, emphasizing robustness and empirical performance across utilization regimes rather than global optimality (e.g., Hyland & Mahmassani, 2018; Li et al., 2019b). A complementary body of work adopts a *control-theoretic perspective*, prioritizing stability, throughput, and delay guarantees over short-horizon performance. In this line, dispatching policies are designed to maximize service rates or ensure bounded waiting times, often yielding analytical characterizations of achievable throughput and minimum fleet sizes under stochastic demand (e.g., Belakaria et al., 2019; Kang & Levin, 2021; Li et al., 2021b). More recent contributions formulate dispatching as an *anticipative optimization or learning problem*, exploiting forecasts, look-ahead structure, or learned value functions. This includes online optimization and incremental re-optimization methods (e.g., Li et al., 2021a; Yu & Shen, 2020), hybrid optimization–learning pipelines that distill high-quality solution structures into fast online policies (e.g., Hu & Dong, 2022; Jungel et al., 2025; Liang et al., 2022), and reinforcement-learning-based approaches that scale dispatching through action-space factorization or multi-agent coordination (e.g., Jindal et al., 2018). Within this class, some works explicitly frame dispatching as a profit-maximization problem with global objectives and rejection decisions (e.g., Enders et al., 2023; Hoppe et al., 2024), while others emphasize scalability through decentralized or weakly coordinated execution across large fleets (e.g., Al-Abbasi et al., 2019).

Beyond dispatching, a substantial body of work studies *rebalancing* as a core operational control problem, focusing on the preemptive relocation of idle vehicles to correct spatial demand imbalances. A large stream formulates rebalancing as a short-horizon optimization problem using time-expanded networks and model predictive control (MPC), enabling forecast-informed control on the city scale (e.g., Aalipour & Khani, 2024; Carron et al., 2021; Iglesias et al., 2018). Building on this core formulation, subsequent work progressively enriches the operational model by incorporating additional constraints and sources of heterogeneity, including limited parking capacity, interactions with ride-sharing, and congestion effects arising from mixed traffic (e.g., Guériau & Dusparic, 2018; Guériau et al., 2020; Ruch et al., 2021; Wallar et al., 2018). As problem dimensions and uncertainty grow, learning-based approaches have emerged as a complementary paradigm, aiming to preserve real-time scalability and robustness while relaxing explicit modeling assumptions and improving transferability across cities and operating regimes (e.g., Gammelli et al., 2021; Tresca et al., 2025; Woywood et al., 2024). A further and closely related extension arises in electrified fleets, where rebalancing decisions must be tightly coupled with charging dynamics and energy uncertainty, motivating energy-aware and robust control formulations that integrate mobility and power-system constraints (e.g., He et al., 2023b, 2023a).

A complementary stream focuses on *vehicle routing*, studying how paths should be selected in real time to improve reliability, mitigate congestion, or influence network-level outcomes. Reliability-oriented routing emphasizes on-time arrival and robustness to travel-time uncertainty using historical or probabilistic traffic information (Liu et al., 2019). Learning-based and traffic-sensitive routing formulations show that controlling autonomous vehicle routes can scale to city-level networks and meaningfully affect system-wide traffic conditions, both in fully autonomous and mixed-autonomy settings (e.g., Han et al., 2016; Lazar et al., 2021). Other works analyze online routing under limited information and capacity constraints, highlighting the limits of greedy policies and proposing data-driven alternatives that exploit historical information (e.g., Jalota et al., 2023).

Finally, a growing literature addresses *integrated routing and rebalancing control*, jointly optimizing customer routes and empty-vehicle repositioning within rolling-horizon frameworks. Typical approaches rely either on data-driven methods or on MPC formulations over time-expanded networks to explicitly capture system dynamics, feasibility, and stability, yielding systematic gains over reactive strategies (e.g., Schmidt et al., 2024; Tsao et al., 2018). This integrated perspective naturally extends to ride-sharing through trip construction and assignment pipelines informed by demand forecasts (e.g., Alonso-Mora et al., 2017; Tsao et al., 2019), as well as to intermodal coordination with public transit (Zgraggen et al., 2019). Electrification further tightens the coupling by introducing battery constraints, charging decisions, and electricity prices, leading to joint routing, rebalancing and charging formulations solved via mixed-integer optimization, MPC, or reinforcement learning (e.g., Iacobucci et al., 2019; Singhal et al., 2024; Turan et al., 2020).

Incentives, markets, and system-level outcomes

A complementary body of work examines AMoD systems through the lens of economic incentives, regulation, and system-level impacts. Although less directly tied to the operational control questions addressed in this dissertation, these contributions form a relevant component of the AMoD literature as they characterize how pricing, market structure, and policy choices shape realized outcomes once control strategies are deployed.

A first subset examines pricing and incentive mechanisms as tools for shaping demand and decentralizing system-level objectives. Fare and toll schemes are analyzed for their ability to trade off market share, waiting times, and operator revenue, or to recover socially optimal outcomes under selfish user behavior, positioning pricing as both a coordination mechanism and a policy lever mediating between operator objectives and traveler responses (e.g., Chen & Kockelman, 2016; Salazar et al., 2020). Beyond pricing, equilibrium and game-theoretic frameworks model strategic interactions among mobility service

providers, users, municipalities, and—particularly in electrified systems—power system operators. These studies characterize equilibrium outcomes, identify regulatory levers, and formalize how policy choices influence adoption, competition with public transit, and welfare, with multimodal and intermodal formulations providing a systematic language to contrast individual optimal outcomes with social objectives and to evaluate incentive-compatible interventions (e.g., Alizadeh et al., 2017; Lanzetti et al., 2023; Zardini et al., 2023).

A further stream evaluates the system-level impacts of AMoD deployment under alternative policy and operational scenarios, typically using activity-based and agent-based simulation frameworks that couple traveler behavior, demand evolution, and network dynamics. This literature quantifies changes in modal split, induced travel, congestion, energy consumption, and emissions under varying assumptions on fleet operations, pricing, and restrictions on private vehicles (e.g., Azevedo et al., 2016; Kamel et al., 2019; Liu et al., 2017; Oh et al., 2021; Oke et al., 2020). Complementary analyses focus on distributional dimensions, examining accessibility outcomes across socioeconomic groups and spatial heterogeneity in benefits (Nahmias-Biran et al., 2021; Zhou et al., 2021), as well as land-use-related impacts such as parking demand and spatial reallocation effects (Zhang & Guhathakurta, 2017). A related subset studies AMoD deployment in conjunction with public transit, using integrated simulation frameworks to assess multimodal interactions and network-level outcomes under alternative integration or competition scenarios (e.g., Basu et al., 2018; Nguyen-Phuoc et al., 2023; Wen et al., 2018).

2.3 Combinatorial optimization–augmented machine learning

The convergence of machine learning (ML) and combinatorial optimization (CO) has emerged as a central development in modern decision science, as documented in recent survey literature (Kotary et al., 2021; Mandi et al., 2024; Mišić & Perakis, 2020; Sadana et al., 2025). While ML extracts patterns from high-dimensional data to predict uncertain parameters, CO provides the algorithmic tools to navigate complex discrete feasible regions and compute high-quality decisions. In practice, this combination typically follows a sequential scheme: predictive models estimate uncertain inputs, and an optimization model computes decisions based on these predictions. In traditional architectures, predictive model training is decoupled from downstream decision-making, leading to misalignment between predictive accuracy and decision quality. Recently, COaML has emerged to address this limitation by embedding combinatorial optimization into end-to-end learning, enabling models to train on decision quality. This paradigm is particularly well suited

to large-scale mobility systems, where operators must repeatedly make routing and dispatching decisions under uncertainty over complex combinatorial action spaces. For a comprehensive overview of this rapidly evolving literature, we refer the reader to Schiffer et al. (2026).

The remainder of this section reviews the COaML developments most relevant to the present work. Section 2.3.1 discusses early hybrid paradigms for integrating optimization and learning in online routing. Section 2.3.2 then presents decision-focused learning and positions COaML as a general framework for aligning training with decision quality. Finally, Section 2.3.3 reviews representative COaML paradigms and applications in routing and mobility, highlighting current limitations in handling spatiotemporal coordination tasks such as balanced and staggered routing.

2.3.1 Toward integrated optimization and learning in online routing

Online routing and mobility decision problems exhibit two defining characteristics: a pronounced *combinatorial structure*, arising from discrete and tightly constrained action spaces, and *multi-stage stochasticity*, driven by uncertain demand arrivals, time-varying travel times, and endogenous system dynamics. Methods for these problems span a spectrum between *pure CO* frameworks and *fully data-driven* approaches. At one end, traditional CO approaches rely on carefully engineered optimization models that repeatedly solve large subproblems under tight time budgets (e.g., Bertsimas et al., 2019). While these methods offer strong interpretability and rigorous constraint satisfaction, their online deployment often incurs substantial computational overhead, particularly in high-frequency decision-making settings. Moreover, anticipating future events typically requires sampling-based lookahead or extensive scenario enumeration, which further increases the computational burden. At the opposite end of the spectrum, purely data-driven approaches pursue anticipative control by learning decision policies from interaction or simulation. In particular, reinforcement learning (RL) optimizes long-run performance and enables constant-time inference at deployment (e.g., Joe & Lau, 2020). However, traditional RL struggles to efficiently explore large, combinatorial, and highly constrained action spaces, where feasible routing decisions must satisfy complex structural and capacity constraints, posing severe scalability challenges (Hildebrandt et al., 2023).

In response to these limitations, the *predict–then–optimize* paradigm emerged as an early attempt to bridge statistical learning and discrete optimization. In this approach, statistical models estimate uncertain parameters, such as demand realizations or travel times, and an optimization model computes decisions based on these predictions (e.g., Alonso-Mora et al., 2017; Bertsimas & Kallus, 2020). The appeal of this paradigm lies in

its practicality: converting uncertainty into point predictions enables the use of mature combinatorial solvers without modifying the underlying optimization models. However, training remains decoupled from the downstream decision, introducing a fundamental misalignment between prediction error and decision error. Specifically, predictive models minimize symmetric statistical losses, while optimization evaluates asymmetric operational costs, a mismatch that is amplified in constrained systems where over- and underestimation carry markedly different consequences. For example, underestimating a resource may lead to minor inefficiencies, while overestimating it can violate capacity constraints and trigger costly recourse actions. These limitations motivate learning approaches that directly account for the downstream decision problem during training.

2.3.2 Decision-focused learning

The disconnect between statistical accuracy and operational utility has given rise to *decision-focused learning*, whose overarching goal is to align training objectives with downstream decision quality (Kotary et al., 2021). This approach inherits the core philosophy of *structured output prediction*, a paradigm developed to predict combinations of interdependent variables forming globally consistent combinatorial objects (Nowozin, Lampert, et al., 2011). Rather than minimizing isolated prediction errors, these methods directly optimize the quality of the resulting decision, ensuring that learning prioritizes operational performance over purely statistical accuracy. A prominent realization of this philosophy is the *smart predict–then–optimize (SPO)* framework (Elmachtoub & Grigas, 2022), which embeds the optimization problem into the learning process via the *SPO+* loss, a convex surrogate tailored to decision quality. By leveraging convex upper bounds on the true decision error together with subgradient information, SPO enables models to directly minimize downstream regret. However, its applicability relies on a linear relationship between the uncertain parameters and the optimization objective, limiting its use in many real-world settings.

In this context, COaML extends this paradigm to environments where the structural assumptions underlying SPO no longer hold. Rather than relying on a decoupled loss formulation, COaML integrates the combinatorial optimization problem as a layer within the statistical model, enabling end-to-end training via backpropagation through the solver (Dalle et al., 2022; Parmentier, 2021). In this setting, the model parameterizes the combinatorial problem by producing cost vectors in a latent space calibrated to yield high-quality decisions. This integration, however, introduces a fundamental analytical challenge: the mapping from parameters to discrete solutions is piecewise constant, rendering gradients zero or undefined almost everywhere. To enable end-to-end backpropagation, COaML methods rely on *surrogate differentiation* and *stochastic perturbations* (Berthet

et al., 2020), which smooth the discrete mapping and enable estimating how changes in the input parameters affect the resulting combinatorial solution.

2.3.3 COaML paradigms and applications

The COaML literature encompasses several learning paradigms that differ in the type of signal used to train the model. One line of work adopts an *empirical cost minimization* perspective, where models directly optimize observed operational costs from sampled instances (Aubin-Frankowski et al., 2025; Bouvier et al., 2025). While this formulation provides strong generalization guarantees, the piecewise-constant nature of combinatorial costs makes gradient estimation challenging, often requiring regularized surrogates or score-based estimators to control gradient variance.

A second paradigm follows an *imitation learning* approach. Here, models learn to reproduce expert decisions rather than directly minimizing operational costs. Techniques based on *Fenchel–Young losses* enable efficient training by differentiating through structured prediction layers (Blondel et al., 2020). These methods avoid the high-variance gradients associated with cost-based objectives, but their performance ultimately depends on the availability and quality of expert labels.

A third class of approaches integrates RL with combinatorial optimization. In this setting, CO layers appear within neural policy architectures to enforce structural constraints during exploration and inference. This design allows reinforcement learning agents to operate in combinatorial decision spaces while respecting feasibility conditions, at the expense of requiring repeated solver calls during training (Hoppe et al., 2025).

The flexibility of COaML has enabled applications across a wide range of mobility and logistics problems, spanning increasing levels of dynamism and uncertainty. In static settings, representative applications include vehicle scheduling (Parmentier, 2021), the fixed-charge transportation problem (Spieckermann et al., 2025), and traffic equilibrium prediction (Jungel et al., 2024). In contextual stochastic settings, predictive models coordinate immediate routing decisions with anticipated future costs, with applications such as inventory routing (Greif et al., 2024) and power scheduling under uncertainty (Kong et al., 2022). In fully dynamic, high-frequency environments, embedded CO layers enable complex, constrained operations such as bipartite matching for dispatching, continuous fleet rebalancing, and large-scale neighborhood search for routing, with applications in AMoD systems, emergency medical services, and multi-stage vehicle routing (Baty et al., 2024; Jungel et al., 2025; Rautenstrauss & Schiffer, 2025).

These developments highlight the strong potential of COaML for spatiotemporal coordination in AMoD systems, where fleets optimize expected travel times under congestion on shared, capacity-constrained networks with uncertain demand. These problems ex-

hibit a pronounced combinatorial structure and require strict adherence to physical and temporal constraints, making COaML particularly well suited to this class of problems. However, applying COaML to this setting remains largely unexplored and requires non-trivial, problem-specific architectural adaptations. Addressing this gap enables scalable, forward-looking spatiotemporal control, with the potential to substantially mitigate congestion in complex transportation systems.

2.4 Synthesis and positioning of the thesis

The literature reviewed in this chapter encompasses traffic assignment theory, AMoD system control, and learning-augmented optimization. While these fields vary in their proximity to the core of this work, they collectively delineate the conceptual and methodological landscape in which this thesis is positioned.

Traffic assignment and congestion modeling provide the analytical foundation for routing under network effects. In particular, user-equilibrium and system-optimal formulations formalize the efficiency gaps arising from uncoordinated infrastructure use. This thesis builds on these principles as the underlying congestion framework and shifts the focus to the operational challenge of computing coordinated routing and departure-time decisions for a centrally managed fleet.

AMoD literature constitutes the primary application domain of this thesis. While prior research has demonstrated that centralized control can reduce empty mileage and improve vehicle utilization, many operational models rely on simplified traffic representations. Explicit demand management through joint routing and departure-time control remains underexplored. In particular, mechanisms that deliberately regulate *where* vehicles travel and *when* they enter the network are largely absent from prevailing AMoD control paradigms. This thesis addresses this gap by proposing a spatiotemporal coordination framework for congestion management in AMoD systems.

Online AMoD routing features high-dimensional action spaces, strict latency requirements, and inherent stochasticity, rendering purely optimization-based or purely learning-based approaches insufficient. The COaML literature provides a methodological foundation for embedding combinatorial structure within trainable decision pipelines, enabling fast online inference while preserving routing feasibility and capacity constraints. Extending these frameworks to spatiotemporal fleet coordination constitutes a novel application of the COaML paradigm.

In summary, this thesis contributes to the literature along three complementary dimensions. First, it translates congestion-theoretic principles into implementable fleet-level control policies. Second, it elevates spatiotemporal coordination—through joint routing

and departure-time control—to a central operational objective in AMoD systems. Third, it extends the application of COaML to predictive congestion modeling for jointly optimizing routing and departure-time decisions in AMoD systems.

3 Staggered Routing in AMoD Systems

3.1 Introduction

Urban congestion has become an urgent challenge for modern cities, with significant economic, environmental, and public health consequences. In 2022, the average U.S. driver spent 51 hours stuck in traffic, resulting in an economic loss of $81 billion (Pishue, 2023). Beyond financial costs, congestion exacerbates air pollution and greenhouse gas emissions, contributing to climate change and adverse health effects (Levy et al., 2010). A major contributor to worsening congestion is the expansion of ride-hailing services, which, rather than alleviating congestion, have been shown to displace public transportation trips and increase overall vehicle miles traveled (Agarwal et al., 2023; Tirachini & Gomez-Lobo, 2020).

Emerging mobility paradigms such as AMoD have been proposed as potential solutions to mitigate these externalities. AMoD systems consist of centrally coordinated fleets of self-driving vehicles providing ride-hailing services (Pavone, 2015). By leveraging real-time system-wide state observation, centralized dispatching, and rebalancing algorithms, AMoD systems have the potential to improve transport efficiency compared to private car travel or uncoordinated ride-hailing services. However, these benefits come with trade-offs, as AMoD services also risk exacerbating peak-hour congestion if not carefully managed. Despite their autonomous capabilities, AMoD systems fundamentally share the same operational characteristics as conventional ride-hailing fleets: they generate additional vehicle miles traveled, induce modal shifts away from public transit, and contribute to traffic demand surges during peak hours. As such, while AMoD systems offer enhanced control and efficiency mechanisms, their core impact on urban congestion dynamics remains closely aligned with that of traditional ride-hailing services.

To address these challenges, researchers and policymakers have explored various strategies to mitigate the congestion impact of ride-hailing and AMoD systems. While AMoD services offer the potential for improved fleet control, their effectiveness in reducing conges-

---

This chapter builds on a paper co-authored with Gerhard Hiermann, Dario Paccagnan, and Maximilian Schiffer, published in the *European Journal of Operational Research*, 327 (2025), 875–891. The paper was presented at the *EURO 2022 Conference on Operational Research* (Espoo, Finland) and the *INFORMS Transportation Science and Logistics Conference 2023* (Chicago, USA).

tion depends on broader traffic dynamics and modal shifts. Accordingly, recent research highlights the importance of integrating AMoD services with public transportation and promoting ride-pooling as primary congestion mitigation strategies (Ruch et al., 2020; Salazar et al., 2020). In addition to these measures, congestion pricing and demand-responsive fleet management have emerged as critical tools. Congestion pricing leverages economic incentives to redistribute traffic across time and space by charging fees for road use during peak hours (Arnott et al., 1990b; Roughgarden, 2005), while demand-responsive fleet management seeks to optimize vehicle routing and dispatching to reduce system-wide inefficiencies (Jahn et al., 2005; Patriksson, 2015). In this context, fleet management and routing decisions taken by an (AMoD) operator must not necessarily align with a municipality system perspective that aims to minimize overall congestion beyond the fleet's operations. While optimizing a fleet's operations with respect to congestion effects firstly improves the customer's experience and convenience it can further allow to mitigate local congestion bottlenecks. In fact, targeted interventions at the fleet level can contribute to reducing congestion at the system level, even if only applied to a subset of the traffic network (Battifarano & Qian, 2023).

Despite extensive research into congestion pricing and fleet optimization, many existing methods predominantly focus on distributing traffic across space rather than time (Jalota et al., 2023). While some demand management strategies incorporate departure delays to alleviate peak-load congestion, their application to individual fleet operators remains underexplored. AMoD systems, with their centralized control, offer an opportunity to implement time-based congestion mitigation strategies at a finer level of granularity.

Against these backdrops, we introduce the concept of *staggered routing*, where an AMoD operator delays the departure of trips over time to reduce local congestion bottlenecks while ensuring all trips' drop-off deadlines are met. Specifically, we take a first step to unravel the *potential* of staggered routing in AMoD systems. This approach complements existing congestion mitigation efforts by leveraging AMoD-specific control capabilities to improve system efficiency while reducing the fleet's negative externalities.

3.1.1 Contribution

Our work aims to inform on the impact of staggering trip departures in reducing congestion. Toward this goal, we present three key contributions. First, we introduce the problem of *staggered routing*, i.e., the problem of staggering the fleet departure times so as to minimize the resulting congestion while meeting drop-off deadlines. We then formulate this problem as a mixed integer linear program (MILP) and show that it is NP-hard. Second, we provide an effective algorithmic framework to solve the staggered routing problem at scale. Specifically, we i) show how a large number of the resulting big-M constraints

can be dropped without altering the optimal solutions, and ii) develop a matheuristic that allows us to solve large-scale instances. This matheuristic bases on a construction heuristic, a problem-specific local search for intensification, while relying on our MILP to escape local optima and verify optimality. Finally, we apply our methodology to a real-world case study for the Manhattan area in New York City. Specifically, we study the impact that staggered routing has in both an offline and an online decision-making setting.

Our results show that in low-congestion (LC) scenarios, i.e., scenarios in which only the AMoD fleet induces congestion but the system without the fleet would remain uncongested, staggering trip departures allow to mitigate on average 98% of the induced congestion in a full-information setting. While not entirely reaching this upper bound, our rolling horizon approach still allows us to reduce 82% of the induced congestion in the online LC scenario. In high-congestion (HC) scenarios, i.e., scenarios in which the system already encounters exogenous congestion and the AMoD fleet induces further congestion, we observe an average congestion reduction of 60% and 30% in the full-information and rolling-horizon settings. Surprisingly, we show that these reductions can be reached by shifting trip departures of a maximum of two minutes in the LC scenario and by six minutes in the HC scenario while meeting each trip's arrival time.

3.1.2 Technical challenges and our approach

Solving the staggered routing problem at scale requires tackling at least three intertwined challenges.

First, following the commonly employed approach whereby one keeps track of the congestion level on each arc at predetermined time intervals (see, e.g., time-expanded construction in Köhler et al. (2002)) is simply not possible as the size of the resulting optimization problem grows too quickly, making the solution of even moderately sized instances beyond reach. We tackle this challenge by taking a so-called Lagrangian perspective, that is, we "follow" each trip along its path and only keep track of the congestion this trip encounters at the time when it enters each arc on its path. Unlike the Eulerian view, which focuses on the congestion at a fixed location in the network and considers the total flow passing through that location over time, the Lagrangian perspective tracks individual trips as they traverse the network (Bennett, 2006). This shift allows us to formalize an optimization problem that is significantly more compact.

Second, while the Lagrangian perspective we adopt allows us to reduce the problem dimensionality, it makes it more difficult to determine the congestion level each trip encounters along the traveled arcs, leading to a set of non-linear constraints. Although easily handled through big-M reformulations, the number of such constraints grows quickly such

that the resulting continuous relaxation performs poorly for large problem instances. We resolve this challenge by showing how to significantly reduce their number without affecting optimality. We do so by carefully reformulating the problem over a multigraph.

Third, the resulting MILP can still not be solved at scale by state-of-the-art solvers. We tackle this challenge by developing a local-search algorithm that we embed within the branch and bound tree generated when solving the MILP. Specifically, whenever we find a new incumbent by solving the MILP, we stop and improve this solution through our local search before feeding it back to warmstart the MILP. While our local search follows a greedy approach based on delay severity, its development requires careful considerations as efficiently evaluating the impact that modifying one departure time has on all other trips is nontrivial.

Complimentary to this, scaling our solution to complex real-world networks introduces additional computational challenges due to the large number of road segments involved. By utilizing contraction hierarchies (cf. Geisberger et al., 2008), we reduce the network's complexity while preserving shortest path lengths. This strategy significantly improves the model's applicability and computational efficiency, making it suitable for studying staggered routing in real-world scenarios.

3.1.3 Related work

Our work connects with various streams of literature for the optimization of AMoD systems, an area that has recently attracted significant attention. Given the sheer volume of literature in this space, providing a comprehensive overview is beyond the scope of this work, and we refer the interested reader to Zardini et al. (2022) and Narayanan et al. (2020).

While our focus is on operational fleet control, we briefly reference strategic models for completeness. These studies typically use time-invariant network flow formulations to assess the broader impact of AMoD trips on traffic (see, e.g., Rossi et al., 2018; Salazar et al., 2020) and guide long-term decisions such as fleet sizing and infrastructure planning. However, since these models operate in steady-state regimes, they do not account for dynamic routing decisions, which are central to our study.

Existing *operational* control approaches are instead based on fine-grained models that do incorporate a time dimension as they aim at devising an online control policy. Several works have been published in this domain, focusing on planning tasks such as vehicle-to-request dispatching, vehicle rebalancing, and request pooling. In this context, various methodological approaches have been studied, ranging from model predictive control for vehicle dispatching (see, e.g., Iglesias et al., 2018; Liu & Samaranayake, 2022; Tsao et al., 2018) and request pooling (see, e.g., Alonso-Mora et al., 2017), to deep reinforcement

learning for rebalancing (see, e.g., Gammelli et al., 2021; Jiao et al., 2021; Liang et al., 2022; Skordilis et al., 2021) and dispatching (see, e.g., Enders et al., 2023; Li et al., 2019a; Sadeghi Eshkevari et al., 2022; Tang et al., 2019; Xu et al., 2018; Zhou et al., 2019), to optimization-augmented learning pipelines (see, e.g., Jungel et al., 2025; Zhang et al., 2017). However, these approaches focus on optimizing the routing decisions over space but do not focus on the possibility of staggering the departures over time, which is the focus of this work. Surprisingly, even studies that explicitly model congestion are scarce as existing research often bases travel times on precise forecasts or deterministic assumptions.

Delaying trip departures as a demand management strategy has been extensively studied to optimize vehicle travel times within street networks. Traditional methods, such as dynamic traffic assignment and route guidance schemes, have utilized complex microscopic models to address congestion, but these become cumbersome in large-scale applications (Chabini, 1998; Daganzo, 2007). More recent approaches have leveraged aggregated traffic models like the MFD, which are favored for their scalability (Ramezani et al., 2015). Applications include maximizing trip completion rate across all regions of a multi-regional traffic network and addressing on-time arrivals to minimize the discrepancy between drivers' desired and actual arrival time (Menelaou et al., 2024; Menelaou et al., 2021). Aggregated accumulation-based approaches have also been coupled with more granular trip-based MFD models that describe individual trip details post-departure time adjustments (Kumarage et al., 2021; Yildirimoglu & Ramezani, 2020). However, these models typically operate at a *system-wide* level. In contrast, a fleet-based approach to managing traffic with staggered time policies has not been explored due to the lack of control of individual vehicle flows prior to the advent of AMoD systems, which we aim to pioneer in this work.

Finally, the MILP framework with arc-based time windows developed in this work shares similar foundational principles with both the core times notion in resource-constrained project scheduling (RCPSP) and the constraint propagation principles used in constraint programming (CP). In RCPSP, core times refer to intervals during which each activity must be active in any valid schedule, reflecting the earliest and latest times activities can start or finish under resource and precedence constraints (Bartusch et al., 1988; Neumann et al., 2002). Similarly, in CP-based scheduling, constraints on activities' time windows are repeatedly propagated and tightened to prune infeasible solutions and reduce the search space (Baptiste et al., 2001). In our approach, we calculate arc-based time windows for each trip, comprising earliest start, latest start, earliest finish, and latest finish. These time windows fulfill the same role of "locking in" partial schedules by revealing infeasible overlaps and ensuring resource-feasible allocations.

3.1.4 Roadmap

The remainder of this paper is structured as follows. Section 3.2 outlines the problem setting for the staggered routing problem. Section 3.3 details our matheuristic developed for solving the staggered routing problem. In Section 3.4, we provide an overview of our experimental design, while in Section 3.5, we present the results obtained from our numerical study. Finally, we conclude in Section 3.6 with some remarks and future research directions.

3.2 Problem statement

We begin by considering an offline problem wherein an AMoD operator coordinates an autonomous vehicle fleet to serve a given set of trips, e.g., trips arising within a day or a specific time window within a city. Each trip is operated by a single vehicle and can correspond to both on-duty, i.e., customer delivery, or off-duty, i.e., rebalancing activities, of the vehicle. Importantly, vehicles operate trips along a *prespecified* route, e.g., the shortest, within the road network. Within this context, we focus on enhancing operational efficiency *after* vehicle routing decisions are taken. That is, we assume that the origin, destination, and route for each trip (equivalently, vehicle) are fixed. To do so, we aim to stagger trip departures, i.e., postpone trip departure times beyond their originally requested times, to reduce congestion caused by route overflows and thus minimize the fleet's travel time.

We model the road network within which the fleet operates as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$. The nodes $\mathcal{V}$ represent trip origins, destinations, and road intersections, while the arcs $\mathcal{A}$ are the road segments connecting these nodes. Given a set of trips $\mathcal{R}$, let a quadruple $(p^r, e^r, \ell^r, \bar{\sigma}^r)$ define a trip $r \in \mathcal{R}$, with p^r denoting the sequence of arcs composing the trip's fixed route through the network, e^r being the earliest departure from the trip's origin, ℓ^r being the latest acceptable arrival at the destination, and $\bar{\sigma}^r$ being the maximum staggering time applicable to trip departure.

A trip r on arc a incurs a travel time $\tau_a^r(f_a^r)$ that is the sum of a free-flow travel time τ_a and a congestion-induced delay, which we determine by the number of AMoD trips f_a^r on arc a when trip r starts traversing the arc.[1] We model such a congestion-induced delay as a piece-wise linear, non-decreasing, and convex function, resulting in a travel time over each arc such as that represented in Figure 3.1. Without loss of generality, the total travel

[1]Note that our formulation easily allows to consider the background presence of non-AMoD traffic, see Section 3.4 for details.

time encountered by trip r on arc a can thus be written as

$$\tau_a^r(f_a^r) = \tau_a + \max_{k \in K} \left\{ 0, \sum_{1 \leq k' \leq k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'}) \right\}, \tag{3.1}$$

where $\tau_a > 0$, $\mu_a^k > 0$ and $\hat{f}_a^k > 0$.

Remark 1 *Two comments are in order. First, while other congestion models exist, e.g., simpler capacity models (Estandia et al., 2021) or more accurate microscopic models (Levin et al., 2017), we believe that our approach to modeling congestion provides a good balance between accuracy and computational complexity that is appropriate for the mesoscopic nature of our study. In this respect, it is important to remark that, while in mesoscopic studies, the relation between travel time and congestion is often taken to be a fourth-order polynomial (see, e.g., US Bur. Public Roads, 1964), our approach not only approximates such a function in a computationally convenient way, but it also allows to accommodate other, potentially different, relations. Second, when detecting the congestion level for a vehicle entering the arc, we ignore whether the other vehicles have just begun traversing the arc or are nearing completion. One possible way to mitigate this overestimation is to subdivide an arc into multiple segments, allowing for more accurate trip interactions. While this reduces aggregation errors, it also increases the computational burden, necessitating a balance between accuracy and tractability. We believe that the resulting trade-off between slightly overestimating congestion and deriving a tractable model is reasonable for the scope of our study.*

In this setting, a solution π is a vector with a departure time for each trip r. As we assume no vehicle idles once its trip has started, this information suffices to determine both the time as well as the number of vehicles f_a^r encountered by every vehicle when entering arc a of its route p^r. In turn, this sequence determines the travel times over all arcs and, thus, the trips' arrival times.

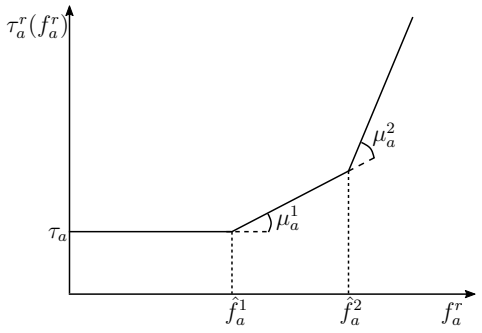


Figure 3.1: Travel time of trip r on arc a as a function of the number of AMoD trips f_a^r.

We require a solution π to satisfy the following constraints, which we collect in the feasible set Π:

i) The departure time of each trip r must occur after e^r.
ii) The departure time of each trip r can only be staggered for a maximum amount of time $\bar{\sigma}^r$.
iii) The departures in π must ensure that the arrival at the destination of each trip r occurs before ℓ^r.

In this work, we seek a solution $\pi^\star \in \Pi$ that minimizes the total fleet travel time, i.e., the sum of the travel times each trip incurs over all arcs of its path

$$\min_{\pi \in \Pi} T(\pi) = \sum_{r \in \mathcal{R}} \sum_{a \in p^r} \tau_a^r(\pi), \tag{3.2}$$

where, with slight abuse of notation, we explicitly denote the fact that the travel time τ_a^r encountered on arc a by trip r depends on the entire vector of departure times.

Finally, while (3.2) describes the offline version of the problem we will consider, one can easily transform this offline setting into an online problem by discretizing time and taking decisions on batches of trips that arrive in the system according to a point process. In our numerical studies, we will comment on the difference between the maximum potential of staggering identified by solving the offline problem and the improvement attained by solving its online counterpart.

Remark 2 *The dynamic traffic assignment literature offers various congestion models, such as the cell-transmission model, which captures complex phenomena like spillbacks (Adacher & Tiriolo, 2018). However, these models become computationally intractable for large-scale problems (Ziliaskopoulos et al., 2004). To ensure tractability and enable a MILP formulation (see Section 3.3.1), we use a piecewise linear delay function, a widely used approximation of nonlinear volume-delay functions in static traffic assignment (see, e.g., Bang & Malikopoulos, 2022). Note that our congestion model is akin to Vickrey's bottleneck model (Vickrey, 1969) in its discretized version (Otsubo & Rapoport, 2008), where n vehicles attempt to traverse an arc at distinct departure times t, with the traversal being characterized by a free-flow travel time and a capacity threshold. Once this threshold is surpassed, a bottleneck arises, modeled as a queue forms at the arc's entrance, regulated by a first-in-first-out (FIFO) policy, with vehicles leaving the queue as others complete their traversal of the bottleneck. In our model, the travel time of an arc, as defined in* (3.1)*, consists of a free flow travel time τ_a and a delay akin to the waiting time in a vertical queue. The waiting time increases when the vehicle count on the link exceeds $\hat{f}_a$. Unlike in Vickrey's model, the waiting time in our model depends on the number of vehicles concurrently on the arc and in the queue at t, and is calculated based on a*

function similar to that shown in Figure 3.1. It's worth mentioning that as a technical difference, our model does not enforce FIFO, as it allows shorter waiting times for later departures. Accordingly, it remains an approximation of the queuing dynamic in Vickrey's model that may show local deviations for single vehicles but sums up to a comparable total travel time.

3.3 Methodology

The staggered routing problem as introduced in Section 3.2 is NP-hard, which we show in Appendix A3.1. This motivates us to develop an efficient matheuristic that allows to solve large-scale instances in the following. First, we introduce a MILP that allows us to obtain solutions for Problem (3.2) in Section 3.3.1. Towards this goal, we introduce a basic model that uses an indicator function to quantify the aggregated flow, before we elaborate on how to model the respective indicator constraints. Second, we show how to tighten the formulation on the MILP by leveraging the problem structure in Section 3.3.2. Third, we present a matheuristic that combines our MILP with a local search scheme to solve large real-world instances in Section 3.3.3. Finally, Section 3.3.4 adapts the previously developed algorithm to solve the online counterpart of our problem in a rolling-horizon fashion.

3.3.1 Mixed integer linear program

We aim to formulate (3.2) as a MILP. Towards this goal, note that minimizing the total travel time is equivalent to minimizing the congestion-induced delay, as we have no control over the arcs' free-flow travel times τ_a. The objective therefore reads

$$\min \sum_{r\in\mathcal{R}} \sum_{a\in p^r} \max_{k\in K} \left\{ 0, \sum_{1\leq k'\leq k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'}) \right\}.$$

With this in mind, we obtain a linear representation of this non-linear objective using an epigraphic reformulation (Boyd & Vandenberghe, 2004). Specifically, we introduce delay variables x_a^r and enforce the constraint $x_a^r \geq \sum_{k'=1}^{k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'})$ for all arcs, trips, and $k \in K$. To represent the entry time of trip r on arc a on its path, we introduce the continuous variable s_a^r. Further, we let $\delta^r(a)$ denote the successor arc of arc a on route p^r, and $\underline{a}_r$, $\bar{a}_r$ denote the first and last arcs of trip r, respectively. Finally, we let $\mathcal{R}_a$ denote

the set of all trips that transit through arc a. Then, problem (3.2) can be written as:

$$\min \sum_{r\in\mathcal{R}} \sum_{a\in p^r} x_a^r, \tag{3.3a}$$

s.t.

$$x_a^r \geq \textstyle\sum_{k'=1}^{k} \mu_a^{k'} \cdot \left(f_a^r - \hat{f}_a^{k'}\right), \qquad \forall k \in K, r \in \mathcal{R}, a \in p^r \tag{3.3b}$$

$$s^r_{\delta^r(a)} = s_a^r + \tau_a + x_a^r, \qquad \forall r \in \mathcal{R}, a \in p^r \setminus \bar{a}_r \tag{3.3c}$$

$$f_a^r = \textstyle\sum_{r'\in\mathcal{R}_a\setminus\{r\}} \mathbb{I}(s_a^{r'} \leq s_a^r < s_a^{r'} + \tau_a + x_a^{r'}), \qquad \forall r \in \mathcal{R}, a \in p^r \tag{3.3d}$$

$$e^r \leq s^r_{\underline{a}_r} \leq e^r + \bar{\sigma}^r, \qquad \forall r \in \mathcal{R} \tag{3.3e}$$

$$s^r_{\bar{a}_r} + \tau_{\bar{a}_r} + x^r_{\bar{a}_r} \leq \ell^r, \qquad \forall r \in \mathcal{R} \tag{3.3f}$$

$$x_a^r, s_a^r, f_a^r \geq 0. \qquad \forall r \in \mathcal{R}, a \in p^r \tag{3.3g}$$

Our objective (3.3a) jointly with Constraints (3.3b) minimizes the total delay over all trips and arcs. Constraints (3.3c) propagate forward the time at which trip r enters arc a along its route p^r, while Constraints (3.3d) calculate the number of vehicles encountered by each trip r on arc a. This is achieved using an indicator function $\mathbb{I}(\cdot)$, which is set to one if trip r enters arc a when r' is traversing the same arc. Constraints (3.3e)-(3.3f) ensure feasible start and drop-off times. Constraints (3.3g) state the variable domains.

To convert the latter model into a MILP, we replace the indicator function in Constraints (3.3d) with big-M constraints that replicate its logic. To do so, we introduce binary variables $\alpha_a^{r,r'}, \beta_a^{r,r'}$ and $\gamma_a^{r,r'}$ for each pair of distinct trips (r, r') traversing the same arc, whose logic is as follows:

$$\alpha_a^{r,r'} = \begin{cases} 1, \text{ if } s_a^r \geq s_a^{r'} \\ 0, \text{ otherwise} \end{cases} \qquad \beta_a^{r,r'} = \begin{cases} 1, \text{ if } s_a^r < s_a^{r'} + \tau_a + x_a^{r'} \\ 0, \text{ otherwise} \end{cases} \qquad \gamma_a^{r,r'} = \begin{cases} 1, \text{ if } \alpha_a^{r,r'} + \beta_a^{r,r'} = 2 \\ 0, \text{ otherwise} \end{cases}$$

We enforce this logic and therefore evaluate f_a^r by replacing constraints (3.3d) with the following:

$$s_a^r - s_a^{r'} + \varepsilon \leq M_1 \cdot \alpha_a^{r,r'}, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.3h}$$

$$s_a^{r'} - s_a^r \leq M_2 \cdot (1 - \alpha_a^{r,r'}), \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.3i}$$

$$s_a^{r'} + \tau_a + x_a^{r'} - s_a^r \leq M_3 \cdot \beta_a^{r,r'}, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.3j}$$

$$s_a^r - s_a^{r'} - \tau_a - x_a^{r'} + \varepsilon \leq M_4 \cdot (1 - \beta_a^{r,r'}), \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.3k}$$

$$\gamma_a^{r,r'} - \alpha_a^{r,r'} - \beta_a^{r,r'} \geq -1, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.3l}$$

$$2\gamma_a^{r,r'} - \alpha_a^{r,r'} - \beta_a^{r,r'} \leq 0, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.3m}$$

$$f_a^r = \textstyle\sum_{r'\in\mathcal{R}_a\setminus\{r\}} \gamma_a^{r,r'} \qquad \forall a \in \mathcal{A}, r \in \mathcal{R}_a \tag{3.3n}$$

where ε is a small constant and M_i are sufficiently large numbers.[2] Constraints (3.3h) ensure that $\alpha_a^{r,r'}$ takes the value one if $s_a^r \geq s_a^{r'}$, while Constraints (3.3i) ensure that it equals zero otherwise. Similarly, Constraints (3.3j)-(3.3k) guarantee the correct behavior of $\beta_a^{r,r'}$. Constraints (3.3l) ensure that $\gamma_a^{r,r'}$ equals one when both $\alpha_a^{r,r'}$ and $\beta_a^{r,r'}$ are one, while Constraints (3.3m) enforce that it equals zero otherwise. Finally, Constraints (3.3n) computes f_a^r as the count of $\gamma_a^{r,r'}$ variables equal to one.

To minimize the M_i constants in the big-M constraints, we derive the following relationships from Constraints (3.3h) − (3.3k):

$$
\begin{aligned}
&M_1 \geq \max(s_a^r - s_a^{r'} + \varepsilon) = \bar{e}_a^r - \underline{e}_a^{r'} + \varepsilon, && \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \\
&M_2 \geq \max(s_a^{r'} - s_a^r) = \bar{e}_a^{r'} - \underline{e}_a^r, && \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \\
&M_3 \geq \max(s_a^{r'} + \tau_a + x_a^{r'} - s_a^r) = \bar{\ell}_a^{r'} - \underline{e}_a^r, && \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \\
&M_4 \geq \max(s_a^r + \tau_a + x_a^r - s_a^{r'} + \varepsilon) = \bar{\ell}_a^r - \underline{e}_a^{r'} + \varepsilon. && \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r'
\end{aligned}
$$

Here, the variables $\underline{e}_a^r$, $\bar{e}_a^r$, and $\bar{\ell}_a^r$ define trip arc time windows, representing, respectively, the earliest departure time, latest departure time, and latest traversal completion time for trip r on arc a. In Section 3.3.2, we present an algorithm to compute tight arc time windows, which can be directly utilized to compute the big-M values.

3.3.2 Constraint reduction and tightening

Although the formulation in Section 3.3.1 is a MILP, it poses significant computational challenges due to the number of big-M constraints required to check if a given pair of vehicles travels simultaneously on the same arc. Specifically, each constraint in (3.3h)-(3.3m) needs to be repeated for all pairs of trips that transit through each given arc. This formulation results, unfortunately, in prohibitive solution times, even for modest problem sizes, as the continuous relaxation used in the solution of the MILP performs poorly. To enhance computational efficiency, we leverage the problem structure to identify and remove as many redundant big-M constraints as possible in the following.

A key observation is that not every distinct pair of trips traversing the same arc may influence each other's travel time, largely because each trip is constrained by specific time windows. However, the original problem formulation introduced in Section 3.2 only defines these windows for each entire trip, not for individual arcs. Based on these trip-specific time windows, our approach constructs arc-specific time windows to identify trips that, by traversing the arc simultaneously, may incur a delay. We then leverage these

[2]Note that the use of a small constant ε is necessary. Indeed, in its absence, given a pair of distinct trips (r, r') jointly traversing arc a, for s_a^r being equal to $s_a^{r'}$, setting $\alpha_a^{r,r'}$ to either zero or one would satisfy Constraints (3.3h)-(3.3i).

bounds to build a representation of the problem on a multidigraph, in which parallel arcs represent identical road segments, but each arc is associated with either a set of potentially conflicting trips, i.e., trips that may influence each other's travel time and thus contain conflicting arcs, i.e., arcs on which trip induced congestion may arise, or trips that will not delay each other, containing only non-conflicting arcs. This differentiation allows us to apply big-M constraints selectively, focusing only on conflicting arcs and relevant trip pairs, thereby significantly reducing the number of big-M constraints appearing in the MILP. Additionally, we explore methods to merge or exclude non-conflicting arcs from the model, further minimizing its complexity and the computational overhead of the local search method discussed later in the section.

In the rest of this section, we will proceed as follows. In Section 3.3.2, we detail how to determine arc-specific time windows for each trip. In Section 3.3.2 and 3.3.2, we detail how we derive the multigraph representation introduced above. Building on this, we ultimately reduce the number of big-M constraints required in our MILP model.

Determining arc-dependent time windows

To distinguish trips that may conflict from trips that do not, we construct upper and lower bounds on the entry and exit times for each given arc on a trip. At a high level, the goal is to make these bounds as tight as possible while ensuring that every feasible solution satisfies them, which allows to reduce the number of big-M constraints.

Specifically, we let $\underline{e}_a^r$, $\bar{e}_a^r$ represent the earliest and latest *entry* time of trip r in arc a. Similarly, we use $\underline{\ell}_a^r$, $\bar{\ell}_a^r$ to represent earliest and latest *exit* times. Note, however, that knowledge of the earliest entry time $\underline{e}_a^r$ and latest exit time $\bar{\ell}_a^r$ for all arcs on a trip suffices to reconstruct also the other quantities, i.e., the latest entry and the earliest exit over all the arcs on that trip. In fact, by continuity, the latest entry on an arc $\bar{e}_a^r$ must equal the latest exit from the previous arc on that trip, while the earliest exit from an arc $\underline{\ell}_a^r$ must equal the earliest entry on the subsequent arc on that trip. This is the case for all arcs in the trip except for the very first and last arc. For the first arc, we set the latest entry as $\bar{e}_{\underline{a}_r}^r = e^r + \bar{\sigma}^r$, while for the last arc we determine the earliest exit by letting the vehicle travel in free flow through that arc so that $\underline{\ell}_{\bar{a}_r}^r = \underline{e}_{\bar{a}_r}^r + \tau_a$.

Thus, in the following, we focus on determining only the earliest entry $\underline{e}_a^r$ and the latest exit $\bar{\ell}_a^r$ times. The idea is to initialize these quantities by propagating the earliest entry times over subsequent arcs on a trip as if vehicles were traveling in free-flow, starting from the earliest departure e^r at the first arc. Similarly, for the latest exit times, which we propagate backward over preceding arcs on a trip, starting from the latest arrival at the last arc ℓ^r. In a second phase, we then refine and tighten these time windows by bounding the minimum and maximum congestion encountered by each trip over its arcs.

We now describe both of these steps in further detail and refer to Algorithm 1 for a formal overview of the approach.

Initialization. For each trip, we set the earliest entry time $\underline{e}_a^r$ for its first arc equal to the earliest departure time of the trip. We then calculate the earliest entry times for the subsequent arc by adding the nominal travel time of the preceding arc, that is $\underline{e}_{\delta^r(a)}^r = \underline{e}_a^r + \tau_a$ for all $a \in p^r \setminus \bar{a}_r$. For each trip, we set the latest exit time $\bar{\ell}_a^r$ for its last arc equal to the latest arrival time of the trip ℓ^r. We then calculate the latest exit time for the preceding arc by subtracting the nominal travel time of the arc, that is $\bar{\ell}_a^r = \bar{\ell}_{\delta^r(a)}^r - \tau_{\delta^r(a)}$ for all $a \in p^r \setminus \bar{a}_r$.

We then construct a queue whose elements are the tuples indexed by (a, r), i.e., arc and trip, and sort such queue based on the smallest earliest entry times. We then process, in order, one element of the queue at a time to update its earliest entry and latest exit time. First, we describe the method for updating time windows. Following that, we detail how employing a priority queue results in tighter bounds compared to an unsorted approach.

Updating earliest entry times. Given an element (a, r), we now update, i.e., move forward in time, the earliest entry times on arcs subsequent to a to tighten the time windows. Towards this goal, we compute the minimum conflicts that trips r inevitably incur on arc a and denote it with $\underline{f}_a^r$. This quantity can be determined by leveraging the existing time windows as

$$\underline{f}_a^r = \sum_{r' \in \mathcal{R}_a \setminus \{r\}} \mathbb{I}(\underline{e}_a^r > \bar{e}_a^{r'} \wedge \bar{e}_a^r < \underline{\ell}_a^{r'}).$$

We denote $\underline{x}_a^r$ as the delay that r incurs on arc a in presence of $\underline{f}_a^r$. As the earliest entries on arcs subsequent to a are surely delayed by $\underline{x}_a^r$, we update $\underline{e}_{a'}^r \leftarrow \underline{e}_{a'}^r + \underline{x}_a^r$ for every arc $a' \in p^r$ such that $\underline{e}_{a'}^r > \underline{e}_a^r$.

Updating latest exit times. To update the latest exit time $\bar{\ell}_a^r$, we determine the maximum number of conflicts, $\bar{f}_a^r$, that a trip r can encounter on arc a. A simple counting argument reveals that this is equal to the number of other trips $r' \in \mathcal{R}_a$ whose time windows $[\underline{e}_a^{r'}, \bar{\ell}_a^{r'}]$ intersect $[\underline{e}_a^r, \bar{e}_a^r]$. We then update the latest exit time from arc a based

Algorithm 1 Compute arc-specific time windows

Input: Set of trips $\mathcal{R}$

1: Q, $\underline{\boldsymbol{e}}$, $\bar{\boldsymbol{\ell}}$, $\underline{\boldsymbol{x}} \leftarrow$ INITIALIZE($\mathcal{R}$)
2: **while** Q is not empty **do**
3: $\quad(a, r) \leftarrow$ Q.POP() ▷ Extract tuple with highest priority.
4: $\quad\underline{\boldsymbol{e}}$, $\underline{\boldsymbol{x}} \leftarrow$ UPDATEEARLIESTENTRYTIMES(a, r)
5: $\quad\bar{\boldsymbol{\ell}} \leftarrow$ UPDATELATESTEXITTIMES(a, r)
6: LB $\leftarrow$ GETINITIALLOWERBOUND($\underline{\boldsymbol{x}}$)
7: **return** ($\underline{\boldsymbol{e}}$, $\bar{\boldsymbol{\ell}}$, LB)

on the latest entry in such arc and the maximum number of conflicts computed, that is, $\bar{\ell}_a^r \leftarrow \min(\bar{\ell}_a^r, \bar{e}_a^r + \tau_a^r(\bar{f}_a^r))$.

Computing an initial lower bound. We conclude Algorithm 1 by calculating an initial lower bound LB, which already allows evaluating the quality of a solution π. After finalizing the computation of arc-dependent time windows, the initial lower bound results from summing up the minimum delays that trips unavoidably incur on each arc:

$$\text{LB} = \sum_{r \in \mathcal{R}} \sum_{a \in p^r} \underline{x}_a^r.$$

The priority queue. The priority queue, organized by ascending earliest entry times $\underline{e}_a^r$, improves upon an unsorted approach to processing the sequence of the time window for two main reasons:

i) Processing pair (a, r) enables us to tighten both the earliest entry times for arcs after a in r's route, as well as the latest exit time from a. In this context, proceeding in an ascending order aligns with this forward-looking update mechanism.

ii) Ordering by $\underline{e}_a^r$ allows to tighten the maximum number of conflicts $\bar{f}_a^r$. As seen above, r may conflict with r' on a if $[\underline{e}_a^r, \bar{e}_a^r]$ overlaps with $[\underline{e}_a^{r'}, \bar{\ell}_a^{r'}]$. At the time the algorithm processes pair (a, r), the forward-looking update logic highlighted in point i) ensures $\underline{e}_a^r$, $\underline{e}_a^{r'}$ and $\bar{e}_a^r$ to be as tight as possible. Nevertheless, $\bar{\ell}_a^{r'}$ may or may not be as tight as possible, depending on the following case distinction:

   1) If $\underline{e}_a^r > \underline{e}_a^{r'}$, the algorithm processed r' before r on a. Thus, $\bar{\ell}_a^{r'}$ is as tight as possible consequent to the update logic of point i).
   2) If $\underline{e}_a^r < \underline{e}_a^{r'}$, r is processed before r'. Although $\bar{\ell}_a^{r'}$ is not as tight as possible, the determinant for overlap is $\underline{e}_a^{r'}$, which is as tight as possible.

Note that changes to a trip's earliest entry time can disrupt the queue's ordering. To address this, we maintain the queue's priorities to ensure that tuples are processed in an ascending time sequence.

Discussion. Notice that our current method tends to overestimate the worst-case delay by focusing solely on the bounds related to the specific arc under consideration. We do not dynamically adjust these bounds based on changing scenarios across consecutive arcs. In Appendix A3.3, we provide a small numerical example demonstrating how such a conservative approach might not capture the interplay of delays across arcs and how one could derive tighter bounds. As the scalability of the approach largely hinges on the tightness of departure bounds, addressing this limitation through a dynamic bound adaptation process presents a promising direction for future research.

Moreover, one can iterate the bounding procedure multiple times to achieve more precise bounds. This can be achieved by iteratively populating Q with newly computed earliest

departure bounds and running the procedure until convergence.

Calculating conflicting sets

After determining arc-specific time windows for all trips, we proceed by calculating conflicting sets $\mathcal{S}_a^i$, i.e., sets containing pairs of trips that can incur delay by conflicting. This process partitions the set of trips using a given arc over time into smaller subsets, where each subset groups trips that share a specific time window. Notably, trips in one subset do not overlap with those in another in their utilization of the arc. To achieve this, we define for each arc a a set $\mathcal{P}_a$ that contains all pairs of distinct trips belonging to $\mathcal{R}_a$ that might potentially conflict:

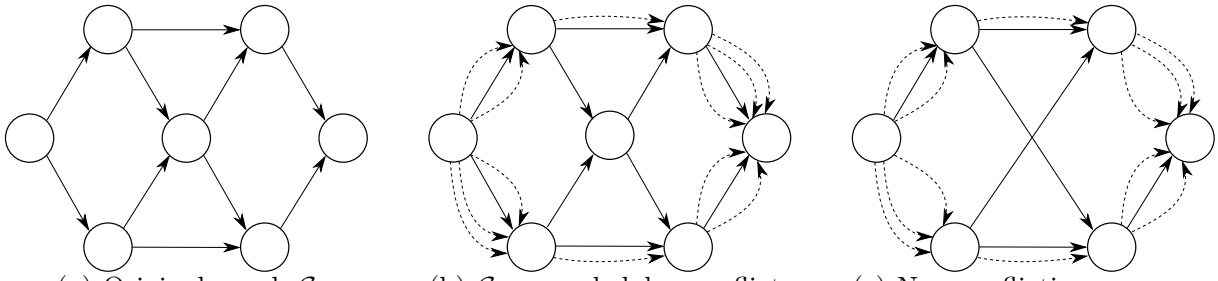
$$\mathcal{P}_a := \{(r, r') \in \mathcal{R}_a \times \mathcal{R}_a \,|\, r \neq r' \wedge [\underline{e}_a^r, \bar{e}_a^r] \cap [\underline{e}_a^{r'}, \bar{\ell}_a^{r'}] \neq \emptyset\}.$$

Note that we use tuples because potential conflicts are asymmetric. Specifically, in our definition, a trip r may conflict with another trip r', but this does not imply the reverse. By construction, $\mathcal{P}_a$ contains all tuples representing potential conflicts between trips that utilize a. Still, within $\mathcal{P}_a$, distinct subsets of tuples built on trips that do not appear in any other subset may emerge. This separation enables us to identify subsets of trips that may conflict with each other but not with other trips utilizing arc a. To isolate these dependencies, we split $\mathcal{P}_a$ into subsets of maximal size such that each subset contains only tuples formed from trips exclusive to that particular subset, ensuring that no trip is included in more than one subset. Technically, we partition $\mathcal{P}_a$ into n sets $\mathcal{P}_a^i$ of maximal size that satisfy the following conditions:

i) The subsets $\mathcal{P}_a^i$ are pairwise disjoint and their union constitutes the entire set $\mathcal{P}_a$, i.e., $\mathcal{P}_a := \bigcup_{i=1}^n \mathcal{P}_a^i$ and $\mathcal{P}_a^i \cap \mathcal{P}_a^j = \varnothing$ for all $i \neq j$.

ii) For any subset $\mathcal{P}_a^i$ containing more than one pair, it holds that for each pair $(r, r') \in \mathcal{P}_a^i$, there exists another pair $(r'', r''') \in \mathcal{P}_a^i$ such that the intersection of their components is non-empty: $\{r, r'\} \cap \{r'', r'''\} \neq \varnothing$. If no such pair (r'', r''') exists, then (r, r') constitutes a singleton subset in the partition.

With $\mathcal{P}_a^i$ containing all tuples relative to a subset of trips, we can finally identify those trips that may incur a delay. According to Equation (3.1), this is the case when the number of conflicts that a trip in $\mathcal{P}_a^i$ has, i.e., the number of trips traversing an arc at the same time, exceeds the first threshold $\hat{f}_a^1$, beyond which it is not possible to travel on arc a at free-flow speed. Then, we can tighten the maximum number of conflicts $\bar{f}_a^r$ by counting the number of pairs in $\mathcal{P}_a^i$ in which a trip r appears as the first element. Accordingly, we filter our partition of $\mathcal{P}_a^i$ to derive conflicting sets $\mathcal{S}_a^i$, each containing



(a) Original graph $\mathcal{G}$ (b) $\mathcal{G}$ expanded by conflicting arcs (c) Non-conflicting arcs processing

Figure 3.2: Schematic representation of the graph transformation process upon identification of conflicting sets: (a) initial graph structure. (b) expansion with conflicting arcs (dashed lines). (c) result of merging sequences of non-conflicting arcs and removing unused arcs. The merging process involves removing the central node and replacing its incident arcs with non-conflicting arcs with the original arcs' cumulative length.

trips that may induce delay, as:

$$\mathcal{S}_a^i := \left\{ (r, r') \in \mathcal{P}_a^i \,|\, \bar{f}_a^r \geq \hat{f}_a^1 \right\}.$$

Note that we omit r' in the set definition, as our goal is identifying trips r with sufficient conflicts (r, r') in $\mathcal{P}_a^i$ to meet the free flow threshold $\hat{f}_a^1$.

Multigraph representation and MILP constraints sparsening

After identifying all conflicting sets, we aim to modify our original graph $\mathcal{G}$ towards a multigraph representation. This allows us to maintain Constraints (3.3h)-(3.3n) at a minimum. Figure 3.2 shows an example of our graph modification that consists of three steps: in the first step, we transition to a multigraph by adding a parallel arc for each conflicting set; in the second step, we sparsen our graph by removing arcs that are no longer used and merging sequences of conflict-free arcs; in the final step, we redefine the conflict-checking constraint sets to include only relevant trip pairs.

Step 1: For each conflicting set $\mathcal{S}_a^i$, we add a conflicting arc $\hat{a}_i$ as a parallel arc to its original counterpart a. This arc shares the characteristics of a, i.e., it has the same start point, endpoint, nominal travel time, and capacity. After adding a conflicting arc $\hat{a}_i$, we modify the routes that belong to the trips contained in $\mathcal{S}_a^i$ by replacing the original arc a with the respective conflicting arc $\hat{a}_i$, such that all trips that share a potential conflict traverse the same conflicting arc. Let the set of conflicting arcs be $\hat{\mathcal{A}} \subset \mathcal{A}$. After Step 1, only trips that cannot induce delay on an arc a will still use arc $a \in \mathcal{A} \setminus \hat{\mathcal{A}}$. Accordingly, we will refer to trips that utilize only arcs $a \in \mathcal{A} \setminus \hat{\mathcal{A}}$ as non-conflicting trips and to the respective arcs as non-conflicting arcs. We remove trips from $\mathcal{R}$ that traverse only

non-conflicting arcs.

Step 2: To sparsen our arc set, we aim to merge as many non-conflicting arcs as possible. We combine sequences of non-conflicting arcs in trip routes into single arcs, which we add to $\mathcal{A}$. Each new arc starts at the origin of the first arc in the sequence and ends at the destination of the last arc, and its length equals the sum of the lengths of all arcs in the sequence. We update accordingly the routes p^r in which such sequences appear. Finally, we remove arcs from $\mathcal{A}$ that do not appear in any trip's route and update $\mathcal{R}_a$ for every arc in $\mathcal{A}$ accordingly. In Section 3.4, we illustrate how this procedure impacts the sizes of sets $\mathcal{R}_a$ and the number of arcs in the multigraph representation in our experiments.

Step 3: The remaining task involves updating the definition of Constraints (3.3h)-(3.3m) as follows:

$$\text{Constraints (3.3h)} - \text{(3.3m)}, \qquad \forall a \in \hat{\mathcal{A}}, (r, r') \in \mathcal{P}_a$$

thereby applying these constraints exclusively to conflicting arcs, $\hat{\mathcal{A}}$, and to each pair of trips that might conflict and potentially cause delays. Finally, we accordingly tighten the corresponding M_i values by incorporating the refined time window bounds into the constraints derived in Section 3.3.1.

3.3.3 Matheuristic

We develop a matheuristic incorporating our MILP into a local search scheme to find good solutions for large real-world instances. While this matheuristic is not guaranteed to find an optimal solution, it leverages the MILP from Section 3.3.1 and can thus detect an optimal solution if encountered. Algorithm 2 shows a high-level pseudo-code of our matheuristic. After creating an initial solution π that is agnostic to staggering, we apply a local search to improve it by staggering its trip departures. We then aim to further improve π by iterating between our MILP and our local search: whenever we find a new incumbent by solving the MILP, we stop and improve this incumbent with our local search.

Algorithm 2 Matheuristic

Input earliest departure times $\boldsymbol{e}$, initial lower bound LB, time limit η

$\pi \leftarrow$ constructSolution($\boldsymbol{e}$)
$\pi \leftarrow$ localSearch(π)
while time elapsed $\leq \eta$ **do**
    LB, $\pi \leftarrow$ solveMILP(LB, π)
    if $D(\pi) =$ LB **then**
        return π
    $\pi \leftarrow$ localSearch(π)
return π

Once the local search terminates, we feed its final solution back to warmstart the MILP. Our matheuristic stops either when the MILP certifies optimality of π, i.e., $D(\pi) = \text{LB}$, or after a maximum time η elapsed. Note that even if the MILP cannot certify optimality within the computational time limit, our matheuristic has the advantage of providing a lower bound to indicate the quality of π. In the remainder of this section, we detail our construction routine to obtain an initial solution (Section 3.3.3.1) before we describe our local search (Section 3.3.3.2).

3.3.3.1 Constructing initial solutions

To construct an initial solution π, we set its departure time variables s_a^r, delay variables x_a^r and conflict counting variables f_a^r to infinity. We then sort the trip-specific earliest departures e^r of every trip r into a priority queue Q and initialize an empty set $\mathcal{T}_a$ for each arc a to track when each trip completes the traversal of a. Then, we construct an initial solution iteratively:

i) We start by extracting the departure time s_a^r with the highest priority from Q.

ii) We then set $f_a^r = |\{\bar{s}_a^{r'} \in \mathcal{T}_a \,|\, s_a^r < \bar{s}_a^{r'}\}|$, that is the number of trips r' that have not completed their traversal of arc a by time s_a^r, and update

$$x_a^r \leftarrow \max_{k \in K}\{0, \sum_{k'=1}^{k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'})\}.$$

iii) Finally, we add the time in which the arc traversal is completed, which is $s_a^r + x_a^r + \tau_a$, to $\mathcal{T}_a$ by setting $\mathcal{T}_a = \mathcal{T}_a \cup (s_a^r + x_a^r + \tau_a)$. If a is not the last arc of route p^r, we insert $s_a^r + x_a^r + \tau_a$ into Q.

We repeat these steps until Q is empty. Note that once we obtained a feasible solution, it is possible to compute values for the MILP's binary variables $\alpha_a^{r,r'}, \beta_a^{r,r'}$ and $\gamma_a^{r,r'}$ with a support function checking their activation conditions as introduced in Section 3.3.1.

3.3.3.2 Local search

Our local search aims to iteratively remove conflicts based on a priority queue, aiming to first resolve conflicts that induce the largest delays. Let the tuple (r, r', a) index the conflict that r has with r' on arc a. Given a conflict (r, r', a), we resolve it by either shifting r backward or r' forward in time without violating the trips' time windows. In this context, we compute additional quantities for each conflict, utilizing the relations shown in Figure 3.3. Specifically, consider a conflict (r, r', a) with $s_a^r > s_a^{r'}$. We then compute the time overlap Δ between r and r', which denotes how much staggering we

(a) Conflict metrics (b) Backward and forward shifts

Figure 3.3: Figure (a) illustrates the quantities computed in the local search for resolving a conflict (r, r', a), where r incurs delay x_a^r. It illustrates the time overlap Δ, the feasible backward shift ρ^{f}, and forward shift ϕ^{f}. Figure (b) depicts trips r and r' after applying these shifts, resulting in conflict resolution and the consequent elimination of the arc delay.

need to apply to resolve the conflict. Formally:

$$\Delta = s_a^{r'} + \tau_a + x_a^{r'} - s_a^r + \varepsilon.$$

Clearly, $\Delta = \phi = \rho$ indicates the required forward shift ϕ for r' and the required backward shift ρ for r to resolve the conflict by staggering just one of the two trips. However, shifting a trip by Δ may cause time window violations. Therefore, we limit ρ and ϕ to their feasible counterparts, ρ^{f} and ϕ^{f}. These values represent the maximum backward and forward time shifts applicable without violating the time window of each trip. Formally, ϕ^{f} reads:

$$\phi^{\mathrm{f}} = \min(s^{r'} - e^{r'}, \Delta),$$

while ρ^{f} reads:

$$\rho^{\mathrm{f}} = \min(\bar{\sigma}^r - (s^r - e^r), \Delta),$$

with s^r denoting the starting time of a trip r. Based on these quantities, Algorithm 3 provides the pseudocode for our local search: given an initial solution π, we sort conflicts into a priority queue Υ, thus ordering conflicts based on their induced delay in decreasing order (l.1). We then process Υ until it is empty (l.2). In each iteration, we pop the first conflict from Υ (l.3) and compute ϕ^{f}, ρ^{f}, and Δ as defined above (l.4). Only if the sum of ϕ^{f} and ρ^{f} allows to compensate Δ (l.5), we resolve the conflict (l.6). If this leads to an improved solution (l.7), we update our solution (l.8) as well as our priority queue (l.9) since new conflicts may arise. If the conflict cannot be resolved, we directly proceed with the next item in Υ without updating π or Υ. In the remainder of this section, we elaborate on constructing and evaluating a neighborhood to create a new solution that resolves a conflict.

While our local search follows a rather simple greedy repair selection based on conflict

Algorithm 3 Local Search

Input: solution π
1: $\Upsilon \leftarrow$ identifyConflicts(π)
2: **while** Υ is not empty **do**
3: $(r, r', a) \leftarrow \Upsilon$.pop()
4: $\phi^{\mathrm{f}}, \rho^{\mathrm{f}}, \Delta \leftarrow$ analyzeConflict(r, r', a)
5: **if** $\phi^{\mathrm{f}} + \rho^{\mathrm{f}} \geq \Delta$ **then**
6: $\bar{\pi} \leftarrow$ resolveConflict$(\pi, r, r', \phi^{\mathrm{f}}, \Delta)$
7: **if** $D(\bar{\pi}) < D(\pi)$ **then**
8: $\pi \leftarrow \bar{\pi}$
9: $\Upsilon \leftarrow$ identifyConflicts($\bar{\pi}$)
10: **return** π

severity, its complexity lies in efficiently evaluating a neighborhood, i.e., computing the impact of resolving one conflict on all other trips' departure times. In the best case, resolving a conflict induces changes only in arc entry times for the conflicting trips, while in the worst case, resolving a conflict can lead to a very large cascade of changes in other trip departures. Algorithm 4 shows how we efficiently update our solution when resolving a conflict: we first apply a minimum staggering to the departures of r and r'. When staggering a trip departure time to resolve a conflict, we insert a corresponding tuple (r, a), consisting of the trip's index r and its route's first arc a into a priority queue Q, which we use to maintain trips that require recalculating arc entry times due to the applied staggering. We sort the tuples in Q in ascending order based on their arc departure times s_a^r (l.1). After resolving the initial conflict, we start processing trips in Q (l.3) by checking whether the trip's updated departure time s_a^r creates a change in the arc entry times of another trip r' (l.4). In this case, we insert r' into Q. If inserting r' into Q invalidates a further propagation of arc entry times for trip r – i.e., if $s_a^{r'} < s_a^r$, which disrupts the order of our priority queue – we reinsert s_a^r into Q and restart evaluating Q (l.5&6). If s_a^r does not induce changes in other trips, we recursively propagate the change in trip r's arc entry times on subsequent arcs until a change arises (l.7–l.13). Our overall update ends once Q is empty and all arc entry times have been correctly adjusted. To conclude this section, we detail how we stagger departure times, check for solution changes, and maintain the priority queue.

staggerDepartures$(r, r', \phi^{\mathrm{f}}, \rho^{\mathrm{f}}, \Delta)$: To resolve a conflict, we need to shift either r or r' or both in time by Δ. Clearly, it is desirable to shift only one of these trips in time to keep the number of newly arising conflicts and entry time changes at a minimum. Accordingly, we proceed as follows to resolve a conflict:

1. If $\rho^{\mathrm{f}} = \Delta$ we adjust $s^r \leftarrow s^r + \Delta$ and push $(r, \underline{a}_r)$ into Q.
2. If a conflict cannot be resolved by shifting only trip r in time, i.e., $\rho^{\mathrm{f}} < \Delta$, we stagger both trips such that $s^r \leftarrow s^r + \rho^{\mathrm{f}}$ and $s^{r'} \leftarrow s^{r'} - \phi^{\mathrm{f}}$ and push $(r, \underline{a}_r)$ as well as $(r', \underline{a}^{r'})$ into Q.

Algorithm 4 resolveConflict

Input: solution π, trips r and r', feasible shifts ϕ^{f} and ρ^{f}, time overlap Δ

$\mathrm{Q} \leftarrow$ staggerDepartures$(r, r', \phi^{\mathrm{f}}, \rho^{\mathrm{f}}, \Delta)$
while Q is not empty **do**
  $(r, a) \leftarrow \mathrm{Q}.\mathrm{pop}()$
  $r', x_a^r \leftarrow$ updateSolution(r, a)
  if $r' \neq$ false **then**
    $\mathrm{Q} \leftarrow$ insertTrips(r, r', a)
  else
    while $r' =$ false **do**
      $s^r_{\delta^r(a)} \leftarrow s_a^r + \tau_a + x_a^r$
      $a \leftarrow \delta^r(a)$
      $r', x_a^r \leftarrow$ updateSolution(r, a)
      if $r' \neq$ false **then**
        $\mathrm{Q} \leftarrow$ insertTrips(r, r', a)
return π

updateSolution(r, a): After staggering trips to resolve a conflict, we need to update the arc entry times of these trips as well as the arc entry times of all other trips that might be affected by the change. To do so, we first compute the number of conflicts f_a^r that arise for the respective trip and arc as:

$$f_a^r = \sum_{r' \in \mathcal{R}_a \setminus \{r\}} \mathbb{I}(s_a^{r'} \leq s_a^r < s_a^{r'} + \tau_a + x_a^{r'}),$$

which subsequently allows us to update the corresponding delay as:

$$x_a^r = \max_{k \in K} \left\{ 0, \sum_{1 \leq k' \leq k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'}) \right\}.$$

We can then check whether a new conflict arises or an existing conflict gets resolved between r and some other trip r' due to the updated entry time of r. If either is the case, we return the index of trip r'; otherwise, we return a false flag to indicate that no new conflict was induced or resolved by staggering r.

insertTrips(r, r', a): If we detect a new conflict between r and r', we add (r', a) to Q. Here, we need to account for the following two cases: if $s_a^r < s_a^{r'}$, changes induced in r' do not affect r. It suffices to push (r', a) into Q. However, if $s_a^{r'} < s_a^r$, changes in r' affect r, which invalidates preceding updates made to r. In this case, we additionally push (r, a) back to Q to ensure valid arc entry time updates upon the algorithm's termination.

3.3.4 Online algorithm

While we discussed our matheuristic for a static problem setting, we can straightforwardly apply it to an online problem setting where trips enter the system over time, e.g., following

a point process. In this context, we focus on batched decision-making, which allows us to apply our matheuristic in a rolling-horizon fashion.

Specifically, we divide the static time horizon into $n \in \{1, \dots, N\}$ distinct epochs, each spanning a fixed time interval of size Δt. Before a new epoch n starts, we then take a staggering decision for all requests that entered the system after the last decision taken, collected in a set $\mathcal{R}^n$, formally:

$$\mathcal{R}^n := \{r \in \mathcal{R} \mid (n-1) \cdot \Delta t \leq e^r < n \cdot \Delta t\}. \qquad \forall n \in \{1, \dots, N\}$$

We can then compute a solution π^n for the respective batch of requests by running our matheuristic with a suitable time limit η. Algorithm 5 shows how we apply our matheuristic in such a rolling horizon setting: we iterate over all epochs (l.2), each time focusing on the respective request set $\mathcal{R}^n$ (l.3), and calculating the relative earliest departure times vector $\mathbf{e}^n$, and a lower bound for epoch n as described in Sections 3.3.2-3.3.2 (l.4). For each epoch, we compute a solution π^n by running our matheuristic (l.5). Afterward, we can fix this solution for all trips in $\mathcal{R}^n$. Still, we need to identify trips that span multiple epochs to identify all conflicts in future epochs during preprocessing. To do so, we collect such trips in a set $\mathcal{R}^T$, formally:

$$\mathcal{R}^T := \{r \in \mathcal{R}^n \mid s^r_{a'} > n \cdot \Delta t\},$$

with a' being the last arc of route p^r. Note that after the respective adjustments, trips in $\mathcal{R}^T$ can also conflict with trips being in $\mathcal{R}^n$ but not in $\mathcal{R}^T$. Maintaining those trips by incorporating them directly in $\mathcal{R}^T$ poses a computational overhead in the matheuristic, as it necessitates the inclusion of additional conflicting trips to accurately calculate their arc entry times. To mitigate this overhead, we identify all conflicts (r, r', a) in π^n, where r is part of $\mathcal{R}^T$, r' is not, and a is included in p^r. For each identified conflict, we generate an

Algorithm 5 Online algorithm

Input Set of trips $\mathcal{R}$, number of epochs N, time limit η

1: $\mathcal{R}^T \leftarrow \varnothing$; $\mathcal{R}^D \leftarrow \varnothing$
2: **for** $n \in \{1, \dots, N\}$ **do**
3: $\quad \mathcal{R}^n \leftarrow$ getTripsEpoch($\mathcal{R}$)
4: $\quad \boldsymbol{e}^n, \mathrm{LB} \leftarrow$ preprocess($\mathcal{R}^n, \mathcal{R}^T, \mathcal{R}^D$)
5: $\quad \pi^n \leftarrow$ matheuristic($\boldsymbol{e}^n, \mathrm{LB}, \eta$)
6: $\quad$ **if** $n < N$ **then**
7: $\quad\quad \mathcal{R}^T \leftarrow$ getTransferTrips(π^n)
8: $\quad\quad \mathcal{R}^D \leftarrow$ getDummyTrips($\pi^n, \mathcal{R}^T$)
9: $\boldsymbol{\pi} \leftarrow (\pi^n)_{n \in \{1,\dots,N\}}$ ▷ Vector of epoch solutions
10: $\pi \leftarrow$ getSolution($\boldsymbol{\pi}$) ▷ Solution for every trip in $\mathcal{R}$
11: **return** π

artificial trip r' with $p^{r'}$ consisting solely of the arc a, $e^{r'}$ being the arc entry time of the conflicting trip $s_a^{r'}$, $\ell^{r'}$ matching the time in which the conflicting trip completes the arc traversal, and $\bar{\sigma}^{r'}$ being zero to preclude any staggering time decisions by the algorithm. We maintain these dummy trips in a separate set $\mathcal{R}^D$ and take them into account in future epochs.

3.4 Design of experiments

We used Python 3.11 and Gurobi 10.0.1 to build and solve the MILP model. To implement our matheuristic, we used C++ to implement the local search and integrated it into the Python framework via pybind11 (Jakob et al., 2017). We performed all experiments on an Intel(R) i9-9900 CPU, 3.1 GHz, with 56 GB of RAM.

To create a realistic case study, we use the New York taxi data set (NYCTLC, 2015) and focus on the area of Manhattan in New York City.

Road network. We extract the Manhattan road network from the OpenStreetMap dataset to obtain a network with 8537 arcs and 3518 nodes (OpenStreetMap, 2024). Studying staggered routing on this raw network remains challenging as it requires checking conflicts for a large number of arcs. To decrease the number of arcs while preserving the length of the shortest paths within the network, we use contraction hierarchies (cf. Geisberger et al., 2008), a well-known reduction technique for online routing, to contract the road network graph. We employ iterative node contractions that merge arcs while preserving the shortest routes between the remaining nodes. The efficiency of this contraction hinges on the order in which nodes are processed. We first contract nodes with the smallest *edge difference* - a metric capturing the difference between the number of shortcuts added at the time of node contraction and the number of incident arcs. Here, we adopt the *lazy update* approach of Geisberger et al. (2008) to reduce edge difference recalculations and enhance efficiency.

Trip data. We obtain trips $\mathcal{R}$ from the NYC taxi dataset, collected in January 2015 (NYCTLC, 2015), which contains taxi trips' origins, destinations, and departure times. We determine trip routes by selecting the shortest route connecting the nearest network nodes to the origin and destination with the minimum number of arcs. Trip start times e^r correspond to the taxi ride's start times provided in the dataset.

If the latest arrival times ℓ^r are too restrictive, the problem may become infeasible. A practical approach to ensure feasibility is setting ℓ^r at least as large as the arrival time in a no-staggering scenario, guaranteeing sufficient time for each trip without rescheduling. In our experiments, we implemented this by setting the maximum travel time to 125% of the nominal route travel time, creating a buffer for congestion-induced delays. This approach

consistently yielded feasible instances across the congestion levels studied, ensuring all trips remained within their time windows.

We calculate the maximum allowable departure time shift $\bar{\sigma}^r$ as a percentage ς^{MAX} of the total nominal travel time required to traverse the route p^r. Unless specified differently, we set this percentage to 10%.

Parameterization. We set the nominal travel times τ_a assuming vehicles traveling at 20 kilometers per hour, which approximates the average traffic speed recorded in New York City downtown in 2022 (Pishue, 2023). As we focus on the impact and control of a large AMoD fleet, we assume a baseload of traffic that is out of the operator's control and adjust residual arc capacities accordingly to account for potential congestion induced by the AMoD fleet. Specifically, we consider two scenarios: first, a LC scenario, which reflects a setting in which the exogenous traffic in the system is uncongested, and the AMoD fleet may induce congestion in case of inefficient operations. Second, a HC scenario in which the exogenous traffic already encounters congestion that might be amplified by the AMoD fleet operations. To create these scenarios, we define the arc travel time functions as piecewise linear functions with three non-flat segments. Residual free-flow capacities are calibrated to allow one AMoD trip every 20 seconds in the LC scenario and every 40 seconds in the HC scenario, setting the first breakpoints as $\hat{f}_a^1 = \tau_a/20$ and $\hat{f}_a^1 = \tau_a/40$, rounded to the nearest integer. The remaining breakpoints are $\hat{f}_a^2 = 1.8 \cdot \hat{f}_a^1$ and $\hat{f}_a^3 = 2 \cdot \hat{f}_a^1$. The slopes are defined as $\mu_a^1 = \frac{0.5 \cdot \tau_a}{\hat{f}_a^1}$, $\mu_a^2 = \frac{0.9 \cdot \tau_a}{\hat{f}_a^1}$, and $\mu_a^3 = \frac{1.1 \cdot \tau_a}{\hat{f}_a^1}$. Appendix A3.4 provides a visualization of the breakpoints and slopes, along with a brief discussion of our choice of parameters, justified by their approximation accuracy.

Instances. We generated instances by sampling 5000 trips between 4:00 and 5:00 p.m. over 31 days, sorting them by start time. To evaluate the complexity of these instances, we computed the uncontrolled solution in both LC and HC scenarios, measuring the total delay (see Figure 3.4). Notably, the total delay in HC scenarios is approximately 10 times higher than in LC scenarios. In this preliminary stage, we also assessed the impact of graph-to-multigraph processing and the trip set splitting procedure described in Sections 3.3.2-3.3.2. Table 3.1 provides an overview of the key network characteristics that influence instance complexity. The average number of arcs $|\mathcal{A}|$ over which trips transit is approximately 8000, while the average number of nodes $|\mathcal{V}|$ is approximately 2500. The processed multigraph features a high number of total arcs, $|\mathcal{A}|^{\text{m}}$, with the LC scenario displaying a higher average. This is attributed to fewer overlapping time windows in the LC scenario, enabling a finer splitting of trips across multiple parallel arcs. Conversely, the number of conflicting arcs, $|\hat{\mathcal{A}}|^{\text{m}}$, is significantly larger in HC scenarios, reflecting the greater congestion levels in the HC environment.

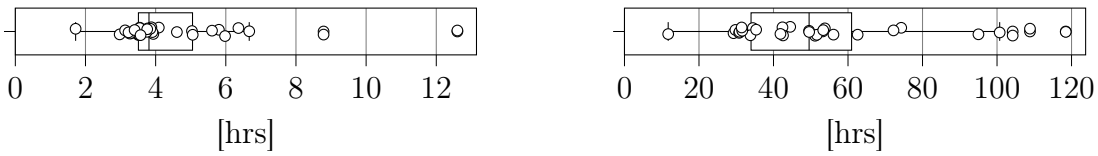
To further evaluate the trip set splitting procedure outlined in Section 3.3.2, Figure 3.5 shows the distribution of set sizes $|\mathcal{R}_a|$ before and after partitioning in both LC and HC scenarios. The partitioning method is particularly effective in reducing large conflicting sets, with a more pronounced impact in the LC scenario due to lower arc utilization and larger gaps between trips. However, the method also significantly benefits the HC scenario by alleviating congestion on heavily utilized arcs, thus improving the overall computational efficiency.

3.5 Results

In the following, we discuss the findings of our numerical studies. First, we analyze our matheuristic performance for both an idealized offline setting as well as a myopic online setting in Section 3.5.1. We then proceed with a fine-grained analysis in Section 3.5.2 before discussing the trade-off between congestion-related delay reduction and trip departure time shifts in Section 3.5.3.

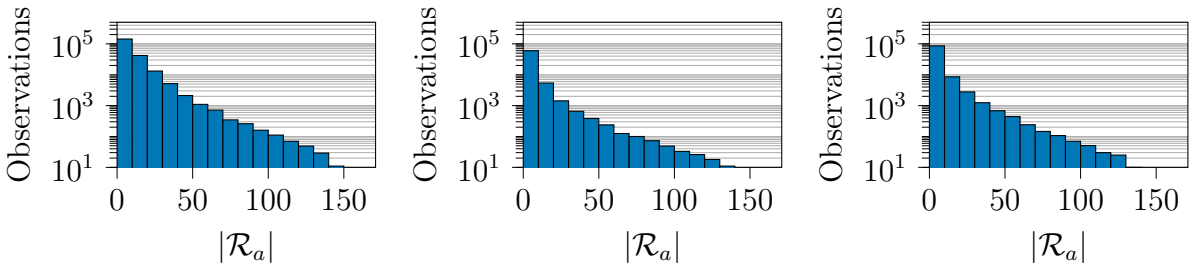
	LC			HC		
	Min	Max	Avg	Min	Max	Avg
$\lvert\mathcal{A}\rvert$	7822	8384	7967	7822	8384	7972
$\lvert\mathcal{V}\rvert$	2430	2655	2487	2430	2655	2489
$\lvert\mathcal{A}\rvert^{\mathrm{m}}$	22986	26115	24461	19704	23355	21136
$\lvert\hat{\mathcal{A}}\rvert^{\mathrm{m}}$	1996	2604	2288	3159	3575	3351

Table 3.1: Network overview for LC and HC scenarios.



(a) LC Scenario (b) HC Scenario

Figure 3.4: Total delay in the uncontrolled solutions for LC and HC scenarios.



(a) Undivided sets (b) LC Scenario (c) HC Scenario

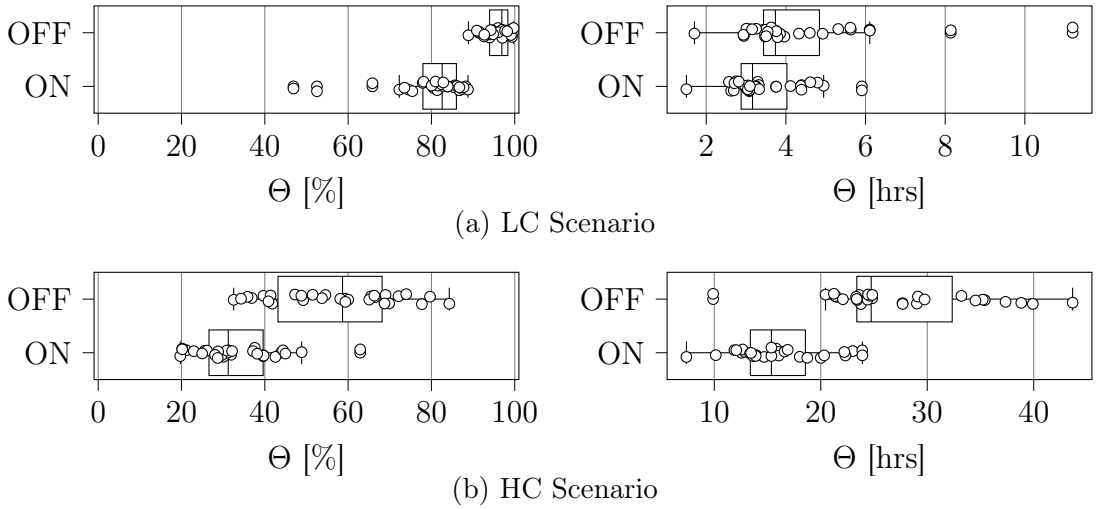
Figure 3.5: Trip set sizes utilizing arcs before and after processing in the LC and HC scenarios.

3.5.1 Algorithmic performance

Figure 3.6 shows the distribution of the delay reduction Θ obtained when using our matheuristic compared to the uncontrolled setting for both the LC and HC scenario. We report the delay reduction obtained when using our matheuristic in a full-information offline scenario (OFF) to obtain an upper bound on the staggered routing performance. Additionally, we report the delay reduction obtained when using our matheuristic in a rolling-horizon fashion in an online scenario (ON) to quantify the benefit of staggering in a real-world setting. For further technical discussions, we refer the readers to Appendix A3.5. We used a time limit of one hour to compute the offline solutions while limiting the computation of the rolling horizon solutions to six minutes, which equals the length of an epoch. Focusing on the offline delay reduction, we observe a median delay reduction of 98% and a minimum reduction of 85% for the LC scenario, which shows that our matheuristic allows us to mitigate at minimum 85% of the fleet-induced congestion in an initially uncongested system. For the HC scenario, we observe a delay reduction within 30% and 85%, with a median reduction of 60%, which shows that our matheuristic effectively reduces fleet-induced congestion even in a setting where the system is already congested by exogenous traffic.

Result 1 (offline performance) *In a full information setting, staggered routing allows for an average delay reduction of* 98 *% and* 60 *%, respectively, for an LC and HC scenario.*

Focusing on the improvement potential in an online decision-making setting, we observe an average delay improvement of 82% and 30% for the LC and HC scenario when



(a) LC Scenario

(b) HC Scenario

Figure 3.6: Distribution of the relative and absolute delay reduction Θ over all instances for a full information offline scenario (OFF) and a rolling-horizon online scenario (ON).

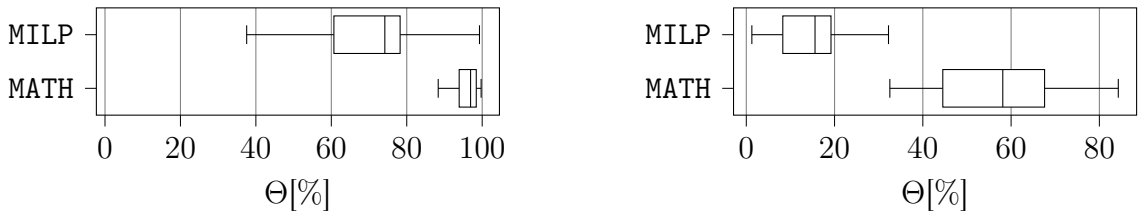
applying our matheuristic in a naive rolling horizon fashion. This underscores that even a simple, temporally local staggering of trips can significantly reduce congestion-related delays, especially in low-congestion scenarios. At the same time, it points to the potential of developing a more sophisticated prescriptive online algorithm that might obtain a performance close to the full information bound, particularly in high-congestion environments.

Result 2 (online performance) *In a rolling horizon setting, staggered routing allows for an average delay reduction of* 82% *and* 30%*, respectively, for an LC and HC scenario.*

To understand how the local search procedure enhances the search for improved solutions, we conducted a comparative analysis between our matheuristic (`MATH`) and the plain `MILP` with a time limit set to one hour. Figure 3.7 summarizes the absolute delay reductions Θ obtained by the two approaches in both the LC and HC scenarios. For the LC scenario, the performance of `MATH` shows a noticeable improvement compared to the `MILP`, achieving higher delay reductions with more consistent results across the instances. In the HC scenario, the advantage of `MATH` is even more pronounced, as it significantly outperforms the `MILP` by achieving greater delay reductions while exhibiting a wider variability in performance. These results highlight the effectiveness of the matheuristic, particularly in high-congestion settings, where it leverages its local search capabilities to explore better solutions under similar computational budgets.

3.5.2 Fine-grained analysis

To deepen our understanding of the impact of staggered routing, Figure 3.8 shows a distribution over arcs that exhibit a certain cumulative delay within the LC or HC scenario. We detail this distribution for the arc-based delay resulting in an uncontrolled setting (UNC) in which no staggering is applied, when solving the offline full information setting (OFF), and when solving an online rolling horizon setting (ON). Note that we exclude arcs that do not exhibit any congestion in either of these settings to focus the distributions on congestion effects. Figure 3.9 complements this analysis by showing the spatial



(a) LC Scenario (b) HC Scenario

Figure 3.7: Comparison of the absolute delay reductions Θ achieved by the `MILP` formulation alone and the full algorithm (`MATH`) for the LC and the HC scenario.

distribution of trips incurring delay on certain arcs.

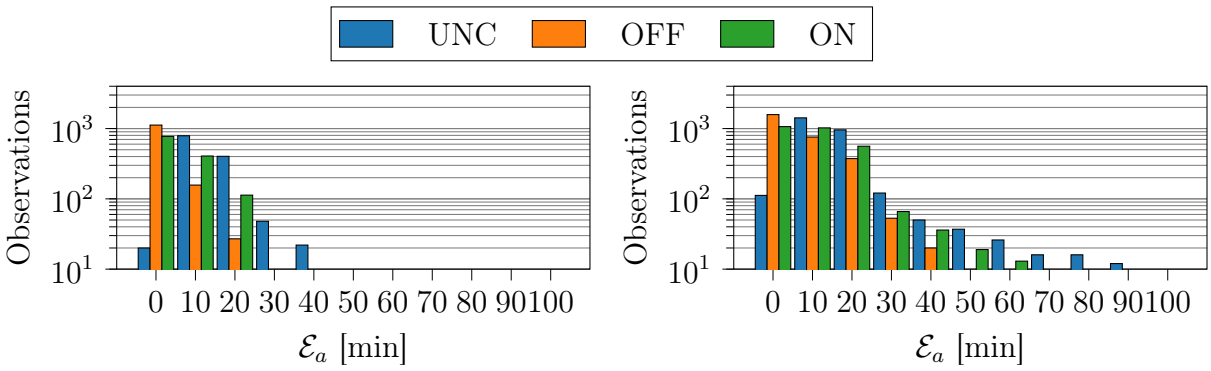
Clearly, the HC scenario exhibits a significantly higher cumulative delay on arcs as the existing exogenous congestion in the system reinforces the congestion induced by the AMoD fleet. However, we observe in both the LC and the HC scenarios that staggering trip departures allow us to mitigate a significant share of the congestion induced by the AMoD fleet. Specifically, our matheuristic completely eliminates congestion for approximately 1100 and 1500 arcs in the LC and HC scenarios in the offline setting, and for 800 and 1000 arcs in the online setting. Interestingly, the differences between the offline and online settings arise for the remaining mildly congested arcs for which the online algorithm does not succeed in anticipating temporal interdependencies.

Figure 3.9 shows the spatial distribution of the number of trips per arc that experience delay in the respective settings. As can be seen, only a few local congestion effects remain in the LC scenario when comparing the offline and online solutions after staggering to the uncontrolled setting. Contrarily, some main roads exhibit a decreased but still significant congestion in the HC scenario.

Result 3 (congestion reduction) *Staggered routing increases the number of uncongested arcs, respectively, for LC and HC scenarios by 1131 and 1472 when comparing the full information solutions and 773 and 957 when comparing the naive online solutions against the uncontrolled solutions.*

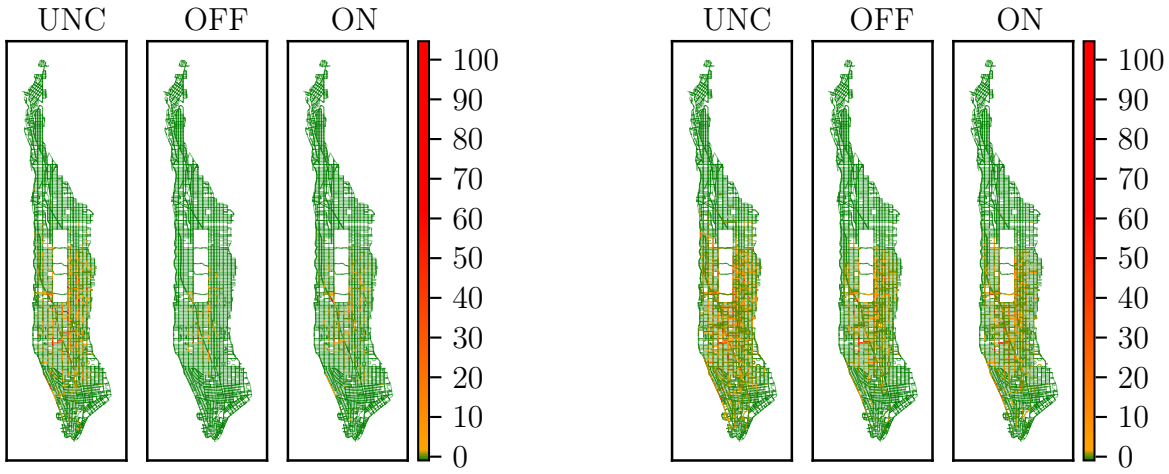
3.5.3 Sensitivity analyses

Clearly, the improvement potential of staggered routing depends on the amount of staggering allowed for each trip and constitutes a trade-off between passenger convenience,



(a) LC Scenario (b) HC Scenario

Figure 3.8: Distribution of the arc-based cumulative delay ($\mathcal{E}_a$) over all instances for the uncontrolled (UNC), offline full information (OFF), and online rolling horizon (ON) settings.



(a) LC Scenario (b) HC Scenario

Figure 3.9: Spatial distribution of the number of trips per arc that experience congestion (Ω_a) in the uncontrolled (UNC), offline (OFF), and online (ON) settings for both the LC and HC scenarios.

i.e., modifying a customer's departure time up to a certain threshold and obtaining a system optimum from a total congestion perspective. For this analysis, we focus on the full information setting for one single instance to discuss the maximum improvement potential of the respective trade-off. To analyze this trade-off, we ignore our basic setting and vary the maximum amount a trip's departure can be staggered within $\varsigma^{\text{MAX}} \in \{0\%, \ldots, 22.5\%\}$ of its nominal travel time with a step width of 2.5 percentage points. Here, $\varsigma^{\text{MAX}} = 0\%$ equals the uncontrolled setting discussed in the preceding subsections.

Figure 3.10 highlights the resulting trade-off by showing the dependency of the total accumulated delay $D(\pi)$ and the distribution of realized trip departure shifts σ^r in dependence of the maximum possible trip departure shift ς^{MAX}, which indicates the maximum possible departure shift relative to each trip's length. We report results for both the LC and HC scenarios. As can be seen, we observe diminishing returns for delay reduction when increasing ς^{MAX} in both scenarios but at different scales. While it is possible to entirely mitigate the average cumulative delay in the LC scenario with any $\varsigma^{\text{MAX}} \geq 7.5\%$, the HC scenario shows a plateau of an average cumulative delay of around 50 hours for $\varsigma^{\text{MAX}} \geq 10.0\%$. Remarkably, mitigating the entire delay in the LC scenario with maximum realized departure time shifts below two minutes is possible when choosing $\varsigma^{\text{MAX}} = 7.5\%$. Choosing a higher ς^{MAX} solely leads to higher departure time shifts. In the HC scenario, we observe increasing realized departure time shifts when increasing ς^{MAX} while yielding a decreasing overall delay in the system. Surprisingly, the realized departure time shifts remain mostly below six minutes even for high ς^{MAX}; only two outliers exhibit a shift of six minutes.

Result 4 (LC scenario trade off) *In the LC scenario, a maximum trip departure shift of 7.5% relative to each trip's length allows to mitigate delays induced by the AMoD fleet entirely at the price of shifting any trip departure by at most two minutes.*

Result 5 (HC scenario trade off) *In the HC scenario, we observe a decreasing cumulative delay at the price of increasing realized trip departure times when increasing ς^{MAX}. Still, the realized trip departure times remain, except from two outliers at most six minutes even for high ς^{MAX}.*

In conclusion, our results show that there is a trade-off between allowing higher maximum and realized departure time shifts and reducing delay in the system. However, a significant delay reduction can be obtained by accepting reasonable departure shifts between two and six minutes for both a system that comprises solely fleet-induced congestion (LC scenario) and a system that is already congested and incurs additional congestion induced by the AMoD fleet (HC scenario). In this context, it's important to recognize that while adjusting the departure time of a trip might not be preferable for a passenger, it nevertheless does not compromise the passenger's primary objective of reaching their destination punctually. This is because, independently of the ς^{MAX} parameterization, alterations to the departure time are only permissible as long as the trip finishes within the designated time window.

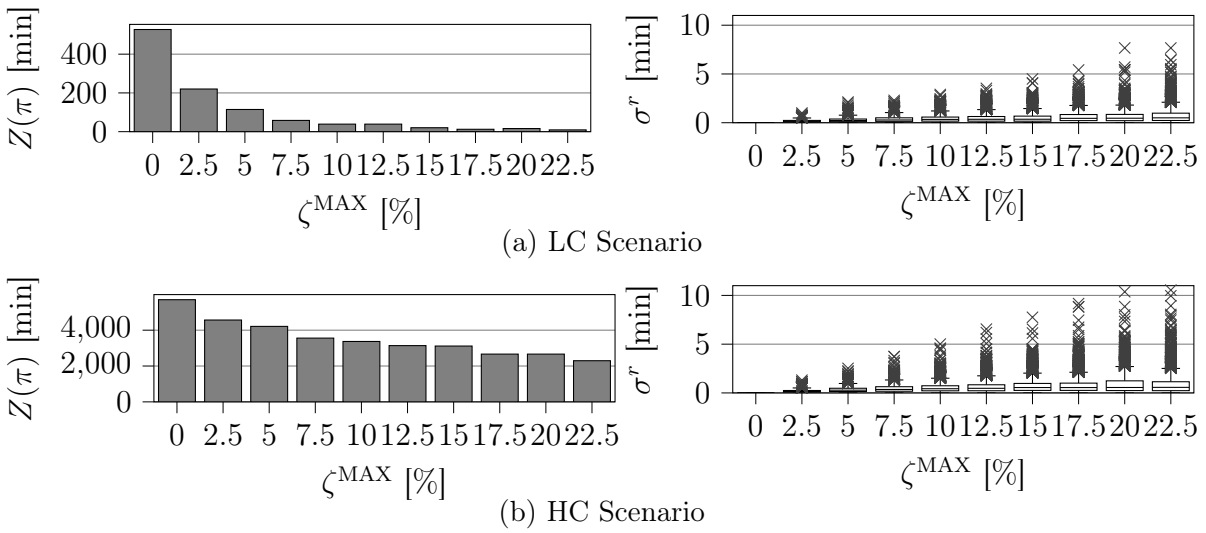


Figure 3.10: Trade-off between realized trip departure time shifts σ^r, the total system delay $D(\pi)$, and the maximum possible shift of a trip's departure relative to its length ς^{MAX} for both the LC and HC scenarios.

3.6 Conclusion

We studied the impact of staggered routing, i.e., delaying the departure of trips as long as a trip's prefixed arrival time will not be exceeded to reduce local congestion bottlenecks when operating an AMoD fleet. We formalized the underlying planning problem in a full information offline setting as a MILP. We leveraged this MILP to devise an efficient matheuristic that can be used to solve large-scale full information offline instances, while at the same time being amenable to be used in a rolling-horizon online setting. We applied this matheuristic to a real-world case study for the Manhattan area in New York City. We compared the impact of staggered routing for both a full-information bound computed in an idealized offline setting and for the solution obtained in a rolling-horizon online setting against an uncontrolled case in which staggering is not allowed. We demonstrate that our matheuristic outperforms a plain MILP formulation in both low- and high-congestion scenarios. Additionally, we show that our approach can efficiently handle multi-piece, piecewise linear approximations of polynomial convex delay functions while maintaining solvability.

Our results show that in low-congestion scenarios, i.e., scenarios in which only the AMoD fleet induces congestion but the system without the fleet would remain uncongested, staggering trip departures allow to mitigate on average 98% of the induced congestion in a full information setting. While not reaching this full information bound, our naive rolling horizon approach still allows us to reduce 82% of the induced congestion in the LC scenario. In high-congestion scenarios, i.e., scenarios in which the system already encounters exogenous congestion and the AMoD fleet induces further congestion, we observe an average reduction of 60% as the full information bound and an average reduction of 30% in our online setting. Surprisingly, we show that these reductions can be reached by shifting trip departures at a maximum of two minutes in the LC scenario and by six minutes in the HC scenario, still preserving each trip's desired arrival time.

Our work presents a first step towards leveraging staggered routing in AMoD systems to reduce congestion. Specifically, we anticipate two fruitful avenues for future research. First, focusing on efficient solution methodologies that solve large-scale instances with high solution quality. Our paper represents a first step in this direction. Still, one may improve upon our results by finding better lower bounds and further enhancing algorithmic components tailored to the respective problem setting. Second, developing prescriptive online algorithms to apply staggered routing in practice. Our myopic rolling-horizon implementation already shows a significant improvement potential, giving hope to achieve even better performance with a more sophisticated online algorithm incorporating a learning-based prescriptive element. Furthermore, our analysis highlights the critical balance between passenger convenience and system-wide congestion mitigation.

While short staggering times can significantly reduce congestion, practical implementation must account for passenger acceptance and market adoption. Future research should explore user behavior dynamics and investigate how staggered routing interacts with other congestion mitigation strategies, such as fare adjustments or congestion pricing.

Finally, integrating staggering directly into the routing decision process is a natural extension of our work, allowing for a more adaptive and anticipatory vehicle allocation strategy. By dynamically adjusting departures and routes in response to congestion patterns, this approach enhances system efficiency and improves overall network performance.

A3.1 Hardness proof

To prove the hardness of the staggered routing problem as defined in Section 3.2, we utilize the hardness result for the three-machine job shop scheduling problem (J3) with unit processing times (UPT) and no-wait constraints (NWT), denoted as J3-UPT-NWT, with makespan (the maximum completion time of a job) minimization as objective, which is NP-hard (Sriskandarajah & Ladet, 1986).

Theorem 1 *The staggered routing problem is NP-hard.*

Proof 1 *We begin by defining instances for J3-UPT-NWT and for the staggered routing problem:*

Definition 1 *Let I be an instance of the J3-UPT-NWT with $m = 3$ machines and jobs J_i, with $i = 1, \ldots, n$, where each job comprises $N(J_i)$ operations that each require one unit of processing time and must be executed continuously from start to finish. The operations are allocated to specific machines, and the order in which machines are visited may vary among different jobs. Each machine can process a maximum of one job at a time. The J3-UPT-NWT seeks for a job schedule S, i.e., the starting times $s(J_i)$ of the first operation of each job J_i, that minimizes the schedule's makespan D.*

Definition 2 *Let I' be an instance of the staggered routing problem with n trips as defined in Section 3.2. Trips must proceed along their routes without interruption within designated time windows. The travel time for a trip along an arc is influenced by the number of trips on the arc at the specific time the trip enters the arc. If the number of trips on the arc falls below a predefined arc-specific threshold capacity, the trip proceeds at the nominal travel time designated for that arc. Conversely, exceeding this threshold triggers a delay, as detailed in Equation* (3.1)*. The objective of the staggered routing problem is to find the trip departures s_a^r that minimize the total travel time.*

Given these definitions, we prove the hardness of the staggered routing problem by contradiction.

Assumption 1 *It exists an algorithm $\mathscr{A}$ that finds a feasible solution for every instance of the staggered routing problem in polynomial time.*

Step 1: Consider an instance I' that comprises n trips traversing $m = 3$ arcs, where at least two trips have different routes. The earliest possible departure time for trips is set to zero, with no restrictions on the staggering of departures. All trips must finish by time D. Each arc has a nominal travel time and a threshold capacity of one; exceeding this threshold on any arc introduces an infinite delay.

Step 2: Instance I' can be transformed in polynomial time to an instance I as follows: the route of each trip corresponds to a job to be executed without interruptions from start to finish. The act of traversing the arcs within a trip's route mirrors the operations within a job. These operations are executed by a dedicated machine (an arc), each with a unit processing time (equivalent to nominal travel time), and concurrent processing of operations on the same machine is prohibited (otherwise, one incurs infinite delay). The latest arrival times translate to the schedule's makespan under consideration. Finally, the departure times from the arcs on a route correspond to the scheduling times of the job operations.

Step 3: Observe that selecting the job's starting times in instance I to decide whether a schedule S of makespan of D exists is equivalent to finding feasible trip departure times in instance I', such that we obtain a polynomial-time transformation from the respective staggered routing problem instance to an equivalent J3-UPT-NWT instance. Then, if $\mathscr{A}$ can find a feasible solution in polynomial time for I', it can also find a feasible solution for I within polynomial time. However, finding a solution for I in polynomial time is impossible unless $P = NP$, contradicting Assumption 1. Accordingly, there exists at least one instance of the staggered routing problem that cannot be solved in polynomial time. □

A3.2 Table of variables in the MILP

See Table 3.2.

Table 3.2: Table of variables in the MILP.

Variable	Description
$\mathcal{R}$	Set of all trips $r \in \mathcal{R}$
$\mathcal{A}$	Set of all arcs $a \in \mathcal{A}$
$\mathcal{R}_a$	Subset of $\mathcal{R}$, set of trips traversing arc a
p^r	Route of trip r
$\delta^r(a)$	Arc succeeding a on route p^r
$\underline{a}_r$	First arc of p^r
$\bar{a}_r$	Last arc of p^r
x_a^r	Delay on arc a for trip r
μ_a^k	Slope of the k-th segment of arc a travel time function.
τ_a	Nominal travel time for arc a
f_a^r	Number of vehicles encountered by trip r on arc a
$\hat{f}_a^k$	Flow value at which the k-th segment of arc a travel time function begins.
s_a^r	Departure time of trip r on arc a
e^r	Earliest possible departure time for trip r
ℓ^r	Latest possible arrival time for trip r
$\bar{\sigma}^r$	Maximum staggering applicable to trip's r departure
$\alpha_a^{r,r'}, \beta_a^{r,r'}, \gamma_a^{r,r'}$	Logical variables checking concurrent presence of trips (r, r') on arc a
ε	Small constant
M_i	Large constant

A3.3 Example of tighter time windows computation

As discussed in Section 3.3.2, examining one arc at a time inevitably overestimates the size of the time windows. This occurs because the current method ignores the travel continuity constraints that trips must respect when transitioning from one arc to the next. We clarify this limitation with the following example. Consider two consecutive arcs, a_1 and a_2, each with unit free-flow travel time. A conflict on either arc causes a unit delay for the trip behind. We analyze two trips, r and r', both traversing a_1 and a_2. Trip r can start on a_1 within $[0, 1]$, while trip r' is fixed to start at 0.5. Using the current procedure, which disregards upstream arcs, we obtain the time windows reported respectively in Tables 3.3-3.4.

Trip	Departure	Arrival
r	$[0, 1]$	$[1, 3]$
r'	0.5	$[1.5, 2.5]$

Table 3.3: Current time windows on a_1.

Trip	Departure	Arrival
r	$[1, 3]$	$[2, 5]$
r'	$[1.5, 2.5]$	$[2.5, 4.5]$

Table 3.4: Current time windows on a_2.

We now recompute the time windows by incorporating upstream constraints on a_1. This requires distinguishing two cases:

Case 1: Trip r starts on a_1 within $[0, 0.5]$. Time windows reported respectively in Tables 3.5-3.6.

Trip	Departure	Arrival
r	$[0, 0.5]$	$[1, 1.5]$
r'	0.5	2.5

Table 3.5: Time windows on a_1 for Case 1.

Trip	Departure	Arrival
r	$[1, 1.5]$	$[2, 2.5]$
r'	2.5	3.5

Table 3.6: Time windows on a_2 for Case 1.

Case 2: Trip r starts on a_1 within $(0.5, 1]$. Time windows reported in Tables 3.7-3.8.

Trip	Departure	Arrival
r	$(0.5, 1]$	$[2.5, 3]$
r'	0.5	1.5

Table 3.7: Time windows on a_1 for Case 2.

Trip	Departure	Arrival
r	$[2.5, 3]$	$[3.5, 4]$
r'	1.5	2.5

Table 3.8: Time windows on a_2 for Case 2.

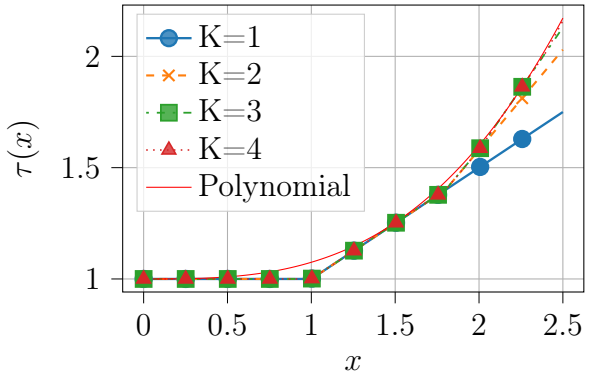
Combining both cases, we obtain the following tighter arrival time windows for a_2: $[2, 4]$ for trip r and $[2.5, 3.5]$ for trip r', compared to the previously computed intervals of $[2, 5]$ and $[2.5, 4.5]$, respectively.

A3.4 Travel time function parameterization

In Figure 3.11, we compare piecewise linear functions with K non-flat segments to assess their approximation error relative to a polynomial function modeling travel times under varying congestion levels. Here, x represents the flow to arc capacity ratio, while $\tau(x)$ denotes the corresponding travel time multiplier. Each function is constructed by sequentially introducing segments k at breakpoints $\hat{f}^k$ with progressively increasing slope μ^k. The selected slopes and breakpoints are $(\mu^1, \hat{f}^1) = (0.5, 1)$, $(\mu^2, \hat{f}^2) = (0.9, 1.8)$, $(\mu^3, \hat{f}^3) = (1.1, 2)$, and $(\mu^4, \hat{f}^4) = (1.4, 2.4)$. To assess the approximation error relative to the polynomial function, we report the mean absolute error (MAE) and root mean square error (RMSE). The results indicate diminishing marginal improvements as additional segments are introduced. Beyond $K = 3$, the reduction in the approximation error becomes negligible, such that choosing an approximation with three segments maintains a reasonable trade-off between accuracy and computational complexity.

A3.5 Optimality gaps

Figures 3.12–3.14 show the distribution of the optimality gap (Figure 3.12), the absolute value of the lower bound (Figure 3.13), and the distance between the upper bound and the lower bound (Figure 3.14) for both the LC scenario and the HC scenario, obtained when running our matheuristic with a time limit of one hour.



(a) Piecewise linear functions vs. polynomial.

K	MAE	RMSE
1	0.068	0.12
2	0.029	0.041
3	0.019	0.027
4	0.018	0.026

(b) Error metrics for piecewise functions.

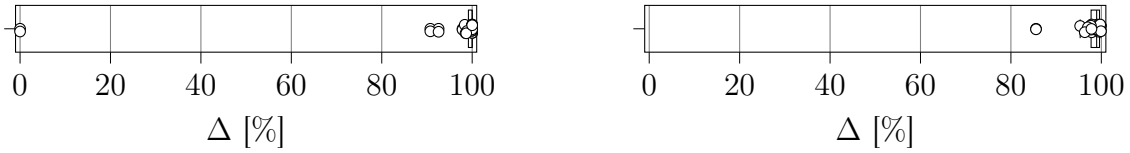
Figure 3.11: Comparison of piecewise linear functions and their error metrics.

Focusing on the relative optimality gaps in Figure 3.12, we observe a bipolar distribution in which optimality gaps accumulate either around 0% or around 100% for the LC scenario. For the HC scenario, most relative optimality gaps range between 90% and 100%. At first sight, these optimality gaps look anything but satisfying in terms of algorithmic performance, but unravel as follows at second sight: as our objective is to minimize the total congestion-related delay in the system, a trivial lower bound on our objective value is an objective value of zero. Clearly, our problem bears a huge degree of symmetry, which leads to weak lower bounds that cannot be significantly improved in our MILP formulation, see Figure 3.13. In such cases, an absolute distance of even a few minutes of delay left in the system can create a large optimality gap.

To support our claim that our matheuristic finds good solutions, we compare these optimality gaps to the delay reductions shown in Figure 3.6. In fact, a comparison to the uncontrolled solution provides more insights compared to a lower bound comparison: relating Figure 3.6 to Figure 3.12, it becomes obvious that the maximum improvement potential to reduce delay remains at 20% and 60% respectively for the LC and HC scenarios, although the optimality gap distribution shows values up to 100%.

Remark 3 (lower bound) *When minimizing the total congestion-related delay in a staggered routing setting, the relative optimality gap is an uninformative indicator of solution quality due to the nature of the objective function and the problem's weak lower bounds.*

In this context, Figure 3.14 reports the difference between the solutions found and the lower bound, indicating that our matheuristic founds good solutions despite large optimality gaps. Specifically, we observe an average distance between the upper (UB) and lower (LB) bound of 10 minutes and 20 hours for the LC and HC scenarios, respectively. As the instances exhibit respectively an average delay of 4 hours and 45 hours in an uncontrolled setting, the absolute distance between the upper and lower bounds appears



(a) LC scenario (b) HC scenario

Figure 3.12: Distribution of the optimality gap Δ [%] over all instances.

(a) LC scenario (b) HC scenario

Figure 3.13: Distribution of the absolute value of the lower bound LB [min] over all instances.

(a) LC scenario (b) HC scenario

Figure 3.14: Distribution of the distance between the upper and lower bound UB-LB [min] over all instances.

relatively small for the LC scenario and still satisfactory for the HC scenario, indicating that our matheuristic effectively reduces congestion via staggering.

4 Integrated Balanced and Staggered Routing in AMoD Systems

4.1 Introduction

The ride-hailing market has recently experienced substantial growth, fueled by evolving consumer preferences and technological innovations. Projections estimate that by 2028, ride-booking platforms will serve nearly 100 million users, driving the U.S. market to a value of approximately $55 billion (Statista, 2024).

While ride-hailing services offer convenient transportation options, they have been criticized for exacerbating urban congestion (Diao et al., 2021), primarily due to uncoordinated operations (Kondor et al., 2022) and drivers favoring personal route choices (Zanardi et al., 2023), which result in suboptimal travel patterns and overloaded networks. This inefficiency carries a dual cost: from a societal standpoint, it worsens congestion and its externalities, including prolonged delays, air and noise pollution, and broader public health impacts; from the operator's standpoint, it drives up operational costs through increased travel time, fuel use, and vehicle underutilization.

AMoD systems, centrally controlled fleets of self-driving vehicles offering door-to-door mobility solutions (Pavone, 2015), promise to overcome these limitations. Their central governance enables the coordination of vehicle flow in response to travel demand and traffic conditions, mitigating the inefficiencies associated with self-interested, uncoordinated fleets of human-driven vehicles. However, without effective routing policies, AMoD systems risk replicating the issues observed in traditional ride-hailing services, potentially further straining the street network (Oh et al., 2020). To address congestion and enhance operational efficiency, AMoD routing policies can employ two main coordination strategies: balanced and staggered routing. Balanced routing distributes AMoD flow across alternative road segments to alleviate congestion on popular routes and smooth overall traffic flow. Staggered routing complements this approach by strategically delaying vehicle departures to spread demand more evenly over time, alleviating temporal imbalances

---

This chapter builds on a paper co-authored with Gerhard Hiermann, Dario Paccagnan, Michel Gendreau, and Maximilian Schiffer, which is currently under review for publication. The paper was presented at the *Optimization Days (JOPT) 2025* in Montréal, Canada.

in network utilization.

Although complementary, these operational strategies lack a unified framework for integrated implementation in AMoD systems. On one hand, most existing studies that address AMoD routing under congestion rely on static models (cf. Zardini et al., 2022), which are well-suited for long-term planning but fall short in supporting AMoD operations for two key reasons: first, they cannot capture the real-time nature of fleet management, which must continuously adapt to evolving travel demand; second, they cannot accommodate departure time assignments, which require explicit temporal modeling. On the other hand, previous work on optimal staggering of AMoD departures lacks a routing control layer that leverages the additional travel time reductions enabled by staggering, thus treating route and departure time assignments in *isolation*.

In contrast, an *integrated* approach combines these strategies by moving the departure time decision to an earlier control stage. This shift influences route selection and can potentially outperform balanced and staggered routing methods when applied in isolation. However, this integrated optimization introduces additional combinatorial complexity and challenges the tractability of existing solution approaches. Against this background, we explore the untapped potential of integrating balanced and staggered routing strategies to coordinate autonomous fleets more efficiently under high travel demand. It introduces scalable algorithmic solutions to jointly optimize route assignment and departure timing, addressing the computational and operational complexity of large-scale, dynamic fleet coordination while accounting for congestion. We adopt this offline, deterministic framework to serve a critical dual purpose. First, it deliberately isolates the best-case system potential of integrated routing from the complexities of stochastic demand; second, it establishes a high-quality computational benchmark essential for evaluating the efficiency of future real-time, online opera- tions.

4.1.1 Contribution

This paper makes the following contributions. First, we formalize the problem of minimizing travel time in a network operated by a AMoD system using an integrated approach that combines balanced and staggered routing. Second, we formally show that the congestion model employed serves as an unbiased estimator of travel times under a discretized version of Vickrey's bottleneck model — a well-established framework in dynamic traffic assignment. Third, given that the staggered routing problem—optimizing AMoD departure times along fixed routes—is already NP-hard (Coppola et al., 2025b), we develop a scalable metaheuristic based on a LNS framework to solve the more general problem of jointly optimizing routes and departure times. We relax the assumption of piecewise-linear delay constraints and enable our algorithm to handle any convex, non-decreasing

delay function. Fourth, we go beyond prior implicit representations of baseload traffic in the staggered routing literature by explicitly modeling a realistic number of trips, approximating real-world congestion conditions. Existing approaches typically rely on a MILP formulation tightened through trip time windows; however, the large number of resulting constraints limits the number of trips that can be explicitly tracked. To circumvent this, prior work approximates baseload traffic by imposing arc capacity limits instead of modeling individual trips — a limitation we overcome by no longer utilizing a MILP formulation. Finally, we conduct a numerical study using real-world data from Manhattan, New York City, to examine the trade-offs between adjusting vehicle routes and departure times to minimize travel times and study network congestion. Our case study initially assumes the AMoD operator has full traffic control, showing that our algorithmic solutions reduce fleet delays by up to 25% and congestion delays by up to 35% compared to a selfish baseline. We then transition to a mixed-traffic setting, where AMoD vehicles serve only a fraction of the demand. This allows us to examine how varying the level of control affects overall network performance. In both welfare- and profit-oriented settings, centralized balanced and staggered routing consistently produce a win–win outcome, reducing travel times for both AMoD and non-AMoD traffic across all scenarios. Notably, controlling just 10% of the system's vehicles already achieves 25% of the maximum delay reduction, while 50% control yields 75%, with further gains becoming more marginal.

4.2 Related literature

Our work builds upon previous research aimed at mitigating congestion through strategic routing and timing of traffic. First, adopting a theoretical perspective, we highlight the intersections of this work with the field of DTA. Second, from an applicative perspective, we focus on control and operational strategies for AMoD systems.

Overview of DTA models The primary objective of traffic assignment is to model vehicular movement patterns based on predefined behavioral rules. A DTA model typically consists of a route and/or departure choice model combined with a traffic congestion model, so we review both components in sequence. Based on the available route and departure time choices, DTA models fall into one of three main categories: (1) pure departure time choice models (see, e.g., Hendrickson & Kocur, 1981; Lindsey et al., 2012; Vickrey, 1969), (2) pure route choice models (see, e.g., Carey & Watling, 2012; Levin, 2017; Long et al., 2013; Mounce & Carey, 2011), and (3) SRDTC models (see, e.g., Han et al., 2013; Huang & Lam, 2002; Jalota et al., 2023; Jauffred & Bernstein, 1996; Ran et al., 1996; Ziliaskopoulos & Rao, 1999).

Focusing on SRDTC models, which are closest in nature to our work, route and departure time choices typically follow either the dynamic user optimal principle—where travelers select routes and departure times to minimize their own travel costs (Friesz et al., 2011; Nie, 2010; Ukkusuri et al., 2012)—or the dynamic system optimal principle, which seeks to minimize total system-wide travel cost (Long et al., 2018). Efforts to bridge the gap between user equilibrium and system optimality in SRDTC models include tolling schemes (Arnott et al., 1990a; Liu & Nie, 2011; Yang & Meng, 1998), demand management and route guidance systems (Makridis et al., 2024; Menelaou et al., 2021), and signal control (Hu & Mahmassani, 1997). Despite these efforts, research on achieving dynamic system optimality or dynamic fleet optimality (Battifarano & Qian, 2023) through centralized control of departures and routes remains limited.

A complete DTA model combines travelers' behavioral choices with a congestion model that captures how traffic evolves over time along routes connecting origin-destination pairs. Commonly adopted congestion models include extensions of Vickrey's bottleneck model, which represent traffic formation and dissipation as a vertical queue at the bottleneck entrance, offering analytically tractable formulations (Li et al., 2020), and cell transmission models, which can capture complex dynamics such as spillback effects (Adacher & Tiriolo, 2018). However, applying these models to large-scale networks remains computationally demanding (Ziliaskopoulos et al., 2004). To address this, researchers have proposed macroscopic fundamental diagram-based models as aggregated tools for analyzing large-scale traffic behavior (Aghamohammadi & Laval, 2020). In this work, we introduce a discretized congestion model inspired by Vickrey's bottleneck framework, enabling explicit vehicle-level routing and timing while preserving computational tractability.

AMoD control & operations Most mesoscopic and macroscopic studies examining the impact of congestion in AMoD systems focus on strategic control settings, optimizing the routes of customer-carrying and rebalancing trips under steady-state conditions (see, e.g., Bahrami & Roorda, 2020; Bang & Malikopoulos, 2022; Estandia et al., 2021; Rossi et al., 2018, 2020; Salazar et al., 2020). While these approaches are valuable for long-term evaluation, they overlook the dynamic nature of AMoD operations, limiting their applicability to our setting.

In contrast, research that considers the dynamic aspects of AMoD operations, including studies on vehicle dispatching (see, e.g., Enders et al., 2023; Li et al., 2019a; Liang et al., 2022; Sadeghi Eshkevari et al., 2022; Tang et al., 2019; Xu et al., 2018; Zhang et al., 2017; Zhou et al., 2019), joint dispatching and rebalancing (see, e.g., Jungel et al., 2025; Tsao et al., 2018), request pooling (see, e.g., Alonso-Mora et al., 2017; Tsao et al., 2019), and fleet rebalancing (see, e.g., Gammelli et al., 2021; Iglesias et al., 2018; Jiao et al., 2021;

Liu & Samaranayake, 2022; Skordilis et al., 2021) rarely explicitly model congestion, as many rely on precise forecasts or deterministic assumptions for travel times.

In this work, we develop a scalable framework that explicitly models congestion and supports balanced and staggered routing for AMoD systems. Our model enables dynamic assignment of routes and departure times at the vehicle level, using a congestion model inspired by Vickrey's bottleneck formulation to provide a tractable representation of traffic dynamics.

4.2.1 Roadmap

The remainder of this paper is organized as follows. Section 4.3 introduces the problem setting for the integrated balanced and staggered routing problem and shows the relation between our congestion model and Vickrey's bottleneck model. Section 4.4 describes our metaheuristic algorithm developed to solve real-world instances. In Section 4.5, we outline our experimental design, and in Section 4.6, we present the results from our numerical study. Finally, Section 4.7 concludes the paper with remarks and future research directions.

4.3 Problem setting

We consider an AMoD operator tasked with coordinating a fleet of autonomous vehicles to serve a given set of trips within a fixed planning horizon. We focus on an offline setting in which the operator has full knowledge of all trip requests in advance. Each trip utilizes a single vehicle and can represent either on-duty activities (e.g., customer delivery) or off-duty activities (e.g., rebalancing). Our objective is to minimize total travel time by jointly optimizing route selection and trip departure times. This involves balancing trips across alternative (potentially longer in distance) routes and staggering departures beyond their originally requested times to prevent congestion and improve fleet efficiency.

We model the road network within which the fleet operates as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$, where the nodes $\mathcal{V}$ represent trip origins, destinations, and road intersections, and the arcs $\mathcal{A}$ correspond to road segments connecting these nodes. We consider a set of trips $\mathcal{R}$, each traveling between two distinct network nodes. Let a quadruple $(\mathcal{P}^r, \underline{e}^r, \ell^r, \bar{\sigma}^r)$ define a trip $r \in \mathcal{R}$, with $\mathcal{P}^r = \{p_1^r ... p_k^r\}$ denoting the alternative k routes – sequences of network arcs – that the vehicle can travel, $\underline{e}^r$ being the earliest departure from the trip's origin, ℓ^r being the latest feasible arrival at the destination, and $\bar{\sigma}^r$ being the maximum staggering time applicable to a trip's departure. A trip r traversing arc a experiences a travel time $\tau_a^r(f_a^r)$, which consists of two components: the free-flow travel time $\underline{\tau}_a$ and a congestion-induced delay $d(f_a^r)$. The delay depends on the number of trips f_a^r present on

arc a at the time r begins its traversal:

$$\tau_a^r(f_a^r) = \underline{\tau}_a + d(f_a^r), \tag{4.1}$$

where $d(f_a^r)$ is a convex, non-decreasing function of flow. A tuple $\pi = (p, s)$ defines a solution to this problem, where p and s denote the route and departure time assigned to each trip r, respectively. As we assume that vehicles do not idle once a trip has started, this information is sufficient to determine the travel time along each selected route and, consequently, the arrival time for every trip. We require a solution π to satisfy the following constraints, which we collect in the feasible set Π:

i) The route p^r assigned to each trip r must belong to $\mathcal{P}^r$.

ii) The start time s^r of each trip r must occur between $\underline{e}^r$ and $\underline{e}^r + \bar{\sigma}^r$.

iii) The combination of start times and routes in π must ensure that the arrival at the destination of each trip r occurs before ℓ^r.

In this setting, we seek for a solution $\pi^* \in \Pi$ that minimizes the total fleet travel time, defined as the sum of travel times incurred by each trip across the arcs of its assigned route:

$$\min_{\pi \in \Pi} \; T(\pi) = \sum_{p^r \in \pi} \sum_{a \in p^r} \tau_a^r(\pi), \tag{4.2}$$

where, with slight abuse of notation, we make explicit that the travel time τ_a^r experienced by trip r on arc a depends on all route and departure time assignments.

Discussion A few comments on our modeling approach are in order. First, we assume that only AMoD trips traverse the network, meaning the operator has full control over all trips in $\mathcal{R}$ to keep this basic problem setting concise. To extend the model to include background traffic, one can designate a subset of trips with routes and departure times not subject to optimization. This extension also enables the definition of two distinct objective functions: one that minimizes total system travel time, and another that minimizes only the travel time of the controlled trips. We present such an extended formulation in Section 4.6.

Second, incorporating route choice enhances behavioral realism beyond pure staggering models but increases computational complexity. To ensure scalability, we precompute a set of alternative routes—an approach aligned with route-based methods commonly used in dynamic traffic assignment (cf. Lo & Szeto, 2002). As detailed in Sections 4.4 and 4.5, we design these routes to support effective balancing while preserving realistic travel patterns within the network.

Finally, although stochastic demand modeling would better capture real-world uncertainty, we assume full knowledge of demand at the central controller. This assumption aligns with the exploratory focus of our study, which aims to quantify the *potential* benefits of integrated balanced and staggered routing in AMoD systems and to provide an upper bound on travel time reductions that can serve as a benchmark for online methods.

Relation to the discretized Vickrey's bottleneck model Consider an arc with nominal travel time τ_a. Let $f(t)$ denote the flow on this arc at time t, and let $\tau(t)$ represent the corresponding travel time, defined as:

$$\tau(t) = \tau_a + (\phi \cdot \tau_a)\, f(t), \tag{4.3}$$

where $\phi \in (0, 1]$ is a scalar parameter that quantifies the sensitivity of travel time to congestion. We claim that, under Poisson arrivals, there exists a unique choice of ϕ that renders (4.3) an unbiased estimator of travel times in the discretized Vickrey bottleneck model (cf. Otsubo & Rapoport, 2008). To establish this claim, we first review the key features of the discretized Vickrey bottleneck model and show that, under a stable Poisson arrival process, it is equivalent to an M/D/1 queueing system (cf. Thomopoulos, 2012). We then map our congestion model onto this queueing framework, allowing us to identify the unique value of ϕ that ensures an unbiased estimation of travel times.

Consider a finite set of trips traveling along a bottleneck road that connects a common origin to a common destination. The bottleneck has a nominal travel time τ_a and a capacity that restricts the number of vehicles simultaneously traversing it. We assume unit capacity, such that at most one trip can pass through the bottleneck at a time, while the remaining trips must wait in a FIFO queue. Figure 4.1 schematizes the evolution of the system. The travel time for a trip departing at time t follows:

$$\tau^{\mathrm{V}}(t) = \begin{cases} \tau_a, & \text{if } f(t) = 0 \\ \Delta\tau_a(t) + f(t) \cdot \tau_a, & \text{otherwise} \end{cases} \tag{4.4}$$

where $\Delta\tau_a(t)$ represents the remaining time for the traveling trip to leave the bottleneck.

Assume Poisson arrivals at the bottleneck with rate $\lambda \leq \tau_a^{-1}$. In this setting, (4.4) functions as a stable M/D/1 queueing system since the service time is constant and equal to τ_a. Let $\rho = \lambda \cdot \tau_a$ denote the traffic intensity of the system. The expected travel time at the bottleneck then follows:

$$\mathbb{E}[\tau^V(t)] = \tau_a + \frac{\tau_a \cdot \rho}{2(1-\rho)}. \tag{4.5}$$

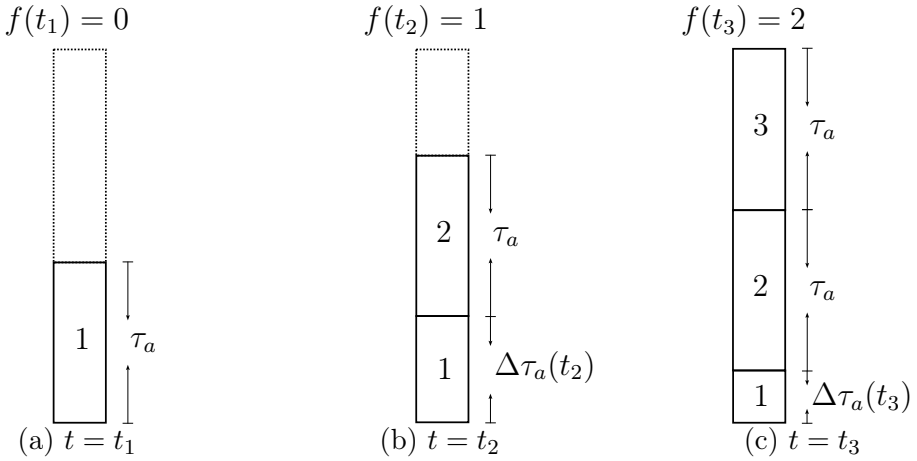


Figure 4.1: Evolution of a bottleneck at three distinct time steps, t_1, t_2, and t_3.

We are now ready to show that, with an appropriate choice of parameters, Equation (4.3) yields expected travel times that match those of the discretized Vickrey bottleneck model in Equation (4.5). We refer the reader to Appendix A4.1 for the proof.

Theorem 2 *Consider an arc with nominal travel time τ_a, where arrivals follow a Poisson process with rate $\lambda \leq \tau_a^{-1}$. Let the arc travel time be defined by Equation* (4.3)*. Then, by setting $\phi = (2-\rho)^{-1}$, the expected travel time under Equation* (4.3) *coincides with the expected travel time given by Equation* (4.5)*.*

This theorem offers the following interpretation. When $\rho = 1$, the system operates at capacity, and the average traversal time $\Delta\tau_a$ equals τ_a. In this case, setting $\phi = 1$ reproduces this behavior, ensuring that delay increases by τ_a per trip ahead. When $\rho < 1$, the average $\Delta\tau_a$ falls below τ_a, so setting $\phi = (2-\rho)^{-1}$ appropriately compensates for the reduced average delay.

Discussion To enable a tractable analytical characterization, the derivation relies on standard simplifying assumptions in bottleneck models, namely, Poisson arrivals and unit capacity, which allow the bottleneck dynamics to be represented as an M/D/1 queue. Here, unit capacity can be interpreted as a normalized abstraction of service rate, while the Poisson assumption enables a closed-form characterization of expected delays. Accordingly, our objective is to provide a principled structural justification for the linear delay term in (4.3), rather than to claim strict behavioral fidelity of the proposed model, thereby supporting its use as a tractable approximation within our framework.

4.4 Methodology

In this section, we present our methodology for solving large instances of Problem (4.2). Section 4.4.1 outlines the approach used to design alternative routes for balancing assignments. Section 4.4.2 introduces a mathematical formulation of the problem, including the notation and constraints that define feasible solutions. While this formulation serves as a valuable *descriptive* tool, it is computationally impractical for large-scale instances. Therefore, in Section 4.4.3, we propose a custom metaheuristic based on a LNS framework, designed to efficiently generate high-quality solutions at scale.

4.4.1 Alternative route design

Designing effective route alternatives is central to traffic balancing and congestion mitigation, yet it poses a trade-off between spatial diversity and route plausibility.

A k-shortest path formulation typically yields highly overlapping routes, concentrating flow on a limited set of arcs. While such routes may better reflect individual driver preferences, they restrict the controller's ability to redistribute traffic and therefore lead to systematically higher congestion in coordinated settings. Conversely, k-disjoint shortest paths maximize diversity but often introduce excessively long detours, which are unlikely to be selected under realistic time constraints and may distort congestion estimates.

Our goal is to construct a *controlled set of feasible alternatives* that enables effective system-level coordination while remaining within reasonable travel time bounds. To this end, we solve a k-shortest paths with limited overlap (KSPwLO) problem for each trip r, which enforces a bounded level of overlap between routes while prioritizing short path lengths.

Given a desired number of alternative routes k and a similarity threshold $\theta\%$, the goal is to compute a set $\mathcal{P}^r$ of k routes connecting the origin and destination of r, such that all route pairs are sufficiently dissimilar. Specifically, for all $\{p_i^r, p_j^r\} \in \binom{\mathcal{P}^r}{2}$, we require:

$$\mathrm{Sim}(p_i^r, p_j^r) = \frac{\sum_{a \in p_i^r \cap p_j^r} \ell(a)}{\min\left\{\ell(p_i^r), \ell(p_j^r)\right\}} \leq \theta,$$

with $p_i^r \cap p_j^r$ denoting shared arcs, and $\ell(\cdot)$ the arc or path length.

The similarity threshold θ should therefore be interpreted as a *design parameter* controlling the trade-off between diversification and the length of the detour. In particular, lower values of θ increase spatial spread at the cost of longer routes, while higher values recover solutions closer to k-shortest paths.

To solve the KSPwLO problem efficiently, we adopt the `SVP`$^+$ algorithm (Chondrogiannis et al., 2020), a performance-oriented heuristic designed for large-scale networks. `SVP`$^+$

restricts the search space to *single-via paths* (Abraham et al., 2013), obtained by concatenating a shortest path from the source to an intermediate node with another shortest path from that node to the target. The algorithm first computes the shortest paths from the source to all nodes and from all nodes to the target. It then evaluates candidate paths formed by combining these partial paths and ranks them by length. As a final step, the algorithm incrementally adds paths to the solution set if they are sufficiently dissimilar from those already selected.

The resulting path sets satisfy the following properties:

i) they always include the shortest path;

ii) all selected paths meet the similarity threshold θ;

iii) among the admissible alternatives, routes are as short as possible.

4.4.2 Mathematical programming formulation

Our goal is to formulate Problem (4.2) as a mathematical program. We begin by introducing the necessary variables, then formalize the constraints using an indicator function, and finally discuss the linearization of this function and explain why it renders the formulation intractable.

For every trip $r \in \mathcal{R}$, we define the binary variable $y_i^r \in 0, 1$, which takes the value 1 when the i-th route serves trip r and remains 0 otherwise, with $1 \leq i \leq k$. To account for increased travel time due to congestion, we introduce a delay variable d_a^r for each arc appearing in the alternative routes $\mathcal{P}^r$ of trip r. This delay satisfies $d_a^r = d(f_a^r)$, where $d(f_a^r)$ is a convex, non-decreasing function of flow. We also introduce non-negative travel time variables $\tau_a^r \geq 0$ for the same set of arcs and trips. These variables take positive values only on arcs that belong to the selected route and are zero otherwise. To enforce this behavior, together with the objective function minimizing the sum of the travel time variables, we apply the following big-M constraints:

$$\tau_a^r \geq \tau_a + d_a^r - M\big(1 - y_i^r\big), \qquad \forall r \in \mathcal{R},\ i \leq k,\ a \in p_i^r$$

where M denotes a sufficiently large constant.

To model the temporal progress of each trip along its route, we introduce continuous variables $\underline{s}_a^r$ and $\bar{s}_a^r$ for each arc in every alternative route, representing the start and completion times of the traversal of arc a by trip r. Finally, we denote $\underline{p}_i^r$ and $\bar{p}_i^r$ as the

first and last arcs of route p_i^r, respectively. We formulate the problem as:

$$\min \sum_{r\in\mathcal{R}}\sum_{i\le k}\sum_{a\in p_i^r}\tau_a^r \tag{4.6a}$$

s.t.

$$\sum_{i=1}^{k} y_i^r = 1, \qquad \forall r\in\mathcal{R} \tag{4.6b}$$

$$d_a^r = d(f_a^r), \qquad \forall r\in\mathcal{R}, i\le k, a\in p_i^r \tag{4.6c}$$

$$\tau_a^r \ge \underline{\tau}_a + d_a^r - M(1-y_i^r), \qquad \forall r\in\mathcal{R}, i\le k, a\in p_i^r \tag{4.6d}$$

$$f_a^r = \textstyle\sum_{r'\in\mathcal{R}\setminus\{r\}} \mathbb{I}(\underline{s}_a^{r'} \le \underline{s}_a^r < \bar{s}_a^{r'}), \qquad \forall r\in\mathcal{R}, i\le k, a\in p_i^r \tag{4.6e}$$

$$\bar{s}_a^r = \underline{s}_a^r + \tau_a^r, \qquad \forall r\in\mathcal{R}, i\le k, a\in p_i^r \tag{4.6f}$$

$$\underline{e}^r \le \underline{s}_a^r \le \underline{e}^r + \bar{\sigma}^r, \qquad \forall r\in\mathcal{R}, i\le k, a=\underline{p}_i^r \tag{4.6g}$$

$$\bar{s}_a^r \le \ell^r, \qquad \forall r\in\mathcal{R}, i\le k, a=\bar{p}_i^r \tag{4.6h}$$

$$\underline{s}_a^r, \bar{s}_a^r, f_a^r, d_a^r, \tau_a^r \ge 0, \qquad \forall r\in\mathcal{R}, i\le k, a\in p_i^r \tag{4.6i}$$

$$y_i^r \in \{0,1\}, \qquad \forall r\in\mathcal{R}, i\le k \tag{4.6j}$$

The objective in (4.6a) minimizes the total travel time of the fleet. Constraint (4.6b) ensures the selection of exactly one route for each trip. Constraints (4.6c)-(4.6d), together with the definitions of the variables d_a^r and τ_a^r in (4.6i), calculate the travel time for trips on the arcs of their selected routes and ensure that travel time is zero for non-selected routes. Constraint (4.6e) determines the number of trips encountered by a trip r upon entering arc a, using the indicator function $\mathbb{I}(\cdot)$. Constraint (4.6f) propagates the departure times of trips along their routes. Constraints (4.6g)-(4.6h) enforce time window restrictions for each trip. Finally, Constraints (4.6i)-(4.6j) define the domains of the decision variables.

Directly solving the formulation presented is impractical due to the presence of the indicator function in (4.6e). To address this, we replace it with an additional set of big-M constraints, obtaining a mixed-integer convex program. For a detailed description of this approach, we refer the reader to Section 3.1 of Coppola et al. (2025b), which provides a procedure that applies almost directly to our extended setting. The key distinction in this case lies in the requirement to define a set of big-M constraints for all pairs of trips that may *potentially* traverse a given arc, that is, for trips for which the arc belongs to at least one alternative route. Even under a simple specification of $d(f_a^r)$, such as a linear or piecewise linear function (resulting in a mixed-integer linear program), the number of big-M constraints required for linearization is significantly higher compared to a setting with fixed routes, rendering the formulation computationally intractable. Therefore, rather than attempting to solve this MILP directly, we develop a metaheuristic approach capable of finding feasible solutions without the need to explicitly define each of these constraints.

4.4.3 Algorithm

To solve large instances of Problem (4.2), we develop a custom metaheuristic based on a destroy-and-repair scheme. At a high level, the algorithm optimizes routes and departure times for newly requested and yet-to-depart trips at each epoch, while treating active trips as fixed. The method generates and evaluates solution updates within timeframes compatible with practical fleet management through an efficient evaluation procedure. The primary focus of this method is to obtain an offline benchmark for centralized traffic coordination, in which the algorithm optimizes over the full set of trips in the planning horizon. The proposed LNS framework can, however, be naturally embedded in a rolling-horizon scheme for real-time operations, where decisions are periodically updated by optimizing only newly arriving trips within a moving time window. Exploring this extension on a more sophisticated online algorithm is left for future work. To solve large instances of Problem (4.2), we develop a custom metaheuristic algorithm. Algorithm 6 outlines the overall structure of our solution approach, while Figure 4.2 illustrates the hierarchical dependencies among the algorithm's operators. Specifically, the algorithm:

i) Builds a reactive dynamic user optimum (RDUO).

ii) Builds a solution that greedily routes trips to minimize the system's total travel time.

iii) Chooses the better of i) and ii) to initialize a LNS for further improvement.

These solution-finding modules rely on several intermediate-level operators:

i) An *insert* operator assigning trips to partial solutions.

ii) A *local search* that adjusts routes and departure times for inserted trips.

Algorithm 6 Framework overview

Input: Set of trips $\mathcal{R}$, parameters for LNS and routing

1: $\pi_{\text{RDUO}} \leftarrow$ BUILDRDUO($\mathcal{R}$)
2: $\pi_{\text{greedy}} \leftarrow$ GREEDYASSIGNMENT($\mathcal{R}$)
3: $\pi \leftarrow \arg\min\{C(\pi_{\text{RDUO}}), C(\pi_{\text{greedy}})\}$
4: $\pi \leftarrow \text{LNS}(\pi)$
5: **return** π

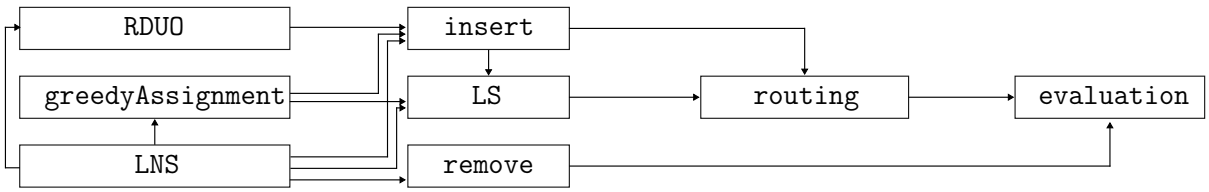


Figure 4.2: Algorithm operators in hierarchical order. Solution-finding modules (left) generate or improve solutions by invoking intermediate-level operators (center), which, in turn, rely on lower-level logic for routing trips and evaluating the resulting solutions (right).

iii) A *remove* operator that eliminates trips from a solution.

The *insert* and *local search* operators share a common low-level logic for identifying route and departure time combinations. After any insertion or removal, the algorithm evaluates the updated solution. Since evaluation constitutes the most computationally expensive step, we develop an efficient procedure to perform it. As we allow the algorithm to explore intermediate infeasible solutions, the algorithm ranks solutions π using the following cost function:

$$C(\pi) = D(\pi) + \alpha \cdot \mathrm{I}(\pi),$$

where $D(\pi)$ is the *total delay* and $\mathrm{I}(\pi)$ is the *total infeasibility* associated with π, and α controls the penalty applied to infeasibility. The total delay $D(\pi)$ measures the excess travel time relative to the shortest possible free-flow travel times. It consists of two components: a *detour delay*, the extra time from selecting routes longer in distance than the shortest routes, and a *congestion delay*, the additional time caused by traffic congestion along the chosen routes. The infeasibility term $\mathrm{I}(\pi)$ quantifies the total excess travel time of trips arriving later than their latest allowed arrival times.

The remainder of this section is organized as follows. First, we describe the solution-finding modules in Section 4.4.3, followed by the intermediate operators in Section 4.4.3. Finally, we present the lower-level operators in Section 4.4.3.

Solution-finding modules

Reactive dynamic user optimum The RDUO models users departing at their preferred times and independently selecting routes that minimize their travel time based on prevailing traffic conditions at the time of departure. As users do not account for future congestion, their chosen routes remain fixed once the journey commences. The RDUO computation starts with an empty solution $\pi = (p, s)$, where $p = \emptyset$ and $s = \emptyset$ denote the initial route and departure time assignment sets. The procedure inserts trips sequentially, in order of increasing departure times. For each trip, $s \leftarrow s \cup \underline{e}^r$ and $p \leftarrow p \cup p_e^r$, where p_e^r minimizes travel time at $\underline{e}^r$ given previously assigned trips. We provide the pseudocode in Appendix A4.2.

Greedy assignment The greedy assignment sequentially adds trips to an initially empty solution, selecting staggering and routing options to minimize total system travel time. As in the RDUO, the algorithm starts from an empty solution and inserts trips one at a time. This process may yield infeasible solutions, as the greedy insertion of early trips can cause later trips to violate their latest arrival times. To address this, the algorithm prioritizes the insertion of trips with the most restrictive latest arrival times, thereby promoting feasibility while still minimizing total delay. If infeasibilities remain

Algorithm 7 largeNeighborhoodSearch

Input: Set of trips $\mathcal{R}$, π_{greedy}, π_{RDUO}, removal pool size x, destruction percentage y, max cycles z

1: $\pi \leftarrow \arg\min\{C(\pi_{\text{RDUO}}), C(\pi_{\text{greedy}})\}$
2: **while** True **do**
3: `improved` $\leftarrow$ `False`
4: **for** *operator* in {`COSTLY_TRIPS, UNTOUCHED_TRIPS`} **do**
5: $\tilde{\pi} \leftarrow \pi$ ▷ Best candidate for this operator
6: $\mathcal{R}_{\text{p}} \leftarrow$ selectRemovalPool($\pi, x,$ *operator*)
7: **for** i in $\{1 \dots z\}$ **do**
8: $\mathcal{R}_{\text{d}} \leftarrow$ randomSample($\mathcal{R}_{\text{p}}, y$)
9: **for** *order* in shuffle(`EARLIEST, LATEST, DELAY`) **do**
10: $\pi' \leftarrow$ remove($\pi, \mathcal{R}_{\text{d}}$)
11: $\pi' \leftarrow$ insert($\pi', \mathcal{R}_{\text{d}},$ *order*)
12: **if** $C(\pi') < C(\pi)$ **then**
13: $\pi \leftarrow \pi'$
14: `improved` $\leftarrow$ `True`
15: **goto** line 6 ▷ Select new removal pool with this operator
16: **else if** $C(\pi') < C(\tilde{\pi})$ **then**
17: $\tilde{\pi} \leftarrow \pi'$
18: $\mathcal{R}_{\text{c}} \leftarrow$ selectChangedTrips($\tilde{\pi}, \pi$)
19: $\pi' \leftarrow$ localSearch($\tilde{\pi}, \mathcal{R}_{\text{c}}$)
20: **if** $C(\pi') < C(\pi)$ **then**
21: $\pi \leftarrow \pi'$
22: `improved` $\leftarrow$ `True`
23: **if not** `improved` **then**
24: **break** ▷ No improvement in any operator
25: **return** π

at the end of the insertion procedure, the algorithm invokes a *local search*, which assigns new route and departure time combinations to late-arriving trips as a repair mechanism. We include the corresponding pseudocode in Appendix A4.2.

Large neighborhood search The LNS iteratively improves a solution by progressively selecting destroy and repair operators applied to subsets of trips, thereby gradually exploring better combinations of trip routes and departure times. Algorithm 7 summarizes the steps of the LNS. The algorithm begins by selecting the lower-cost solution between the RDUO and the greedy assignment as the *incumbent* (l.1). Each iteration begins by selecting the *remove* operator, applying one of the following strategies:

i) removal of the trips with the highest individual cost;

ii) removal of the trips scheduled earliest along their shortest-distance route.

These two operators target distinct classes of trips, yet both offer strong potential for improvement: the first removes the most costly trips that contribute significantly to the overall solution cost, while the second focuses on reassigning trips with untapped potential for improved balancing or staggering. After the operator selection, the algorithm builds a removal pool (l.6) consisting of $x\%$ of all trips, along with any infeasible ones. Up to

z attempts (l.7), it randomly samples $y\%$ of the pool (l.8) and removes the selected trips from the current solution (l.10). It then uses the *insert* operator to reinsert the removed trips, which explores up to three insertion orders applied in random sequence (l.9–11):

- smallest earliest departure first,
- largest earliest departure first,
- highest delay first.

If any reinsertion produces a new incumbent solution, the algorithm immediately restarts the destroy-repair cycle by selecting a new removal pool using the last remove operator invoked (l.13–15). If no incumbent updates occur after z cycles, the algorithm invokes the *local search* procedure to improve the quality of the best solution found from this phase (l.18–22). While effective, the local search is computationally expensive, as it operates on full solutions. Therefore, during its first invocation, the algorithm applies the local search to all trips in $\mathcal{R}$, and restricts it in subsequent calls to trips whose delay or infeasibility has changed since the previous invocation. To support this, the algorithm maintains two vectors storing each trip's total delay and infeasibility, updated after every *local search* call.

After completing the local search, the algorithm proceeds to the next destroy operator. If at least one incumbent update occurs after evaluating all destroy operators, the algorithm restarts from the first operator (l.22). Otherwise, it terminates (l.24) and returns the best solution found (l.25).

To help the LNS balance feasibility and solution quality, we utilize an adaptive infeasibility weight α, which penalizes constraint violations in the solution cost. Starting with α equal to ten, we divide it by ten if ten consecutive *insert* calls yield feasible solutions, thereby reducing the penalty and encouraging exploration near the boundary of the feasible region. Conversely, if ten consecutive calls result in infeasible solutions, we multiply α by ten to guide the search back toward feasibility. To prevent both excessive penalization and loss of feasibility, we constrain α to the interval $[10^{-2}, 10^{3}]$.

Intermediate-level operators

Insert The *insert* operator takes as input a partial solution, a set of trips to insert, and a rule specifying the insertion order. Operating on one trip at a time, it tests and evaluates a series of alternative route and departure time combinations $(p^r, \underline{s}_r)$, identified using the low-level *routing* logic. It selects the combination yielding the lowest cost and updates the solution by adding the chosen route and departure time, updating $p \leftarrow p \cup p^r$ and $s \leftarrow s \cup \underline{s}_r$.

Local search The *local search* operator takes as input a solution π and reassigns routes and departure times for a specified subset of its trips. Unlike *insert*, it does not alter the number of trips in the solution. Proceeding in order of earliest permissible departure times $\underline{e}^r$, the operator iteratively generates a set of alternative route and departure time combinations $(p^r, \underline{s}_r)$ for each trip using the low-level *routing* logic. It evaluates all combinations, selects the one with the lowest cost, and updates the solution by replacing the trip's current route and departure time accordingly.

Remove The *remove* operator takes as input a solution and a set of trips to remove. For each trip, the operator deletes the corresponding route and departure time from the solution, updating $p \leftarrow p \backslash p^r$ and $s \leftarrow s \backslash \underline{s}_r$. After removing all specified trips, it evaluates the resulting solution.

Low-level operators

Evaluation Every modification to a solution — whether a trip insertion, removal, departure time shift, or trip rerouting — alters the solution cost, which the algorithm must quantify. Computing $C(\pi)$ requires assessing both the congestion experienced by trips and any late arrivals at their destinations. To this end, the algorithm assigns each solution a *schedule* S, which specifies, for each trip, the set of arc departure times $\underline{s}_a^r$ determined by the selected start times and routes in π. To determine the actual travel time of a trip r on arc a, the algorithm first calculates the flow on the arc, f_a^r, and then applies Equation (4.1). The algorithm calculates f_a^r by counting the number of *conflicts* with other trips r' on the same arc. A conflict occurs when r departs from a after r' has started but before r' has completed its traversal of the arc, including any delays; that is, when $\underline{s}_a^{r'} \leq \underline{s}_a^r < \bar{s}_a^{r'}$. Previous work on staggered routing proposes the constructSchedule procedure to compute a schedule from scratch; we provide its details in Appendix A4.3. While this function still finds application in our algorithm—for instance, after the *remove* operator deletes a subset of trips from the current solution— it becomes inefficient to evaluate a large number of new assignments, as it rebuilds the entire schedule even when the change affects only the trip under consideration. Algorithm 8 outlines our proposed approach for efficiently updating the schedule during conflict resolution. The core challenge that the algorithm addresses lies in identifying how a modification to a trip's route or departure time propagates through the schedule. In the best case, the change affects only the modified trip; in the worst case, it triggers a cascade of updates that shift the departure times of many other trips. Our method extends the updateSchedule procedure from previous work on staggered routing by supporting multiple route alternatives and introducing further refinements to improve computational efficiency.

To initiate the search, we begin with an existing schedule that serves as a benchmark for evaluating changes resulting from start time or route adjustments. At the core of the algorithm is a priority queue Q, initially empty. This queue holds tuples that include trip-arc-departure labels $(r, a, \underline{s}_a^r)$. To manage Q, we sort the tuples in ascending order based on their start times $\underline{s}_a^r$. We fill the queue with all trips potentially impacted by modifications in their scheduled departures (l.1). The approach to populating this queue varies based on the specific operation being executed:

i) *Trip insertion or staggering:* We add an initial tuple containing the trip, the first arc of its assigned route, and its scheduled departure time.

ii) *Trip rerouting:* We add all trips affected by the change on the previous route to the queue, along with a tuple for the rerouted trip at its earliest entry time on the new route. Specifically, for each arc a along the previous route, we insert $(r', a, \underline{s}_a^{r'})$ for every trip r' that conflicts with the rerouted trip in the original schedule.

We start processing trips in Q (l.3) by checking whether the trip's updated departure time $\underline{s}_a^r$ creates a change in the arc entry times of another trip r' (l.4). In this case, we insert r' into Q. If this insertion disrupts the queue order — specifically, if $\underline{s}_a^{r'} < \underline{s}_a^r$ — we flag the conflict by setting $r' \neq$ FALSE, reinsert $\underline{s}_a^r$ into Q, and restart the evaluation from line 5 to 6. If $\underline{s}_a^r$ does not induce changes ($r' =$ FALSE) in other trips, we recursively propagate the change in trip r's arc entry times on subsequent arcs until the trip reaches its destination (l.7-13). Our overall update ends once Q is empty and all arc entry times have been correctly adjusted.

To efficiently determine which trips to push to Q and count the resulting conflicts when processing trip r on arc a, we maintain auxiliary sets R_a, which store the labels of all trips currently assigned to arc a, sorted by departure time. We partition this set into:

i) R_a^{I}: labels of *inactive* trips, i.e., those not yet enqueued in Q.

Algorithm 8 updateSchedule

Input: Schedule S, solution π

1: Q $\leftarrow$ getChanges(S, π)
2: **while** Q is not empty **do**
3: $(r, a, \underline{s}_a^r) \leftarrow$ Q.pop()
4: $S, r' \leftarrow$ processTrip$(r, a, \underline{s}_a^r)$
5: **if** $r' \neq$ FALSE **then**
6: Q $\leftarrow$ insertTrips$(r, r', a, \underline{s}_a^r, \underline{s}_a^{r'})$
7: **else**
8: **while** $r' =$ FALSE **do**
9: $a' \leftarrow$ getNextArc(r, a, π)
10: $S, r' \leftarrow$ processTrip$(r, a', \underline{s}_{a'}^r)$
11: $a \leftarrow a'$
12: **if** $r' \neq$ FALSE **then**
13: Q $\leftarrow$ insertTrips$(r, r', a, \underline{s}_a^r, \underline{s}_a^{r'})$
14: **return** S

ii) R_a^{A}: labels of *active* trips, i.e., those already present in Q.

Initially, $R_a^{\mathrm{I}} = R_a$ and $R_a^{\mathrm{A}} = \emptyset$. As the algorithm activates new trips, it removes their labels from each R_a^{I} for all arcs in their route and adds them to the corresponding R_a^{A}. We deliberately avoid maintaining R_a^{A} in sorted order, since active departure times may change multiple times due to propagation effects, making constant re-sorting computationally expensive. We then compute the flow on arc f_a^r as:

$$f_a^r = \underline{f}_a^r + \bar{f}_a^r,$$

where $\underline{f}_a^r$ and $\bar{f}_a^r$ represent the flows contributed by inactive and active trips, respectively. Since R_a^{A} is unsorted, we evaluate $\bar{f}_a^r$ by iterating over all active trips:

$$\bar{f}_a^r = \sum_{r' \in R_a^{\mathrm{A}} \setminus \{r\}} \mathbb{I}(\underline{s}_a^{r'} \leq \underline{s}_a^r < \bar{s}_a^{r'}).$$

Conversely, we exploit the sorted structure of R_a^{I} to compute $\underline{f}_a^r$ through an efficient counting procedure: we locate the latest trip r' in R_a^{I} with $\underline{s}_a^{r'} < \underline{s}_a^r$ and traverse the set backward from that position, incrementing a counter at each step, until we find a trip with $\bar{s}_a^{r'} \leq \underline{s}_a^r$. When the FIFO property does not hold on an arc, the procedure may yield occasional inaccuracies in delay calculations. We therefore repair the schedule upon completion of each operator by invoking constructSchedule. Finally, the algorithm calculates the corresponding delay using the delay function $d(f_a^r)$ defined in Equation (4.1). The algorithm adds trip r' to Q if an update to r either introduces a new conflict or resolves an existing one between r and r'. To identify such trips efficiently, we leverage the sorted structure of R_a^{I}. We define four indices in R_a^{I}, each representing the insertion position of r based on:

i) its departure time *before* the update (i_{old}).
ii) its departure time *after* the update (i_{new}).
iii) its arrival time *before* the update (j_{old}).
iv) its arrival time *after* the update (j_{new}).

Trips located between i_{old} and i_{new} (departure range) or between j_{old} and j_{new} (arrival range) become active, as their conflict status with r has changed. Trips outside both ranges remain inactive. Those within the intersection of the two ranges also stay inactive, since their conflict status with r is unaffected. Figure 4.3 illustrates these cases with an example.

Routing This operator identifies new combinations of start times and routes, either to insert a trip with minimal impact on the solution cost (*insert*) or to improve an existing

assignment (*local search*). First, it iteratively refines the departure time on the trip's initial route to maximize the number of resolved conflicts. As an additional refinement step, the method removes portions of the applied departure shifts that do not contribute to reducing the solution cost, thereby promoting parsimonious staggering. Then, it reroutes the trip as early as possible on each alternative route and repeats the iterative start time identification procedure. After evaluating all routes, it selects the route–departure time combination that yields the lowest cost. Figure 4.4 illustrates this process using a small example.

Given a trip r, the algorithm analyzes conflicts along its current route p^r on an arc-by-arc basis, based on its current departure time. On each arc a of the route, the algorithm evaluates whether delaying r can resolve conflicts without introducing new ones. These evaluations are performed independently for each arc and ignore how staggering affects delays on preceding arcs, which may, in turn, influence downstream conflicts. As a result, the conflict-based evaluation serves only as a heuristic proxy for the true impact on overall solution quality.

To detect conflicts efficiently, the algorithm uses the sorted set R_a, which orders trip labels on arc a by their departure times. It locates r in R_a and identifies the immediately preceding trip r'. If $\bar{s}_a^{r'} < \underline{s}_a^r$, no overlap occurs, and the algorithm proceeds to the next

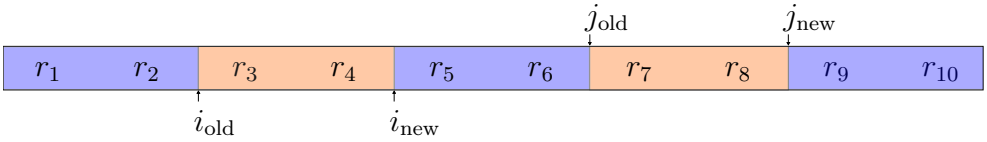
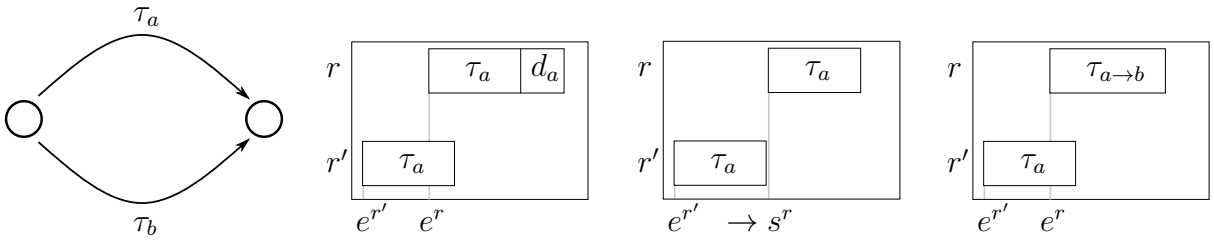


Figure 4.3: Identification of trips to activate on an arc, given a set of labels sorted by departure time. The indices i_{old} and i_{new} mark the previous and updated departure times of trip r, while j_{old} and j_{new} correspond to its previous and updated arrival times. Trips before (r_1, r_2), after (r_9, r_{10}), or overlapping (r_5, r_6) remain unaffected (highlighted in blue). The algorithm pushes the remaining trips (highlighted in orange) to Q because their conflict status with r changes: either the update resolves a previous conflict (r_3, r_4) or introduces a new one (r_7, r_8).



(a) Alternative routes (b) Initial schedule (c) Staggering (d) Rerouting

Figure 4.4: Examples of new start times and route identification on alternative routes a and b, with $\tau_a < \tau_b$.

arc. Otherwise, it delays r just enough to eliminate the conflict, as long as the new departure time remains within its feasible window:

$$s^r \leftarrow \min\left(s^r + \left(\bar{s}_a^{r'} - \underline{s}_a^r\right), \underline{e}^r + \bar{\sigma}^r\right).$$

If trips on the arc follow a FIFO discipline, resolving the conflict with r' also eliminates all earlier conflicts on the arc. While this assumption does not always hold, the algorithm adopts it as a computational shortcut. After updating the departure time of r, the algorithm first estimates its new arrival time, incorporating both the applied staggering and any delay reductions due to resolved conflicts with preceding trips. It then scans forward through R_a to detect changes in conflict status with subsequent trips r'. A new conflict arises if $\bar{s}_a^r > \underline{s}_a^{r'}$, whereas no such conflict existed before the update. Conversely, a conflict is resolved if the reduced delay leads to $\bar{s}_a^r \leq \underline{s}_a^{r'}$. The algorithm aggregates all resolved conflicts as f_a^-, and all newly introduced conflicts as f_a^+, and computes the net conflict gain as:

$$\Delta f_a = f_a^- - f_a^+.$$

This value heuristically indicates the benefit of setting the trip's start time to s^r on arc a. After evaluating all arcs, the algorithm selects the departure time candidate with the highest net conflict gain Δf_a. If the adjustment improves the solution, the algorithm accepts it and continues refining the start time on the same route until it finds no additional improvement. Before concluding the analysis of the current route, the method trims any part of the applied departure shift that does not reduce the solution cost. This refinement brings two main advantages: it unlocks further staggering opportunities for the same trip in later iterations and minimizes overlap with downstream trips on shared arcs. To compute the maximum feasible forward shift for a trip, we examine each arc along its path and determine how much earlier it can depart without introducing new conflicts. For each arc a, we define R'_a as the set of trips $r' \in R_a$ such that $\underline{s}_a^{r'} < \underline{s}_a^r$. If $R'_a \neq \emptyset$, we compute the shift to apply to r's departure time to match it with the latest departure or arrival time among these preceding trips as follows:

$$\Delta s(a) = \underline{s}_a^r - \max_{r' \in R'_a}\left\{\underline{s}_a^{r'},\ \bar{s}_a^{r'}\right\}.$$

We shift the trip's departure time forward by the smallest $\Delta s(a)$ across all arcs, as long as the resulting time respects feasibility constraints. Formally:

$$s^r \leftarrow \max\left(\underline{e}^r,\ s^r - \min_{a \in p^r} \Delta s(a)\right).$$

Once the procedure converges on the currently selected route, the algorithm resets r's departure time to $\underline{e}^r$ and proceeds to evaluate the next available route. For each alternative route, it applies the same start time selection strategy. After exploring all route options, the algorithm selects the route–departure time combination that yields the lowest overall cost.

To conclude this section, we note that by slightly modifying the logic of the *routing* operator, we can disable either the departure time assignment or the route assignment, resulting in variants that solve only the staggering or balancing problem, respectively. In our case study, we use these modes to isolate and compare the individual contribution of each component to the algorithm's overall performance.

4.5 Design of experiments

In this section, we present the setup for our case study, including the preparation of the street network, the parameterization of the travel time function, and the trip design. We conclude with a description of the algorithm specifics.

Network We obtain the street network for Manhattan, New York City, from OpenStreetMap (OpenStreetMap, 2024). Its dense, grid-like topology, with abundant alternative routes, makes it a suitable testbed for studying traffic balancing and congestion mitigation. Performance may differ in less dense or more hierarchical networks (e.g., suburban or highway-dominated settings), where route alternatives are limited and temporal coordination may follow different patterns. We therefore use Manhattan as a representative high-congestion setting and leave a systematic evaluation across network topologies to future work. As part of preprocessing, we remove motorway roads, retain only the shortest arc among parallel connections, and merge intersections within a 20-meter radius. We then restrict attention to the southernmost 10% of nodes and their connecting arcs, retaining the largest strongly connected component. The resulting network comprises 302 nodes and 734 arcs. Nominal travel times are computed assuming a constant speed of 20 km/h.

Trip design To model realistic traffic demand, we estimate the number of trips within our study area based on empirical data. During the evening peak on a typical fall weekday, approximately 70000 motorized vehicles — including both base load and taxi demand — enter and leave Manhattan's Central Business District (NYMTC, 2023). Assuming this approximates the total number of trips within Manhattan, and given that our study focuses on a subnetwork covering roughly 10% of the area, we scale this value

accordingly and consider 7000 trips per instance to be a reasonable estimate.

We consider 31 instances, one for each day of a month. For each instance, we construct the trip set $\mathcal{R}$ using the 2009 NYC Taxi & Limousine Commission dataset (NYCTLC, 2009), which provides origin, destination, and departure time information for individual taxi trips. We include all trips with both origin and destination located within the sub-network over the full 24-hour period of each day, using data from the months of January, March, May, and July (i.e., all 31-day months). To simulate realistic congestion, we compress the departure times into a one-hour window while preserving the original minute and second resolution. This procedure yields instances with an average of approximately 6000 trips, which aligns well with our target scale.

The design of alternative routes is central to the characterization of trip data in our case study. It affects both traffic balancing in our algorithmic solutions and congestion levels observed in the RDUO baseline, as trips in both settings select routes from the same predefined set. We solve the KSPwLO problem with a limited overlap setting the number of alternative paths k to five and the similarity threshold θ to 60%. This choice aligns with the goal of ensuring good network coverage through alternative routes, both to reproduce realistic congestion levels in RDUO and to facilitate traffic balancing, while keeping the number of alternatives small enough to maintain a manageable search space. We discuss this design choice in further detail in Appendix A4.4.

To complete the trip's characterization, we define each trip's latest arrival time ℓ^r based on the travel time observed in the RDUO solution, extended by 25% to account for congestion variability. We also set the maximum staggering time $\bar{\sigma}^r$ to 20% of the shortest-route free-flow travel time. These values are chosen to reflect moderate and practically plausible flexibility levels, balancing realism with sufficient room for coordination effects to emerge. We provide a detailed overview of all instances in Appendix A4.5.

Delay function parameterization We characterize the delay function $d(f_a^r)$ as a polynomial expression of the form:

$$d(f_a^r) = \tau_a \cdot \alpha \left[\left(\frac{f_a^r + \beta}{\tau_a} \right)^\gamma - \left(\frac{\beta}{\tau_a} \right)^\gamma \right],$$

where α controls the overall scaling of the delay, β introduces a baseline shift to regulate delays under low congestion, and γ is the polynomial exponent. The subtraction term ensures that the delay function evaluates to zero in the absence of other trips.

To parameterize this function, we used data from the TLC dataset, which includes pickup times, drop-off times, and trip distances. Assuming a constant free-flow speed of 20 km/h, we computed the free-flow travel time and compared it with the trip-recorded

durations. In the same network area considered for our experiments, the data indicate a median delay of approximately one and a half minutes during peak hours. Furthermore, commercial mapping services report that travel between the Financial District and Lower Manhattan experiences a delay of approximately 25% relative to the free-flow travel time during the peak hour.

Given these observations, we set the parameters of the delay function to $\alpha = 0.1$, $\beta = 35$, and $\gamma = 3$. This configuration yields instances with a RDUO characterized by an average total delay representing 12% of the total travel time, and individual trip delays with a median of approximately 0.7 minutes. Although these figures indicate that our instances may underestimate real-world congestion, they effectively serve to demonstrate the core concepts of our algorithm.

Note on baseline selection. While dynamic user equilibrium (DUE) or system optimum (SO) assignments provide tighter theoretical bounds, we adopt RDUO as our primary baseline to reflect the behavior of self-interested agents who lack perfect information about future network states. This reactive baseline captures the myopic routing decisions typical for current navigation-dependent urban traffic, providing a realistic benchmark for the marginal gains achievable through centralized coordination. A comparison to establish the absolute efficiency gap against computationally intensive models such as SO or DUE is reserved for future work.

Algorithm specifics We configure the LNS algorithm with the following parameters: an initial pool size of $x = 40\%$, a sample size of $y = 10\%$ drawn from this pool, and a maximum of $z = 2$ destroy-repair cycles before invoking the local search for intensification. These values reflect the outcome of a structured parametric analysis, which we report in detail in Appendix A4.6.

4.6 Results

In this section, we present the results of our numerical experiments. As a first step, we compare the staggering-only variant of our algorithm with the matheuristic proposed in Coppola et al. (2025b) to benchmark performance and demonstrate that the new algorithm consistently outperforms it across different levels of flexibility in trip start times.

We then move to the analysis of an idealized scenario in which we assume full control over all vehicles in the system. This setup allows us to establish an upper bound on the performance achievable through centralized routing. Within this framework, we isolate the individual effects of staggering and balancing on fleet and network performance and

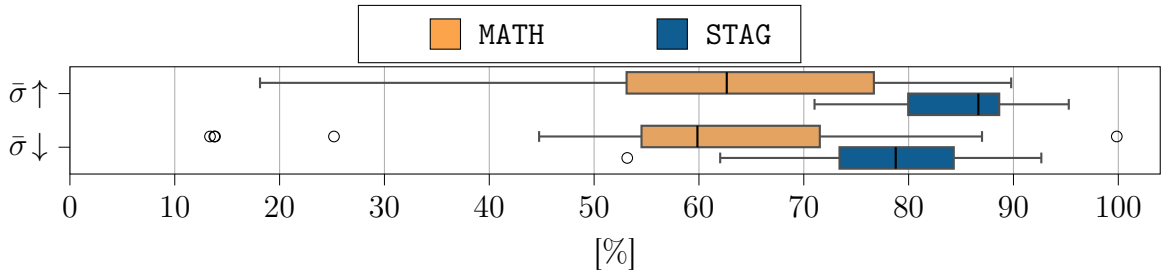


Figure 4.5: Relative delay reductions achieved by `STAG` and `MATH` under reduced ($\bar{\sigma}\downarrow$) and increased ($\bar{\sigma}\uparrow$) levels of flexibility in trip start times.

compare them to the performance of the fully integrated algorithm.

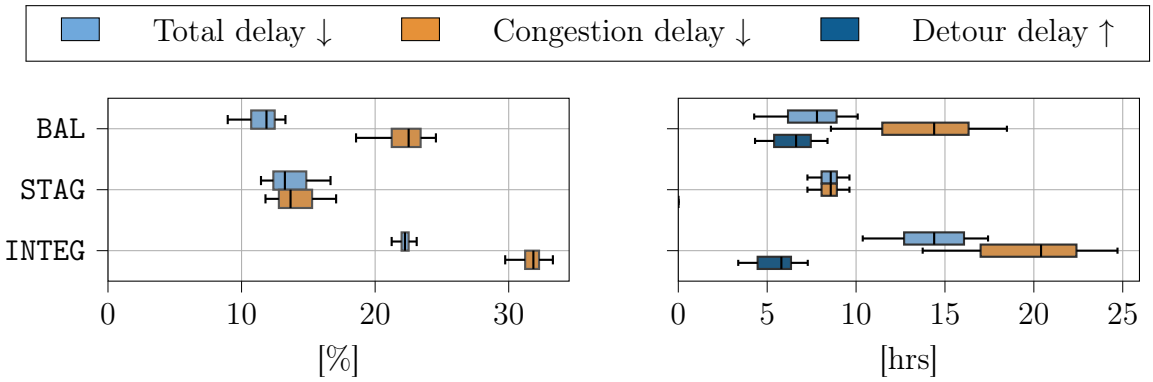
We then examine how to apply staggering and balancing in practice, assess their impact on individual travel and arrival times, and analyze how centralized control reshapes the spatial distribution of congestion. To complement this analysis, we consider more realistic scenarios in which only a portion of the traffic is under centralized control. We explore this from two perspectives: first, a municipality aiming to minimize total system-wide travel time; second, a profit-oriented AMoD service provider focused solely on minimizing the travel time of its own fleet.

We implemented our metaheuristic in C++ and compiled it using GCC version 11.1.0 with `-O3` optimization flags. We ran all experiments on an Intel(R) i9-9900 CPU at 3.1 GHz with 10 GB of RAM. The full implementation and experimental setup are available at https://github.com/tumBAIS/integ_bal_stag.

4.6.1 Comparison with matheuristic baseline

Figure 4.5 compares the performance of our staggering-only algorithm, `STAG`, with the matheuristic `MATH` from the literature, using relative delay reduction as the performance metric. We evaluate both methods on a set of 31 instances under two levels of departure-time flexibility: $\bar{\sigma}\downarrow$, where the maximum allowable start time shift $\bar{\sigma}_r$ is set to 10% of the nominal travel time along each trip's assigned route, and $\bar{\sigma}\uparrow$, where this shift is increased to 20%. For a detailed description of the experimental instances, we refer to Appendix A4.7.

`STAG` outperforms `MATH` in terms of median delay reduction by approximately 20 and 25 percentage points under $\bar{\sigma}\downarrow$ and $\bar{\sigma}\uparrow$, respectively. Interestingly, increasing flexibility from $\bar{\sigma}\downarrow$ to $\bar{\sigma}\uparrow$ brings only modest gains for `MATH` (around 3 percentage points in the median) and coincides with the loss of the sole optimal solution found. This highlights how `MATH` benefits from tightly bounded departure times, which enable leaner MILP models with fewer big-M constraints. However, as flexibility grows, these structural advantages vanish: the search space becomes more symmetric, relaxations weaken, and performance declines.



(a) Relative delay variations (b) Absolute delay variations

Figure 4.6: Delay metrics under full traffic control, illustrating the reductions in total and congestion delay as well as the detour-related increases resulting from balancing.

In contrast, `STAG` benefits from increased flexibility: its computational complexity and efficiency remain stable even with wider time windows, allowing it to improve median delay reduction by around ten percentage points and enhance robustness through stronger worst-case performance and reduced variance.

Result 6 *Across different levels of flexibility in trip start times,* `STAG` *consistently outperforms* `MATH`*, delivering higher delay reductions with reduced variance.*

Notably, the instances considered in this comparison already push `MATH` to its computational limits, since tracking pairwise trip interactions requires an exponential number of constraints, which rapidly increase memory usage. In contrast, `STAG` circumvents this exponential growth entirely, making it well-suited for large-scale applications. The combination of superior performance and scalability makes `STAG` the natural choice for solving the staggering-only variant of the problem in the remainder of this study, where the number of pairwise trip interactions is significantly higher than in the present benchmark.

4.6.2 Algorithmic performance

Figure 4.6 illustrates the performance of three variants of our algorithm on the instances described in Section 4.5: `BAL`, which applies only balancing operations; `STAG`, which only staggers trip departures; and `INTEG`, which combines both strategies. The figure reports total and congestion-induced delay reductions relative to RDUO under full traffic control, expressed in both percentage and absolute terms, along with the absolute detour delay introduced by balancing. All experiments terminated within a four-hour time limit.

Total delay reductions. When isolating individual mechanisms, we observe that balancing alone (`BAL`) yields a median delay reduction of 12% (approximately 8 hours). This result suggests that self-interested users, unaware of future congestion, often select suboptimal routes, while the balancing operator distributes traffic more efficiently across available paths. Staggering alone (`STAG`) achieves a comparable median delay reduction of 13% (approximately 9 hours), demonstrating that our greedy staggering strategy, which adjusts departure times to resolve arc-level conflicts without accounting for upstream congestion effects, serves as an effective heuristic proxy for identifying promising trip start times.

The fully integrated algorithm (`INTEG`) consistently delivers strong performance across all real-world instances, achieving total delay reductions consistently around 23%. In absolute terms, this corresponds to 10 to 17 hours of reduced travel time. Crucially, the integrated approach appears to combine the benefits of both rerouting and rescheduling in a near-additive manner, highlighting their complementary roles in mitigating delay.

Result 7 *The integrated algorithm achieves total delay reductions of 23%, translating into up to 17 hours of saved travel time across the entire system.*

Trade-offs between congestion and detour. From the operator's perspective, total delay reduction translates directly into shorter travel times and economic savings. Yet, it is equally important to assess network efficiency. Specifically, we assess the reduction in congestion delay, potentially at the cost of increased detour delay.

In `STAG`, all delay reduction corresponds to congestion relief, since routes remain fixed. Interestingly, balancing alone well mitigates congestion: `BAL` removes over 23% of congestion-induced delay in median, which translates to approximately 15 hours. However, this improvement comes with additional detour delay — about six hours on average. Remarkably, the integrated approach delivers an even more favorable trade-off. By leveraging staggering to reduce congestion without modifying routes, `INTEG` removes more congestion delay compared to balancing alone (up to 25 hours) while inducing less additional detour delays.

Result 8 *The integrated algorithm removes more congestion delay (up to 25 hours) than balancing alone (up to 18 hours), while introducing less detour delays (under 6 hours).*

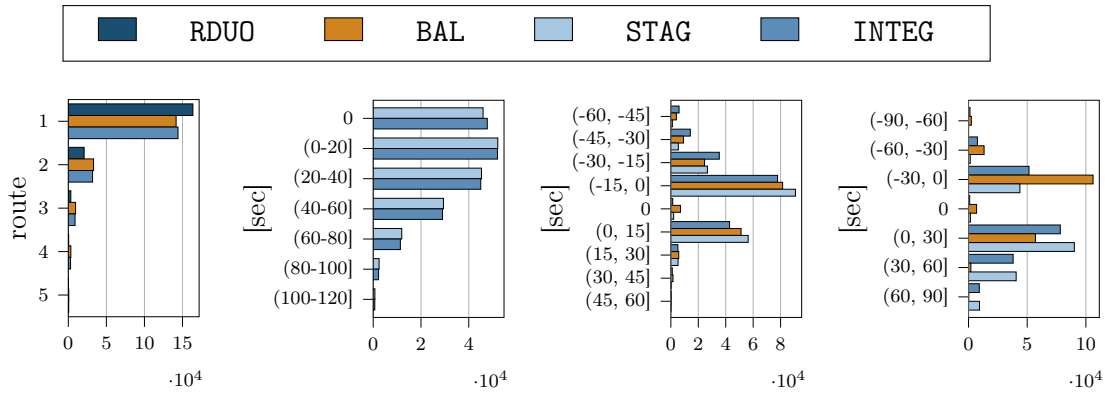
4.6.3 Structural analysis

In this section, we analyze the structural characteristics of the solutions generated by different control strategies. Figure 4.7 presents: (a) the route assignment distribution for

the baseline RDUO and for the rebalancing variants `BAL` and `INTEG`, (b) the frequency of staggering in `STAG` and `INTEG`, (c) travel time differences relative to the baseline, and (d) differences in arrival times.

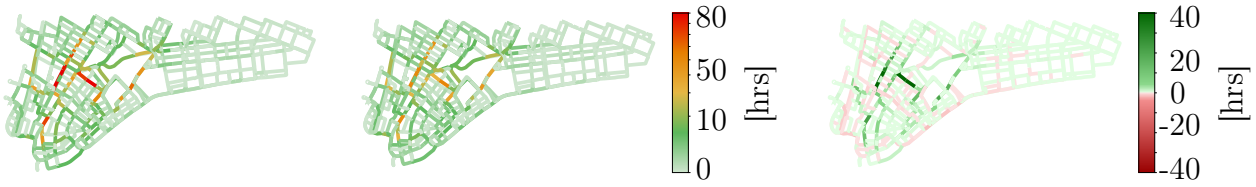
We first observe that the RDUO configuration tends to overload the shortest available routes. In contrast, both `BAL` and `INTEG` redistribute a portion of the demand to longer distance alternatives to mitigate congestion. Expectedly, `INTEG` assigns a slightly larger fraction of trips to the shortest route than `BAL`: by also applying staggering, `INTEG` reduces reliance on detouring to achieve higher performance gains. Regarding staggering behavior, both `INTEG` and `STAG` most frequently apply small shifts, as shown by the higher concentration of trips in the zero to twenty-second departure shift bin. Generally, `INTEG` applies staggering to a larger share of trips and with greater intensity than `STAG`, highlighting how the additional flexibility offered by route rebalancing unlocks further staggering opportunities. The resulting staggering distribution exhibits a clear decay: most adjustments fall within the first 40 seconds, with diminishing frequency thereafter.

The distribution of travel time differences reveals that achieving system-wide efficiency requires some trips to incur longer travel times. These increases are typically modest, mostly within 15 seconds, and only a small fraction exceeds 30 seconds. This trade-off is counterbalanced by a greater number of trips benefiting from travel time reductions, with improvements reaching up to 60 seconds. Finally, arrival time differences are distributed almost symmetrically around zero, with only a small fraction of trips arriving at exactly



(a) Selected routes in solutions (b) Staggering applied in solutions (c) Travel time difference (d) Arrival time difference

Figure 4.7: Frequency plots comparing the structure of the baseline and optimized solutions: (a) route assignments, with route IDs sorted from shortest to longest distance; (b) staggering operations applied, where zero indicates no staggering; (c) travel time differences, where negative values indicate shorter travel times in the solution relative to the baseline, and positive values indicate increases; and (d) arrival time differences, where negative values indicate earlier arrivals and positive values indicate delays relative to the baseline.



(a) Total arc delays in RDUO and `INTEG` (b) Delay differences: RDUO – `INTEG`

Figure 4.8: Spatial distribution of arc-level delays over 31-day instances in RDUO and `INTEG` (a), and their delay differences (b).

the same time as in the RDUO configuration. In the `BAL` variant, the distribution is skewed toward earlier arrivals, whereas configurations involving staggering exhibit a slight skew toward positive differences, consistent with their design to delay trips strategically.

Result 9 *Centralized coordination yields solutions that are structurally distinct from RDUO: (i) it alleviates congestion by rebalancing demand away from the shortest routes, (ii) it integrates balancing and staggering to identify larger pools of promising departure times, and (iii) it enhances overall efficiency by allowing slight travel time increases for some trips, while maintaining tight control over arrival times.*

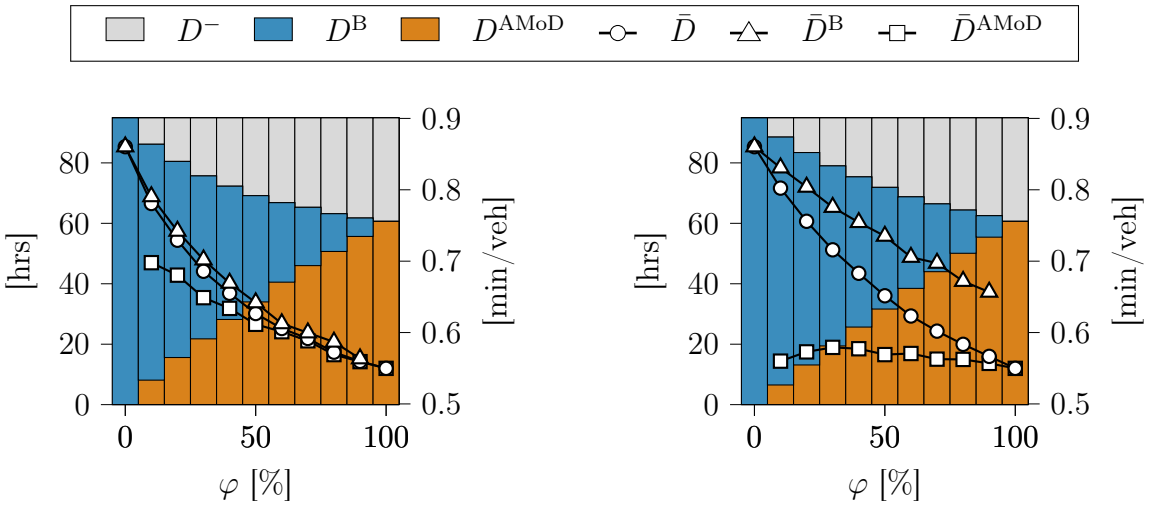
To complement the structural analysis and illustrate how our algorithm reshapes congestion patterns, Figure 4.8 compares cumulative arc-level delays across all 31-day instances between the RDUO and `INTEG` solutions.

In the RDUO configuration, congestion remains concentrated in the central region of the network. By contrast, the `INTEG` solution achieves a more balanced traffic distribution, effectively alleviating these bottlenecks, particularly in areas where alternative routes support rerouting. Importantly, this improvement does not cause delay displacement or introduce new congestion hotspots. As illustrated in the differential map, central arcs benefit from substantial delay reductions of up to 40 hours, while newly utilized arcs see only minor increases, typically below 5 hours.

Result 10 *Our algorithm alleviates major bottlenecks, reducing delays by up to 40 hours, without shifting congestion elsewhere. Delay increases on newly utilized arcs remain minimal, generally under 5 hours, effectively preventing the emergence of new congestion points.*

4.6.4 Flow control analysis

This section analyzes how total delay evolves as the fraction of controlled traffic φ increases. In each instance, trips are randomly designated as AMoD trips with probability φ. Baseload trips follow fixed behavior, consistent with the RDUO solution; we leave the



(a) Welfare-focused operator

(b) Profit-oriented operator

Figure 4.9: Impact of the controlled traffic fraction φ on total and average delay metrics. Figure 4.9a presents results for a welfare-focused operator aiming to minimize total system travel time. Figure 4.9b shows results for a profit-oriented operator focused on minimizing the travel time of its own fleet. Metrics include total delay removed D^-, baseload traffic delay D^{B}, fleet delay D^{AMoD}, and average delays for all trips $\bar{D}$, baseload trips $\bar{D}^{\mathrm{B}}$, and fleet trips $\bar{D}^{\mathrm{AMoD}}$.

study of responsive baseload dynamics to AMoD behavior to future work. We consider two control paradigms. In the first, a welfare-oriented authority (e.g., a regulator or centrally coordinated platform) aims to minimize total system delay by optimizing the travel times of all trips. This setting serves as a benchmark to assess the maximum system-level benefits achievable under partial coordinated control. In the second, a self-interested operator controls only a fleet of AMoD vehicles and minimizes the delay experienced by its own fleet, without accounting for delays among baseload trips, reflecting the competitive behavior of a private provider within a shared urban network.

Figure 4.9 presents results for a representative instance (day 21), illustrating how system-wide delay evolves as φ increases. In the welfare-oriented case, we observe diminishing returns, with the largest marginal delay reductions occurring when fewer than 50% of trips are under control. In contrast, the profit-driven scenario exhibits a more linear trend, as improvements are concentrated on the operator's own fleet.

In the welfare-driven scenario, average delays for AMoD and baseload trips remain similar, especially for $\varphi > 30\%$. In contrast, the profit-driven controller maintains minimal delay for its fleet, while baseload traffic delays remain relatively high. Despite their differing objectives, both strategies lead to clear reductions in total and average delays for both trip classes, suggesting that even partial control yields substantial system-wide benefits.

Result 11 *Controlling only a fraction of the traffic with staggered and balanced routing yields a win–win outcome: AMoD fleets achieve lower delays, while baseload traffic also benefits from improved flow. These system-wide gains arise regardless of whether the operator optimizes for overall welfare or fleet-specific efficiency.*

4.7 Conclusion

In this work, we introduced a novel framework for integrating staggered and balanced routing into AMoD system operations. Our approach minimizes fleet travel times while enhancing overall network efficiency by strategically assigning trips to optimized routes and departure time patterns. We described the problem through a mathematical formulation of the problem and showed that, under mild assumptions, our congestion model provides an unbiased approximation of a discretized Vickrey bottleneck model.

To solve large-scale instances of the problem, we developed a custom metaheuristic based on a LNS framework. In a case study using real-world data from taxi trips in the Manhattan street network, we addressed a realistic volume of peak-hour demand approximating real-world traffic congestion levels. To generate route alternatives, we employed a k-shortest path algorithm with overlap constraints, enabling a diverse yet efficient set of routing options while maintaining computational tractability. Our results show that, under full control, our method can reduce total delays by up to 25% compared to selfish routing. When focusing on network congestion, improvements reach up to 35%. These gains result from redistributing trips onto slightly longer routes and applying modest staggering to departure times. While a small number of trips experience modest travel time increases, the broader gains across the network outweigh these losses. Spatially, the improvements result from congestion relief at bottlenecks, without generating new hotspots elsewhere in the network.

We also examined how the system responds to varying fractions of controlled traffic under two paradigms: a welfare-oriented authority optimizing total system travel time, and a profit-driven AMoD operator focused solely on fleet performance. In both scenarios, staggered and balanced routing consistently improve outcomes for both AMoD and exogenous trips across all control levels.

This research opens several avenues for future work. First, replacing the assumption of precomputed routes with methods that compute globally optimal ones could unlock further improvements in balancing operations. Second, advancing departure time selection policies may further enhance overall system performance. Third, evaluating the framework under higher congestion levels would provide a more realistic assessment of algorithm performance in real-world conditions. Additionally, incorporating scenarios with incom-

plete demand information would improve realism and allow benchmarking the cost of limited information. Finally, future work should explore how baseload traffic dynamically adapts to the behavior of AMoD services.

A4.1 Proof of theorem 2

Proof 2 *Consider the travel time function defined in equation* (4.3)*. Taking expectations, we have:*

$$\mathbb{E}[T(t)] = \tau_a + (\phi \cdot \tau_a)\mathbb{E}[f(t)].$$

To find the expected number of trips $\mathbb{E}[f(t)]$ *in the system, we apply Little's Law, which relates the expected number of trips in a stable system to the arrival rate and the expected time a trip spends in the system:*

$$\mathbb{E}[f(t)] = \lambda \mathbb{E}[T(t)].$$

Substituting the expression for $\mathbb{E}[T(t)]$ *into* $\mathbb{E}[f(t)]$*, we have:*

$$\begin{aligned}
\mathbb{E}[f(t)] &= \rho + (\phi \cdot \rho)\,\mathbb{E}[f(t)] \\
&\implies \mathbb{E}[f(t)] - (\phi \cdot \rho)\,\mathbb{E}[f(t)] = \rho \\
&\implies \mathbb{E}[f(t)] = \rho \cdot (1 - \phi \cdot \rho)^{-1}.
\end{aligned}$$

The expected number of trips $\mathbb{E}[f(t)]$ *is finite when the condition* $\phi \cdot \rho < 1$ *holds. Since* $\phi \in (0, 1]$ *and* $\rho \in (0, 1)$*, this inequality is always satisfied, ensuring system stability for any valid choice of* ϕ*. Substituting* $\mathbb{E}[f(t)]$ *back into the expression for* $\mathbb{E}[T(t)]$*:*

$$\mathbb{E}[T(t)] = \tau_a + (\phi \cdot \tau_a)\mathbb{E}[f(t)] = \tau_a + (\phi \cdot \tau_a \cdot \rho)(1 - \phi \cdot \rho)^{-1}.$$

To match the expected travel time with that of Vickrey's model in Equation (4.5)*, we set:*

$$\begin{aligned}
\tau_a + (\phi \cdot \tau_a \cdot \rho) \cdot (1 - \phi \cdot \rho)^{-1} &= \tau_a + (\tau_a \cdot \rho) \cdot (2(1 - \rho))^{-1} \\
&\implies (\phi \cdot \tau_a \cdot \rho) \cdot (1 - \phi \cdot \rho)^{-1} = (\tau_a \cdot \rho) \cdot (2(1 - \rho))^{-1} \\
&\implies \phi \cdot (1 - \phi \cdot \rho)^{-1} = 2(1 - \rho)^{-1} \\
&\implies 2\phi \cdot (1 - \rho) = 1 - \phi \cdot \rho.
\end{aligned}$$

Rearranging the expression to solve for ϕ *gives:*

$$\phi = (2 - \rho)^{-1}.$$

Algorithm 9 buildRDUO

Input: Set of trips $\mathcal{R}$

$\pi \leftarrow (\emptyset, \emptyset)$ ▷ Initialize empty solution.
$\pi \leftarrow$ insertAtEarliestTimes$(\pi, \mathcal{R}, \texttt{EARLIEST})$ ▷ Insert trips by increasing $\underline{e}^r$.
return π

Algorithm 10 greedyAssignment

Input: Set of trips $\mathcal{R}$

$\pi \leftarrow (\emptyset, \emptyset)$ ▷ Initialize empty solution.
$\pi \leftarrow$ insert$(\pi, \mathcal{R}, \texttt{DEADLINE})$ ▷ Insert trips with tightest ℓ^r first.
$\mathcal{R}^I \leftarrow$ getInfeasibleTrips(π)
if $\mathcal{R}^I \neq \emptyset$ **then**
    $\pi \leftarrow$ localSearch$(\pi, \mathcal{R}^I)$
return π

Since $0 < \rho \leq 1$*, it follows that* $\phi \in (0, 1]$*. This demonstrates that, for this choice of* ϕ*, the expected travel time* $\mathbb{E}[T(t)]$ *matches the expected travel time in Vickrey's model.* □

A4.2 Solution-finding modules pseudocode

We provide pseudocode for the two baseline assignment procedures used to generate initial solutions: the reactive dynamic user optimum (buildRDUO) in Algorithm 9, and the greedy assignment heuristic in Algorithm 10.

A4.3 Description of `constructSchedule`

Algorithm 11 details the procedure to construct the schedule S associated with π. We start by sorting the trip start times s^r into a priority queue Q and initialize an empty set $\mathcal{T}_a$ for each arc a to track when each trip completes the traversal of a. Then, we construct an initial solution iteratively:

i) We start by extracting the departure time $\underline{s}_a^r$ with the highest priority from Q.
ii) We then set $f_a^r = |\{\bar{s}_a^{r'} \in \mathcal{T}_a \,|\, \underline{s}_a^r < \bar{s}_a^{r'}\}|$, that is the number of trips that have not completed their traversal of arc a by time $\underline{s}_a^r$, and compute $d_a^r = d(f_a^r)$.
iii) Finally, we add the time in which the arc traversal is completed, which is $\underline{s}_a^r + d_a^r + \underline{\tau}_a$, to $\mathcal{T}_a$ by setting $\mathcal{T}_a = \mathcal{T}_a \cup (\underline{s}_a^r + d_a^r + \underline{\tau}_a)$. If a is not the last arc of route p^r, we insert $\underline{s}_a^r + d_a^r + \underline{\tau}_a$ into Q.

We repeat these steps until Q is empty.

Algorithm 11 constructSchedule

Input: Solution π

```
Q ← initializePriorityQueue(π)
S ← initializeSchedule(π)
while Q is not empty do
    (r, a) ← Q.pop()
    f_a^r ← computeFlowOnArc(s_a^r)
    d_a^r ← computeDelay(f_a^r)
    a' ← getNextArc(r, a, π)
    s_{a'}^r ← s_a^r + τ_a + d_a^r
    S ← addDeparture(s_{a'}^r)
    if a' ≠ ā_r then
        Q ← pushTrip(r, a')
return S
```

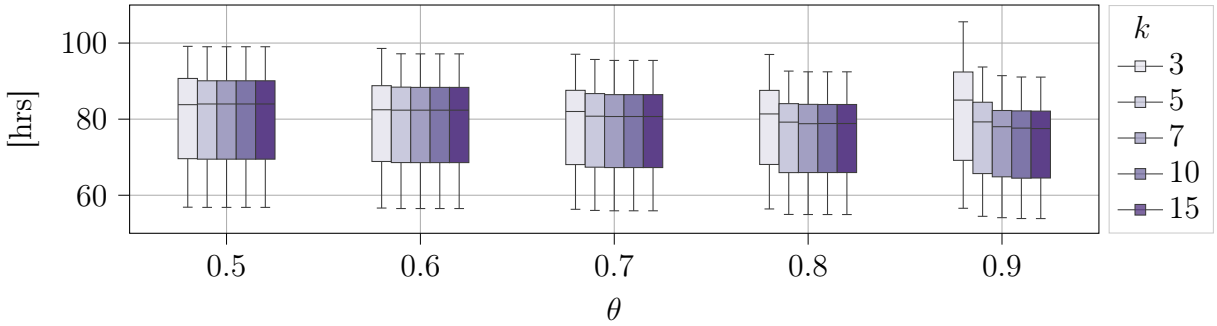
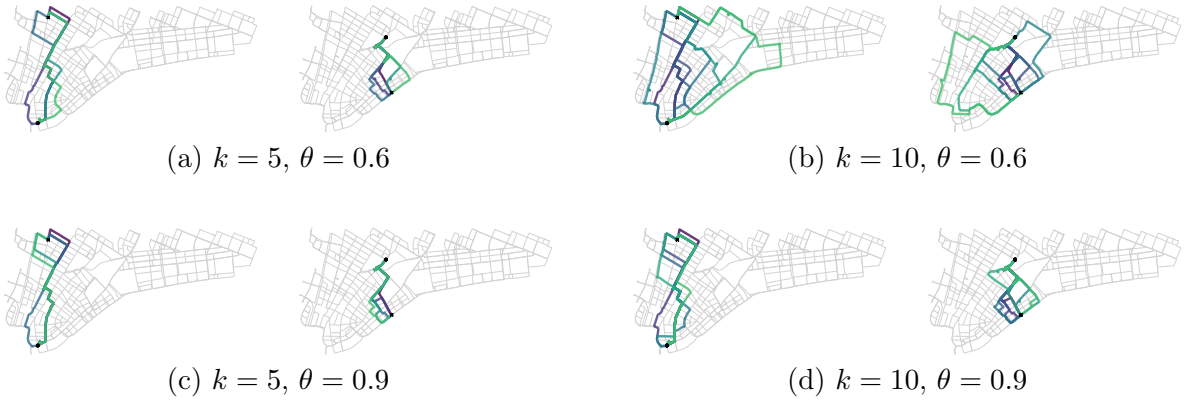


Figure 4.10: Total delay in the RDUO for various combinations of k and θ.

A4.4 Alternative route design

In this section, we explain the rationale for selecting alternative routes for trips by solving a KSPwLO problem. The aim is to find k routes connecting an origin-destination pair, each as short as possible, while ensuring that any two have at most θ % of shared path length. An important consideration is that the route selection influences both our baseline RDUO solution and our proposed algorithmic solutions since both draw routes from the same candidate set. Thus, choosing appropriate parameters k and θ is crucial. Figure 4.10 illustrates the total delays in the RDUO across the 31 instances considered for various combinations of k and θ. Evidently, higher values of k in conjunction with larger θ values provide users with greater flexibility in their route choices, resulting in the smallest median total delay (approximately 78 hours) observed for configurations where $\theta = 0.9$ and $k \geq 7$. However, since our algorithm aims to evenly distribute traffic across alternative network segments to enhance overall efficiency, diversity in route selection becomes highly beneficial. In particular, the scenario with $\theta = 0.6$ and $k = 5$ leads to an increase in median delay of approximately five hours compared to the optimal flexible scenario ($\theta = 0.9$ and $k \geq 7$). We consider this trade-off acceptable given the clear advantage of promoting more diverse route choices and consequently improved network balance.



(a) $k = 5$, $\theta = 0.6$ (b) $k = 10$, $\theta = 0.6$

(c) $k = 5$, $\theta = 0.9$ (d) $k = 10$, $\theta = 0.9$

Figure 4.11: Alternative routes for a long and a short trip with different k and θ values.

To illustrate the visual differences among alternative route configurations, Figure 4.11 presents examples of route variations for both a long and a short trip under various parameter settings. High values of k combined with low θ generate routes excessively long in distance, which frequently must be discarded due to violating the latest arrival constraints even under free-flow conditions. Conversely, configurations with low k and high θ produce highly similar routes that cover limited network areas, complicating effective traffic distribution. Optimal choices thus generally fall into two categories: either low k with low θ, or high k with high θ. Although higher values may achieve more comprehensive network coverage, we select the smaller parameter combination ($k = 5$, $\theta = 0.6$) to substantially reduce computational complexity while maintaining sufficient route diversity.

A4.5 Detailed instances description

In this section, we provide a detailed overview of the characteristics of the instances under consideration. Figure 4.12 illustrates various key attributes of these instances aggregated together. The free-flow travel time for the routes ranges from approximately 3 to 15 minutes, including alternative route options. The relative maximum detour, defined as the percentage increase in free-flow travel time between the shortest and longest (in distance) available route for a given trip, most frequently centers around 25%, although in some cases the longest route covers twice the distance of the shortest. Most trips have a service time window between six and twelve minutes, with a few extending up to twenty minutes. Lastly, most trips allow for a maximum staggering period of roughly one minute, with a few cases extending up to three minutes. Table 4.1 summarizes the main characteristics of the RDUO solution for each day considered. For each day, we report the number of trips $|\mathcal{R}|$, the total travel time $T()$, the total delay D in the RDUO and their ratio, as well as statistics related to the delay experienced by individual trips d^r and the ratio

Table 4.1: Summary of RDUO characteristics for the instances considered. We report the day, the number of trips $|\mathcal{R}|$, the total travel time $T(\pi)$, the total delay D, their ratio, and the average, median, and maximum values of the total delay per trip d^r. We also include the corresponding statistics for the ratio between each trip's travel time and its free-flow travel time (with a value of 1 indicating free-flow conditions).

Day	$\|\mathcal{R}\|$	$T(\pi)$ [hrs]	D [hrs]	$T(\pi)/D$ [%]	d^r [min]			$\tau^r(r)/\underline{\tau}^r(r)$		
					Avg	Med	Max	Avg	Med	Max
1	5283	561	54	9.63	0.65	0.55	2.90	1.11	1.09	1.62
2	5428	578	60	10.38	0.70	0.56	2.77	1.12	1.10	1.59
3	5557	598	64	10.70	0.73	0.59	3.60	1.12	1.10	1.68
4	5448	575	56	9.74	0.65	0.55	2.80	1.11	1.09	1.55
5	5399	575	57	9.91	0.67	0.58	2.78	1.11	1.10	1.54
6	6309	689	81	11.76	0.81	0.69	3.18	1.14	1.11	1.70
7	5857	626	70	11.18	0.76	0.65	3.04	1.13	1.11	1.68
8	5937	635	70	11.02	0.75	0.63	3.00	1.13	1.11	1.61
9	6325	690	83	12.03	0.83	0.72	3.45	1.14	1.12	1.70
10	5807	628	67	10.67	0.73	0.62	2.97	1.12	1.10	1.83
11	5574	599	62	10.35	0.71	0.60	2.96	1.12	1.10	1.50
12	6142	656	72	10.98	0.75	0.64	3.12	1.13	1.11	1.57
13	6572	715	88	12.31	0.85	0.74	3.14	1.14	1.12	1.71
14	6275	675	78	11.56	0.79	0.67	3.52	1.13	1.11	1.82
15	6281	679	81	11.93	0.82	0.69	3.33	1.14	1.12	1.83
16	6627	725	90	12.41	0.86	0.73	3.64	1.14	1.12	1.76
17	6344	688	83	12.06	0.84	0.68	3.37	1.14	1.12	1.85
18	5747	624	69	11.06	0.75	0.63	3.04	1.12	1.11	1.61
19	5611	603	63	10.45	0.71	0.61	3.59	1.12	1.10	1.59
20	6478	708	85	12.01	0.84	0.71	3.09	1.14	1.12	1.87
21	6622	722	90	12.47	0.87	0.75	3.54	1.15	1.13	1.78
22	6194	673	76	11.29	0.78	0.68	3.02	1.13	1.11	1.65
23	6471	707	88	12.45	0.86	0.72	3.38	1.14	1.12	1.82
24	6109	668	78	11.68	0.81	0.67	3.31	1.13	1.11	1.67
25	5355	573	58	10.12	0.68	0.56	3.22	1.11	1.10	1.54
26	6240	673	79	11.74	0.80	0.67	3.56	1.14	1.11	1.96
27	6593	722	88	12.19	0.85	0.72	3.56	1.14	1.12	1.71
28	6443	698	84	12.03	0.84	0.70	3.59	1.14	1.12	1.72
29	6353	680	79	11.62	0.79	0.67	3.07	1.14	1.11	1.64
30	6715	736	92	12.50	0.87	0.74	3.82	1.15	1.13	1.90
31	6144	680	81	11.91	0.84	0.68	3.60	1.14	1.11	1.64
Avg	6072	657	76	11.57	0.78	0.66	3.26	1.13	1.11	1.70

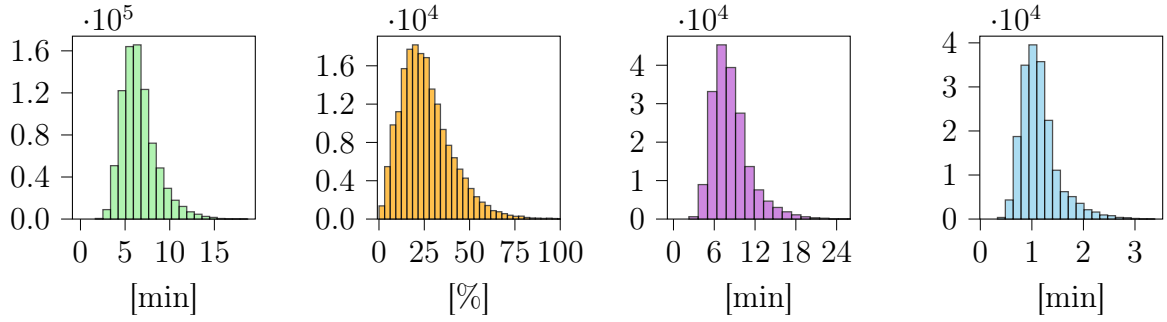
between their actual travel time $\tau^r(r)$ and the corresponding free-flow travel time $\underline{\tau}^r(r)$. On average, the instances contain approximately 6000 trips, and the RDUO total delay accounts for roughly 12% of the total travel time. The individual trip delays pass basic sanity checks: no trip experiences more than four minutes of delay, and travel times never exceed twice the corresponding free-flow travel time.

A4.6 Parameter sensitivity analysis of the LNS

To evaluate the impact of parameter settings on the performance of the LNS algorithm, we conduct a sensitivity analysis by varying three key parameters: the initial pool size x, the sample size y drawn from the pool, and the number of destroy-repair cycles z performed before invoking the local search for intensification. Figure 4.13 presents heatmaps illustrating how different combinations of these parameters affect total delay reduction across five representative days—specifically, days 27 to 31 of the instances described in Appendix A4.5. We also report the average delay reductions across these days. Each run is subject to a time limit of three hours.

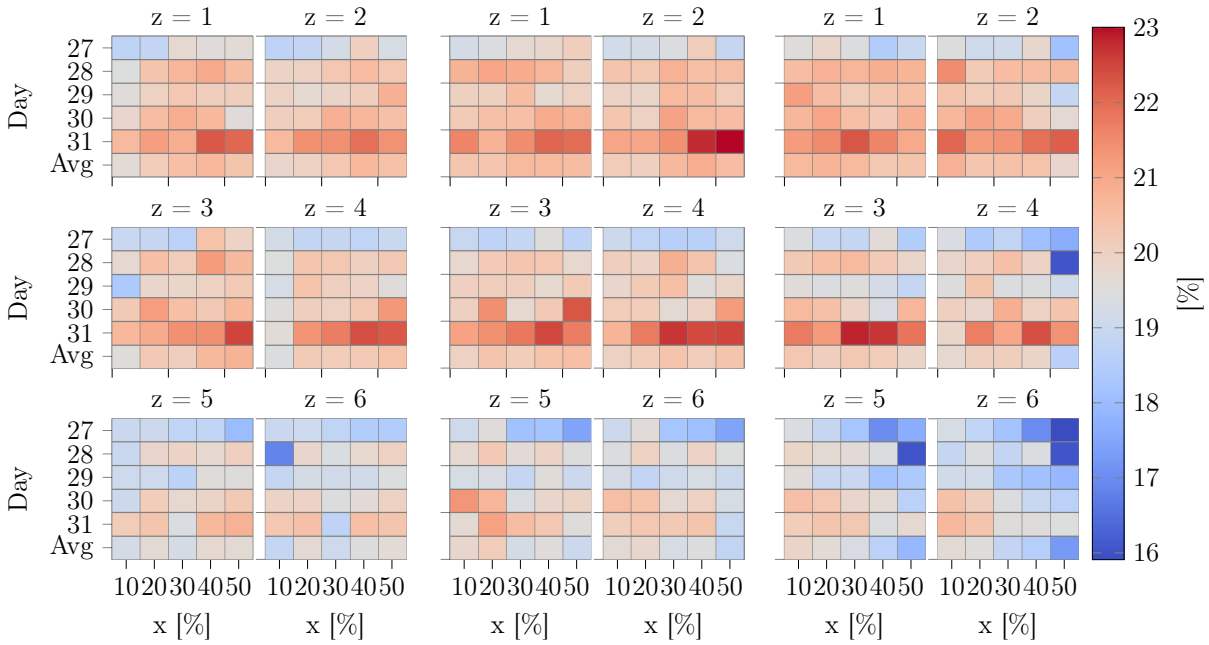
First, increasing the number of destroy-repair cycles z tends to slow down the algorithm and may reduce its effectiveness, with the highest delay reductions often observed between $z = 2$ and $z = 4$. This suggests that it is more advantageous to intensify early using local search and then rebuild a new pool of trips, rather than repeatedly destroying and repairing the same set. Second, a moderate initial pool size, especially $x = 30\%$ or 40%, tends to give better results in most settings. Starting from $x = 50\%$, we generally observe a drop in performance. A larger pool appears beneficial as it includes both highly congested trips - prime candidates for improvement - and less congested ones, thereby supporting more effective exploration of the solution space.

Regarding the sample size y, the configuration with $y = 10\%$ offers the most robust performance, achieving high average delay reductions with relatively low variability across days. In contrast, sample sizes of $y = 5\%$ and $y = 20\%$ lead to greater variability and more frequent underperformance. This suggests that while the sampling procedure is helpful, once a promising pool of trips has been identified, it is more effective to focus on optimizing a moderately sized subset of trips rather than attempting to destroy and reinsert larger portions at once.



(a) Routes travel times in free-flow (b) Relative max. detours (c) Time-window lengths (d) Maximum staggering

Figure 4.12: Frequency histograms summarizing key trip features across all instances.



(a) $y = 5\%$ (b) $y = 10\%$ (c) $y = 20\%$

Figure 4.13: Total delay reduction across five representative days for different parameter configurations of the LNS. We vary the initial pool size $x\%$, the sample size $y\%$ drawn from the pool, and the maximum number of destroy–repair cycles z.

A4.7 Experimental setup for the STAG and MATH comparison

This appendix describes the experimental setup used to compare the staggering-only variant of our algorithm, STAG, with the matheuristic MATH developed for the same problem.

We run the experiments on the full Manhattan network, preprocessed with contraction hierarchies using a maximum shortcut length of 500 meters (cf. Geisberger et al., 2008),

and compute nominal travel times assuming a constant speed of 20 km/h. We extract trip origins, destinations, and earliest departure times from the NYC taxi dataset collected in January 2015. We construct 31 instances, each consisting of 5000 trips sampled between 4:00 and 5:00 p.m. on a different day of the month. As a baseline, we simulate an uncontrolled scenario in which each trip departs as early as possible and follows the shortest-distance route. To model congestion, we define arc travel times using piecewise linear functions:

$$\tau_a^r = \underline{\tau}_a + \max_{k \in K} \left\{ 0, \sum_{1 \le k' \le k} \mu_a^{k'} \cdot \left(f_a^r - \hat{f}_a^{k'} \right) \right\},$$

where $\underline{\tau}_a > 0$, $\mu_a^k > 0$, $\hat{f}_a^k > 0$, and $K = 3$ denotes the number of non-flat segments. We control traffic intensity by restricting arc capacities to allow one AMoD trip every 15 seconds. Accordingly, we set the first breakpoint to $\hat{f}_a^1 = \lceil \underline{\tau}_a / 15 \rceil$, and define the remaining breakpoints as $\hat{f}_a^2 = 2 \cdot \hat{f}_a^1$ and $\hat{f}_a^3 = 3 \cdot \hat{f}_a^1$. The corresponding slopes are:

$$\mu_a^1 = \frac{0.5 \cdot \underline{\tau}_a}{\hat{f}_a^1}, \quad \mu_a^2 = \frac{\underline{\tau}_a}{\hat{f}_a^1}, \quad \mu_a^3 = \frac{1.5 \cdot \underline{\tau}_a}{\hat{f}_a^1}.$$

Under the chosen parameter configuration, total delays in the baseline setting span from two to thirty hours, with a median of approximately six hours. To explore the impact of departure-time flexibility, we generate two sets of instances — $\bar{\sigma}\downarrow$ and $\bar{\sigma}\uparrow$ — by setting the maximum allowable start time shift $\bar{\sigma}^r$ to 10% and 20% of the nominal travel time along each trip's assigned route, respectively. The latest arrival time ℓ^r allows for a maximum travel time equal to 125% of the free-flow time of the assigned route.

To conclude, we configure `STAG` with parameters $x = 50\%$, $y = 10\%$, and $z = 25$, chosen via a calibration procedure similar to that described in Appendix A4.6. For `MATH`, we set a time limit of two hours. Further implementation details are available at https://github.com/tumBAIS/integ_bal_stag.

5 Online Balanced and Staggered Routing in AMoD Systems

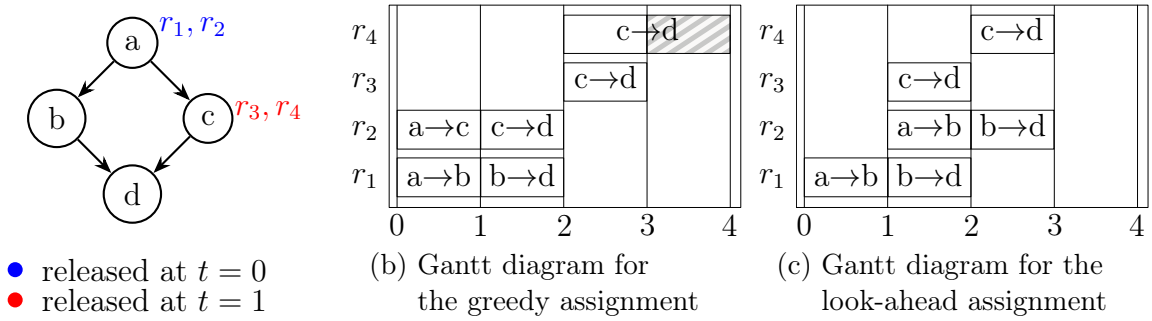
5.1 Introduction

In recent years, digital ride-hailing platforms have scaled rapidly, now serving billions of trips annually. In 2024, little more than a decade after its launch, Uber operated in over 70 countries, completed more than 11 billion trips, and generated nearly USD 44 billion in net revenue (Statista, 2025). At the same time, this growth led to measurable implications for network-level traffic conditions: across U.S. metropolitan areas, the introduction of ride-hailing services increased congestion intensity by roughly one percent and congestion duration by nearly five percent (Diao et al., 2021). These outcomes arise from mutually reinforcing adverse trends. First, a substantial share of ride-hailing demand is induced rather than substitutive: survey evidence from major U.S. cities indicates that up to 61% of ride-hailing trips replace walking, cycling, or public transit, or correspond to trips that would not have occurred in the absence of ride-hailing services. (Clewlow & Mishra, 2017). Second, ride-hailing operations generate large volumes of empty and inefficient travel: in New York City, out-of-service movements account for roughly 50% of total ride-hailing vehicle miles (Erhardt et al., 2019). Third, as drivers compete to serve or anticipate travel demand, both passenger-serving and off-duty vehicles converge on the same high-demand corridors, overloading critical links and degrading overall street network performance, especially during rush hours (Fiedler et al., 2017). In response, cities have sought to regulate these externalities through pricing-based instruments, yielding mixed results. In New York City, congestion surcharges reduced ride-hailing volumes by approximately eleven percent, yet failed to deliver measurable congestion relief, indicating that pricing alone is insufficient to manage ride-hailing–induced congestion (Li & Vignon, 2024). Combined with persistent technical, political, and equity concerns surrounding congestion pricing, these findings motivate complementary congestion-mitigation approaches based on fleet-level operational control (Ke & Qian, 2023).

This chapter builds upon a working paper co-authored with Gerhard Hiermann, Michel Gendreau, and Maximilian Schiffer.

The advent of AMoD systems – centrally coordinated fleets of autonomous vehicles providing door-to-door mobility services (Pavone, 2015) – opens new pathways in this direction. By replacing human drivers, autonomous mobility enables coordinated routing and vehicle repositioning, thereby offering the potential to internalize congestion externalities through fleet-level decisions rather than external pricing. Yet, in practice, autonomous fleets may still generate inefficient travel patterns and can even exacerbate congestion relative to conventional ride-hailing systems in the absence of carefully designed routing and control policies (Oh et al., 2021). Recent work addresses this challenge by introducing two optimization-based coordination mechanisms for AMoD systems: *balanced routing* and *staggered routing*. Balanced routing exploits spatial flexibility by distributing vehicle flows across alternative routes, mitigating congestion at overloaded links. Staggered routing complements this approach by introducing temporal coordination, delaying a subset of departures during peak periods to smooth demand surges. When combined, these spatial and temporal control mechanisms reinforce each other, with strong potential to yield substantial network-wide reductions in travel time during peak periods (Coppola et al., 2025a, 2025b). These results, however, rely on an assumption that is rarely satisfied in practice: complete foresight of future demand. As such, the reported reductions constitute *upper bounds* on achievable performance and may not reflect performance in real-world applications. Moreover, existing approaches are not readily deployable in operational settings, as trip requests arrive stochastically and only partial information is available at decision time (Wen et al., 2019). This motivates the development of methods that transition from offline to online settings, a shift that introduces two fundamental challenges. First, fleet operators must commit to routing and departure time decisions under incomplete demand information. This introduces a trade-off between prioritizing short-term gains at the expense of future efficiency, and employing anticipatory planning that accounts for forthcoming demand (see Figure 5.1). Second, real-time operation requires rapid decision computation, which is challenging due to the combinatorial complexity of jointly optimizing routing and departure times at scale.

This work addresses these challenges by developing a fast, online-deployable approach for balanced and staggered routing in AMoD systems that integrates CO and ML within a unified control pipeline. Optimization components encode the constraints and combinatorial structure of the joint routing and departure time assignment problem, while learning-based components, trained to imitate offline solutions on historical data, capture demand regularities and enable rapid and scalable decision-making. This design enables a centralized operator to dynamically adjust routes and departure times, leveraging contextual information to anticipate congestion and recover a substantial share of the performance achievable under perfect demand foresight.



(a) Network and trips

(b) Gantt diagram for the greedy assignment

(c) Gantt diagram for the look-ahead assignment

Figure 5.1: Toy example illustrating the value of anticipating future trips. Two trips are released at $t = 0$ at node a and two at $t = 1$ at node c, all destined for node d. All arcs have a unit free-flow travel time. Each trip entering an arc incurs a one-unit delay for every other trip already traversing it. Trips from a have two routing options, while trips from c have a single route. The operator may delay each trip's departure by at most one time unit. The greedy assignment in (b) immediately routes trips from a along different paths, congesting arc `c-d` for trip r_4. In contrast, the forward-looking strategy in (c) anticipates the trips released at c and delays one trip starting from a, thereby avoiding congestion on arc `c-d`.

5.1.1 Contributions

This work advances operational control for AMoD systems through three primary contributions. First, we formulate the online balanced and staggered routing problem as a novel event-based MDP that captures the interplay between routing and departure time decisions under partial demand information. Trips arrive sequentially, and the operator jointly assigns each trip a route and a departure time within a bounded delay window, enabling coordinated spatial and temporal control to minimize the system's expected travel time. Flow-dependent delay functions govern arc travel times, capturing the feedback between newly revealed demand, assignment decisions, and evolving network conditions.

Second, we develop a COaML pipeline for real-time decision-making that integrates a statistical learning model with a CO layer. We reformulate the joint route and departure time assignment problem as a shortest-path problem on a time-expanded graph, ensuring feasibility with respect to all physical and temporal constraints while enabling linear-time assignment computation. The learning component parameterizes the arc weights of the time-expanded network to produce anticipatory decisions that balance routes and stagger departures. Trained via imitation learning on full-information offline solutions, the pipeline enables end-to-end training while remaining scalable to large action spaces.

Third, we provide a numerical benchmarking study comparing COaML with reactive heuristics and full-information offline benchmarks in a stylized urban environment. Our

results demonstrate how the COaML policy closely mirrors the behavior of the offline oracle by strategically diverting traffic to alternative routes and smoothing departures during congestion surges. While reactive baselines exhibit significant performance degradation under high-demand regimes, COaML maintains a stable delay profile, preserving a high fraction of the theoretical coordination potential without requiring perfect foresight of future demand.

5.1.2 Roadmap

The remainder of this paper is organized as follows. Section 5.2 reviews the literature on operational control in AMoD systems and on COaML methods for online routing and mobility control. Section 5.3 formulates the online balanced and staggered routing problem in AMoD systems. Section 5.4 describes the COaML pipeline developed to solve instances of this problem. Section 5.5 outlines the experimental design of the numerical study, and Section 5.6 presents the corresponding results. Finally, Section 5.7 concludes the paper and discusses directions for future research.

5.2 Related work

Our work relates to two main research streams. From an application perspective, it builds on the literature on routing and operational control in AMoD systems. From a methodological perspective, it connects to recent advances in COaML pipelines for online routing and mobility control.

5.2.1 Routing and operational control in AMoD systems.

The AMoD literature spans strategic planning, system design, and operational control. Long-term studies on fleet sizing, infrastructure planning, and steady-state behavior establish the structural feasibility and efficiency potential of centrally coordinated fleets (e.g., Cáp & Alonso-Mora, 2018; Fagnant & Kockelman, 2018; Spieser et al., 2014; Vazifeh et al., 2018). Related work couples fleet design with charging infrastructure and grid constraints, jointly optimizing mobility demand and power-system operations over long planning horizons (e.g., Estandia et al., 2021; Iacobucci et al., 2018; Luke et al., 2021). Strategic congestion analyses rely on aggregate traffic representations, equilibrium models, and asymptotic approximations to characterize system-optimal operating regimes (e.g., Braverman et al., 2019; Iglesias et al., 2019; Rossi et al., 2018; Solovey et al., 2019; Wollenstein-Betech et al., 2020). Collectively, these studies provide background on system-level feasibility and design, but lie outside the scope of the real-time

operational control mechanisms examined in this work. For a systematic review of the strategic AMoD literature, we refer the reader to Zardini et al. (2022).

In contrast to strategic planning studies, research on the operational control of AMoD systems addresses real-time decision-making under stochastic demand, limited vehicle availability, and network capacity constraints. A first stream focuses on vehicle dispatching and customer matching, ranging from control-theoretic formulations that provide stability and throughput guarantees (Belakaria et al., 2019; Kang & Levin, 2021; Li et al., 2021b) to decision-focused and learning-based approaches that incorporate anticipation through demand forecasts, look-ahead optimization, hybrid optimization–learning pipelines, and reinforcement learning (e.g., Hu & Dong, 2022; Jindal et al., 2018; Jungel et al., 2025; Li et al., 2021a; Liang et al., 2022; Yu & Shen, 2020). A second stream studies vehicle rebalancing via short-horizon optimization and MPC, with extensions accounting for congestion effects, parking constraints, ride-sharing, and electrification (e.g., Aalipour & Khani, 2024; Carron et al., 2021; Guériau & Dusparic, 2018; Guériau et al., 2020; He et al., 2023b, 2023a; Iglesias et al., 2018; Ruch et al., 2021; Wallar et al., 2018). A third stream examines routing as a mechanism for congestion mitigation, including data-driven and learning-based approaches evaluated in both stylized settings and city-scale networks (e.g., Jalota et al., 2023; Lazar et al., 2021; Levin et al., 2017; Liu et al., 2019). A fourth stream investigates integrated routing and rebalancing formulations that jointly optimize customer routes and empty-vehicle movements, often within data-driven or MPC frameworks (e.g., Schmidt et al., 2024; Tsao et al., 2018), and naturally extends to ride-sharing, intermodal coordination, and electric fleets (e.g., Alonso-Mora et al., 2017; Boewing et al., 2020; Iacobucci et al., 2019; Singhal et al., 2024; Tsao et al., 2019; Turan et al., 2020; Zgraggen et al., 2019). Taken together, the operational AMoD literature either assumes constant travel times or relies on exogenous or historically estimated congestion effects; works that consider more detailed dynamic congestion models typically assume perfect demand knowledge, leaving joint routing and departure time assignment for AMoD systems under uncertainty in dynamic settings largely unexplored.

5.2.2 Combinatorial optimization–augmented machine learning.

Methods for online routing and mobility control span a spectrum from optimization-centric to fully data-driven paradigms. At one end, optimization-based approaches handle complex combinatorial action spaces by repeatedly solving large-scale assignment problems; however, they often encounter scalability bottlenecks under uncertainty (e.g., Bertsimas et al., 2019). At the other end, reinforcement learning frameworks enable rapid inference and anticipatory decision-making, yet face fundamental challenges in handling the highly constrained and discrete structures inherent in fleet-level coordination (e.g.,

James et al., 2019; Joe & Lau, 2020). Predict–then–optimize approaches seek a middle ground by embedding learned predictions within explicit optimization models, yet suffer from mismatches between statistical prediction accuracy and downstream decision quality (e.g., Bertsimas & Kallus, 2020). COaML pipelines address this limitation by tightly integrating combinatorial optimization and machine learning within unified decision architectures, embedding optimization problems as differentiable decision layers trained end-to-end against downstream objectives (cf., Dalle et al., 2022; Parmentier, 2021). In the following, we briefly recap the paradigm's foundations, recent advances, and applications. For a more in-depth survey of the field, we refer the reader to Schiffer et al. (2026).

Conceptually, these pipelines build on structured learning over combinatorial output spaces (Nowozin & Lampert, 2011) and differentiate through discrete combinatorial layers via relaxations or surrogate smoothing (Berthet et al., 2020; Blondel et al., 2020; Vlastelica et al., 2019). Early COaML approaches rely on imitation learning, therefore requiring access to expert solutions. More recent work explores reinforcement-learning-based formulations that embed combinatorial layers within policy architectures, addressing the exploration limitations of imitation-based methods in combinatorial MDPs (Hoppe et al., 2025). Across a range of applications in dynamic, large-scale decision-making, including AMoD dispatching, emergency services, inventory routing, and vehicle routing, COaML pipelines have demonstrated strong performance (Baty et al., 2024; Greif et al., 2024; Jungel et al., 2025; Rautenstrauss & Schiffer, 2025). Beyond direct operational control, these architectures have also proven effective for high-fidelity traffic flow prediction (Jungel et al., 2024). These results motivate their application to traffic prediction integrated with active route and departure time assignment in AMoD control, raising the question of how such pipelines can be adapted to this joint setting.

5.3 Problem statement

We consider a AMoD system operating in an urban environment, in which a centralized operator coordinates a fleet of autonomous vehicles to serve trips that are revealed dynamically over a finite planning horizon. Each trip requires a single vehicle and represents either an on-duty task (e.g., customer transport or delivery) or an off-duty task (e.g., vehicle rebalancing). Upon the arrival of each request, the operator assigns a route and departure time, accounting for congestion from ongoing trips and anticipating the impact of future, unrevealed demand. The objective is to minimize the expected total system travel time over the planning horizon.

To formalize this setting, we represent the road network as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$,

where nodes $\mathcal{V}$ correspond to trip origins, destinations, and road intersections, and arcs $\mathcal{A}$ represent road segments connecting them. The set $\mathcal{R}$ collects all trips to be served by the operator as they traverse the network. Each trip $r \in \mathcal{R}$ is defined by the quadruple $(\mathcal{P}^r, \underline{e}^r, \ell^r, \bar{\sigma}^r)$, where:

i) $\mathcal{P}^r = \{p_1^r, \ldots, p_k^r\}$ denotes the set of k alternative routes (arc sequences) available;

ii) $\underline{e}^r$ denotes the time at which the trip becomes available for assignment;

iii) ℓ^r denotes the latest permitted arrival time at the destination;

iv) $\bar{\sigma}^r$ denotes the maximum permitted delay from the earliest departure $\underline{e}^r$.

The travel time $\tau_a^r(f_a^r)$ incurred by a trip r on arc a consists of a free-flow component τ_a and a congestion-induced delay. The delay depends on the number of trips f_a^r simultaneously using arc a at the time r begins to traverse it. We model this delay as a convex, non-decreasing function $d(f_a^r)$ of the arc flow:

$$\tau_a^r(f_a^r) = \tau_a + d(f_a^r). \tag{5.1}$$

Based on this structure, we model the operator's decision-making as an event-based MDP, represented by the tuple $\langle \mathcal{X}, \mathcal{Y}, P, R, \gamma \rangle$. A decision is made each time a new trip r is revealed, with the corresponding decision epoch indexed by $t = \underline{e}^r$. The decision process consists of the following components:

i) **State space** ($\mathcal{X}$): A state $x_t \in \mathcal{X}$ characterizes the system at decision epoch t by capturing the positions of all active trips $\mathcal{R}^t$, defined as those currently en route between their release and their respective destinations. This set includes both traveling trips and the newly revealed trip, which is waiting at its pickup location for assignment. The initial state x_0 is marked by the release of the first trip at its origin node, while the rest of the network is empty.

ii) **Action space** ($\mathcal{Y}$): At each decision epoch t, the operator selects an action, denoted as $y_t \in \mathcal{Y}(x_t)$, consisting of assigning the newly revealed trip r a departure time $s^r \in [\underline{e}^r, \bar{e}^r]$, where $\bar{e}^r = \underline{e}^r + \bar{\sigma}^r$, and a route $p^r \in \mathcal{P}^r$. The notation $\mathcal{Y}(x_t)$ indicates that the feasible actions depend on state x_t; we omit this dependency when clear from context. As vehicles do not idle once a trip starts, the current state x_t combined with the selected action y_t fully determines the travel and arrival times of all active trips in $\mathcal{R}^t$. Accordingly, each action must ensure that no active trip arrives at its destination later than its latest permitted arrival time ℓ^r.

iii) **Transition function** (P): The system transitions to a new state x_{t+1} after the operator executes action y_t in state x_t, and a new trip is revealed at time $t+1$. The transition function $P(x_{t+1} \mid x_t, y_t)$ captures this evolution by updating the positions of all active trips based on their progress along assigned routes between decision

epochs t and $t+1$. Trips that reach their destination by $t+1$ leave the system.

iv) **Reward function** (R): The reward at each decision epoch reflects the negative of the system-wide travel time. Specifically, we define:

$$R(x_t, y_t) = -\sum_{r \in \mathcal{R}^t} \tau^r(x_t, y_t), \tag{5.2}$$

where $\tau^r(x_t, y_t)$ denotes the total travel time of trip r along its assigned route, as determined by the current state and selected action.

v) **Policy and objective**: A (deterministic) policy δ is a function $\delta : \mathcal{X} \to \mathcal{Y}$ mapping each state to a corresponding action. The objective is to find an optimal policy $\delta^\star$ that maximizes the expected cumulative reward over the planning horizon T:

$$\delta^\star = \arg\max_{\delta} \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t R(x_t, \delta(x_t)) \mid x_0\right], \tag{5.3}$$

where $\gamma \in [0, 1]$ is the discount factor.

Discussion. A few remarks are in order. First, we assume that only AMoD trips traverse the network, granting the operator full control over all trips in $\mathcal{R}$. This assumption keeps the formulation concise; background traffic can be incorporated by designating a subset of trips with fixed routes and departure times that are not subject to optimization. Second, to ensure scalability, we precompute the set of alternative routes for trips, consistent with route-based approaches commonly used in dynamic traffic assignment (cf. Lo & Szeto, 2002). Third, while we model departure times as continuous decision variables for generality, we restrict departures to a finite set of integer-valued time points in the methodology and implementation. In particular, we adopt a one-second resolution, which provides a sufficiently fine approximation in practice and can be refined if higher or coarser temporal accuracy is required.

5.4 Methodology

In this section, we introduce a COaML pipeline for learning a control policy δ that jointly selects routes and departure times for dynamically arriving trips in an AMoD system. The key challenge in this online setting is the absence of future demand information, requiring the operator to commit to spatial and temporal assignments without knowing how subsequent requests will affect network congestion. The pipeline maps the current system state x_t to a feasible decision y_t by integrating a ML model with a CO layer. A central

idea of our approach is to reformulate the joint assignment problem as a shortest-path problem on a time-expanded graph. This structural encoding guarantees that all decisions satisfy physical and temporal constraints by construction, avoiding the feasibility issues that can arise with purely black-box predictors. Moreover, the directed acyclic structure of the resulting graph enables assignment computation in linear time, allowing the system to remain responsive under high-frequency request streams. The ML component parameterizes the objective of the CO layer by assigning weights to graph arcs, ensuring that its solution yields anticipatory decisions that internalize network congestion effects. The model is trained end-to-end via imitation learning using full-information offline solutions, allowing the online policy to emulate the coordinated behavior of an offline oracle with perfect foresight while preserving real-time computational tractability.

The remainder of this section is organized as follows. Section 5.4.1 details the graph construction that enables this integrated optimization. Section 5.4.2 then describes the overall COaML architecture, including the training-set construction and the end-to-end learning procedure.

5.4.1 Path-based reformulation of the route–departure assignment problem

To cast the joint route and departure time assignment for a newly revealed trip r as a shortest-path problem, we construct a time-expanded graph $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{A}^t)$ that encodes all its feasible assignments. For each route $p^r \in \mathcal{P}^r$ and each feasible departure time $s \in [\underline{e}^r, \bar{e}^r] \cap \mathbb{Z}$, we construct a linear chain of nodes representing the trip's progression along the physical arcs $a \in p^r$. We index each arc of the resulting time-expanded graph by a pair $(s, a) \in \mathcal{A}^t$ and denote the corresponding weight as θ^s_a. We introduce a

(a) Available routes and departures for a trip.

(b) Associated time-expanded graph.

Figure 5.2: Transformation from the physical network to the time-expanded graph for a trip r. In the physical network, arc labels τ^s_a represent travel times for trip r, accounting for interactions with other trips on the same arcs. In the time-expanded graph, the shortest path from the source node `src` to a destination node v corresponds to a joint route and departure time assignment. Arc weights are denoted by θ^s_a, allowing flexibility in their specification. By adjusting these weights, we can steer the shortest path solution toward different types of assignments for the original problem.

designated source node `src` that connects to the entry node of each constructed chain. Consequently, each path from `src` to the destination node – shared by all chains – uniquely encodes a valid route and departure time assignment. Figure 5.2 illustrates the resulting graph, while Algorithm 12 summarizes the corresponding construction procedure. Since the time-expanded graph $\mathcal{G}^t$ is a directed acyclic graph (DAG) by construction, we can compute shortest paths in linear time via a single pass over a topological ordering of the nodes. This approach guarantees linear-time complexity in the number of nodes and arcs, i.e., $\mathcal{O}(|\mathcal{V}^t| + |\mathcal{A}^t|)$, and is therefore suitable for large-scale graphs. For a detailed description of the procedure, we refer the reader to Appendix A5.1.

The solution returned by the shortest path algorithm – and hence the selected route and departure time assignment – depends on the arc weights of the time-expanded graph $\mathcal{G}^t$, collected in the vector θ. The choice of these weights is critical, as it determines how assignment decisions align with the overarching objective of minimizing the system's expected total travel time. To illustrate this dependency, we contrast two alternative interpretations of arc weights and the routing behaviors they induce. First, given an assignment of active trips $\mathcal{R}_t$, one can define θ by assigning to each arc of $\mathcal{G}^t$ the travel time incurred by the newly revealed trip r, thereby ignoring its impact on other trips. Using these weights yields a form of *selfish routing*, in which the shortest path corresponds to the route–departure assignment that minimizes the travel time of trip r alone. Second, consider an offline benchmark in which the operator assigns only trip r, while all other trips are optimally assigned and held fixed. In this setting, weighting each arc by the marginal increase in total system travel time induced by routing r through that arc at a given route and departure time ensures that the resulting shortest-path solution yields the optimal assignment for r, minimizing overall system travel time. Given our objective, arc weights aligned with the second setting are desirable. In an online setting, however, future trip requests are unknown. Clearly, considering only the impact of an assignment on the travel time of currently active trips leads to a greedy assignment. In contrast, anticipating interactions with future, yet unrevealed demand at decision time leads to a

Algorithm 12 buildTimeExpandedGraph

Input: Trip routes $\mathcal{P}^r$, release time $\underline{e}^r$, latest departure $\bar{e}^r$

Initialize $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{A}^t)$ with $\mathcal{V}^t \leftarrow \{\texttt{src}\}$ and $\mathcal{A}^t \leftarrow \emptyset$
for $p^r \in \mathcal{P}^r$ **do**
  for $s \in [\underline{e}^r, \bar{e}^r] \cap \mathbb{Z}$ **do**
    $u^s_{\text{prev}} \leftarrow \texttt{src}$
    for each physical arc $a = (u, v) \in p^r$ **do**
      $\mathcal{V}^t \leftarrow \mathcal{V}^t \cup \{v^s\}$
      $\mathcal{A}^t \leftarrow \mathcal{A}^t \cup \{(u^s_{\text{prev}}, v^s)\}$ ▷ time-expanded arc (s, a)
      $u^s_{\text{prev}} \leftarrow v^s$
return $\mathcal{G}^t$

forward-looking assignment. Importantly, while marginal system costs provide a useful conceptual benchmark, the weights θ used in practice do not have to correspond to physically interpretable quantities. In fact, it suffices that they parameterize the shortest-path problem so that the resulting solutions align with the system objective. The remainder of our methodology is therefore devoted to retrieving arc weights that enable such forward-looking decisions.

5.4.2 The COaML pipeline

Given an arc-weight vector θ for the time-expanded network $\mathcal{G}^t$, solving a shortest-path problem suffices to generate real-time assignments for newly arriving trips. However, determining arc weights θ that move beyond purely greedy behavior remains an open challenge. To achieve this goal, we develop a COaML pipeline for encoding predictive information into the weight vector θ. The pipeline, illustrated in Figure 5.3, receives as input the system state x_t, which the DAG generator transforms into a time-expanded graph $\mathcal{G}^t$. The second layer, the ML layer, receives the graph $\mathcal{G}^t$ and uses a statistical model φ_w, parameterized by the vector w, to predict the arc weights θ. The subsequent CO layer computes the shortest path y_t^p on the weighted time-expanded graph $\mathcal{G}_\theta^t$. Finally, a simple mapping decodes the path y_t^p to a feasible balancing and staggering decision y_t.

To obtain an effective policy, the predictor φ_w must output arc weights θ that induce high-quality, anticipative balancing and staggering actions y_t when solving the shortest-path problem on the weighted graph $\mathcal{G}_\theta^t$. Since performance critically depends on the choice of the predictor parameter set w, we devote the remainder of this section to the methodology used to train φ_w. In Section 5.4.2.1, we describe the procedure utilized to construct the dataset used to train the pipeline within an imitation learning framework. Section 5.4.2.2 then introduces the learning paradigm that enables end-to-end training of φ_w. Finally, Section 5.4.2.3 details the ML predictor φ_w and the approach used to solve the associated learning problem.

5.4.2.1 Building the training set

Our goal is to approximate full-information decisions in an online setting, where future demand information is not observable. Accordingly, we construct a training set of target actions derived from full-information solutions computed on historical data. We consider

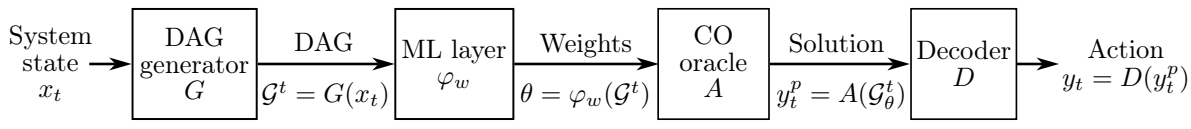


Figure 5.3: Overview of our COaML pipeline.

a collection of M historical problem instances, where instance m comprises n_m trips. For each instance, we solve the full-information problem using a problem-specific metaheuristic, summarized in Appendix A5.2, to obtain a high-quality route and departure-time assignment for every trip. Since online decisions concern one request at a time, we *slice* each full-information solution to obtain the training set

$$\mathcal{D} = \{(x_1, y_1^*), \ldots, (x_n, y_n^*)\},$$

which contains $n = \sum_{m=1}^{M} n_m$ trip-level datapoints. Each datapoint (x_t, y_t^*) comprises the system state x_t at reveal time $t = \underline{e}^r$ and the corresponding route and departure time assignment y_t^*, encoded as a binary vector over the arcs of $\mathcal{G}^t$ representing all feasible assignments for trip r. To prevent leakage of future information, we construct the state x_t using only information from trips r' in the same instance whose release time $\underline{e}^{r'}$ is less than or equal to the release time $\underline{e}^r$ of r. Formally:

$$\mathcal{R}_t = \left\{ r' \in \mathcal{R} \,\middle|\, \underline{e}^{r'} \leq \underline{e}^r \right\}.$$

This specification excludes trips r' revealed after time t but dispatched earlier than r in the full-information solution as a result of staggering applied to r's departure. Given an assignment π over the trip set $\mathcal{R}_t$, we construct the system state x_t using the routine `constructState`(π), described in Appendix A5.3.

5.4.2.2 Structured learning methodology

The pipeline's effectiveness critically depends on the arc weights θ predicted by the statistical model φ_w in the ML layer. Since routing decisions are determined by the CO problem output, training must embed CO problem calls within the learning loop. We therefore adopt a structured learning approach that directly links the predicted weights θ to the CO problem output and enables supervised learning from past full-information solutions drawn from a finite, structured, and combinatorially large solution space. To this end, we first formulate the CO problem as a maximization problem by simply negating the arc weights of the shortest path problem:

$$\hat{y}_i(\theta) = \underset{y \in \mathcal{Y}(x_i)}{\arg\max}\, \theta^\top y. \tag{5.4}$$

Given a training set $\mathcal{D} = \{(x_1, y_1^*), \ldots, (x_n, y_n^*)\}$ of n CO problem instances x_i and their corresponding target solutions y_i^*, our goal is to find a weight vector θ such that the predicted solutions $\hat{y}_i(\theta)$ imitate the target solutions contained in $\mathcal{D}$. We formulate the

learning problem as

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(\theta, x_i, y_i^*\right). \tag{5.5}$$

A natural choice for the loss $\mathcal{L}\left(\theta, x_i, y_i^*\right)$ is

$$\mathcal{L}\left(\theta, x_i, y_i^*\right) = \max_{y \in \mathcal{Y}(x_i)} \theta^\top y - \theta^\top y_i^*, \tag{5.6}$$

which measures the non-optimality of y_i^* under θ. Clearly, $\mathcal{L}\left(\theta, x_i, y_i^*\right) \geq 0$. Minimizing (5.6) encourages $\hat{y}_i(\theta)$ to approach y_i^*, driving $\mathcal{L}\left(\theta, x_i, y_i^*\right)$ toward zero.

However, combining (5.5) with (5.6) poses two issues. First, the mapping $\theta \mapsto \hat{y}_i(\theta)$ in (5.4) is piecewise constant and thus non-differentiable, preventing the application of gradient-based methods such as stochastic gradient descent (SGD) to solve (5.5). Second, the formulation is degenerate: $\theta = 0$ makes any feasible $y \in \mathcal{Y}(x_i)$ optimal for (5.4) and yields zero loss, leading to arbitrary, uninformative solutions. Following Berthet et al. (2020), we add a Gaussian perturbation $Z \sim \mathcal{N}(0, I)$ to θ and define

$$\mathcal{L}_Z\left(\theta, x_i, y_i^*\right) = \mathbb{E}_Z\left[\max_{y \in \mathcal{Y}(x_i)} (\theta + \epsilon Z)^\top y\right] - \theta^\top y_i^*, \tag{5.7}$$

where $\epsilon > 0$ controls the perturbation magnitude. In this way, we turn the deterministic CO problem in (5.4) into a probabilistic CO *layer*. The objective

$$F(\theta) \coloneqq \mathbb{E}_Z\left[\max_{y \in \mathcal{Y}(x_i)} (\theta + \epsilon Z)^\top y\right] \tag{5.8}$$

is convex and smooth, hence differentiable (Dalle et al., 2022, Prop. 3.1). The gradient of the regularized loss with respect to the parameters θ is given by the following expectation:

$$\nabla_\theta \mathcal{L}_Z\left(\theta, x_i, y_i^*\right) = \mathbb{E}_Z\left[\arg\max_{y \in \mathcal{Y}(x_i)} (\theta + \epsilon Z)^\top y\right] - y_i^*. \tag{5.9}$$

This probabilistic formulation implicitly regularizes the CO layer's predictions (Dalle et al., 2022, Prop. 3.2). Specifically, $\mathcal{L}_Z\left(\theta, x, y^*\right)$ can be interpreted as the left-hand side of the Fenchel-Young inequality

$$\Omega^*(\theta) + \Omega(y) - \theta^\top y \geq 0.$$

Here $\Omega \coloneqq F^*$ is the convex conjugate of (5.8), and $\Omega^* = F$, as F is strictly convex (Berthet et al., 2020, Prop. 2.2). Hence, the learning problem is strictly convex and admits a unique minimizer, which is unlikely to be zero. Indeed, $\mathcal{L}_Z\left(\theta, x_i, y_i^*\right) = 0$ if and

only if $\theta = \nabla\Omega(y_i^*)$. If y_i^* is a vertex of $\mathcal{Y}(x_i)$, this equality is attained only asymptotically, requiring θ to diverge along the interior of the normal cone of y_i^*. Consequently, the unique finite minimizer lies strictly inside that cone, ruling out the degenerate solution $\theta = 0$.

Finally, although all components of (5.5) are now specified, the expectation in (5.7) and its gradient (5.9) remain intractable. We therefore approximate them via Monte Carlo sampling of the perturbation Z, yielding a stochastic gradient estimator, and solve the resulting problem using SGD.

5.4.2.3 ML predictor

The regularized loss (5.7) provides a differentiable learning signal with respect to the arc weights θ, enabling gradient-based optimization of the learning problem (5.5). We backpropagate the resulting gradient through the statistical model φ_w to update its parameters w, which therefore requires φ_w to be differentiable. Accordingly, we leave the functional form of φ_w unconstrained and require only differentiability with respect to its parameters w. We model the weight of each time-expanded arc $(s,a) \in \mathcal{A}^t$ as the output of φ_w applied to an arc-level feature representation:

$$\theta_a^s = \varphi_w\big(\phi_a^s(x_t); w\big), \quad \forall\,(s,a) \in \mathcal{A}^t, \tag{5.10}$$

where $\phi_a^s(x_t) \in \mathbb{R}^d$ denotes a d-dimensional feature vector encoding properties of the time-expanded arc (s,a) and the current system state x_t. Appendix A5.4 details the feature mapping $\phi_a^s(x_t)$, while Appendix A5.5 compares alternative architectures for φ_w.

5.5 Design of experiments

This section details the experimental framework used to evaluate the proposed approach. We first describe the network employed and the trip generation process. We then introduce the benchmark methods and performance metrics used for comparison, and assess the complexity of the instances under consideration. Finally, we summarize the statistical model selection procedure and computational environment.

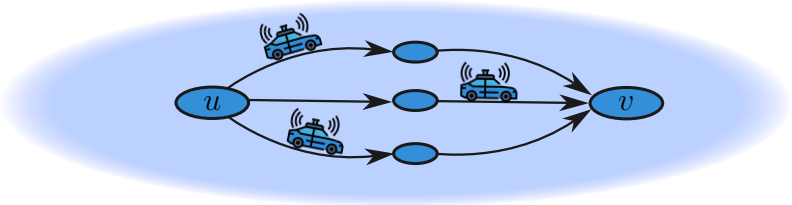


Figure 5.4: Synthetic parallel network employed in our numerical study.

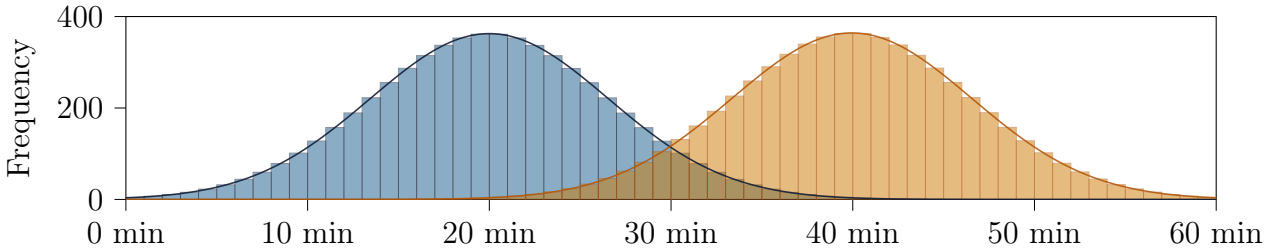


Figure 5.5: Temporal distribution of trip requests. Release times follow a bimodal truncated Gaussian mixture over a 60-minute horizon, simulating urban rush-hour peaks at 20 and 40 minutes.

Network The experimental network models a simplified urban environment with a single origin-destination pair connected by three parallel routes, with nominal travel times $\tau \in \{60, 90, 120\}$ seconds (see Figure 5.4). An intermediate node bisects each route into two arcs of equal length, serving as a spatial checkpoint to capture mid-route state updates and localized traffic dynamics. Delays follow a polynomial expression of the form:

$$d(f_a^r) = \tau_a \cdot \alpha \left[\left(\frac{f_a^r + \beta}{\tau_a} \right)^\gamma - \left(\frac{\beta}{\tau_a} \right)^\gamma \right], \tag{5.11}$$

where we set $\alpha = 0.5$ to scale the delay magnitude, $\beta = 40.0$ to shift the baseline and stabilize sensitivity under low-flow conditions, and $\gamma = 4.0$ as the polynomial exponent. The subtraction term ensures the function evaluates to zero in the absence of other trips.

Test set We construct a test set of 30 instances representing diverse traffic scenarios. Each instance spans a planning horizon of $T = 60$ minutes and includes $n = 400$ trip requests. The earliest departure times $\underline{e}^r$ are sampled from a bimodal mixture of truncated Gaussian distributions to generates a non-uniform temporal demand profile that simulates realistic urban peak-period fluctuations, with request density concentrated around two distinct intervals (see Figure 5.5). We characterize each trip r by a 30-second departure window $[\underline{e}^r, \bar{e}^r]$ and a total service window of 150 seconds, defined as $[\underline{e}^r, \ell^r]$. Together, these parameters bound the system's temporal flexibility and establish a maximum allowable duration for each request.

Benchmarks We evaluate the performance of our proposed approach (`COaML`) against three benchmarks representing different levels of coordination and information availability:

(i) `RDUO`: A baseline implementing selfish routing, in which trips are processed sequentially, each selecting the route that minimizes its individual travel time and departing as early as possible. This uncoordinated scenario serves as the primary reference point to quantify the benefits of active coordination.

(ii) `GREEDY`: An online heuristic that assigns each request to the route and departure time minimizing system-wide travel time based on the currently available network information.

(iii) `META`: An offline metaheuristic that computes solutions under full demand information. This serves as a theoretical upper bound on performance for any online strategy.

KPIs We evaluate all methods using the following key performance indicators (KPIs):

(i) *Congestion delay* (CD): The aggregate delay resulting from network flow interactions and link-level congestion.

(ii) *Detour delay* (DD): The additional free-flow travel time incurred when routing trips via routes longer than their respective shortest distance routes.

(iii) *Total delay* (TD): The cumulative time loss across all trips, defined as the sum of congestion-induced and detour-induced delays.

Note that total delay differs from total travel time only by a constant—the sum of free-flow travel times along shortest routes—which is invariant across methods. We therefore use total delay as the primary performance metric. To assess instance complexity, we evaluate the respective `RDUO` traffic metrics and report the results in Table 5.1. At the aggregate level, the median total travel time is approximately 11 hours. System-wide delays constitute a substantial overhead: the median total delay of 4.4 hours accounts for about 40% of total travel time. Congestion is the primary contributor, with a median of 2.9 hours (about 26% of travel time), while detour-related delays account for 1.5 hours (roughly 14%). The relatively tight aggregate ranges indicate that overall instance difficulty is consistent across the generated scenarios. At the individual-trip level, delays are heterogeneous but remain well-behaved. The median per-trip total delay is 42 seconds, corresponding to approximately 41% of individual travel time, with maxima of 66 seconds and 52%, respectively. Congestion delays exhibit a similar pattern, with a median of 26 seconds (25%) and upper tails reaching 65 seconds. Detour delays are more concentrated: while their aggregate contribution is moderate, they can reach up to 50% of travel time

Table 5.1: Summary of traffic metrics for baseline solutions across instances.

	Aggregate [h]			Aggregate [%]			Per trip [s]			Per trip [%]		
KPI	Min	Med	Max	Min	Med	Max	Min	Med	Max	Min	Med	Max
TD	4.2	4.4	4.6	38.7	39.7	40.9	5.0	42.0	66.0	7.7	41.2	52.4
CD	2.7	2.9	3.0	25.0	25.8	26.7	1.0	26.0	65.0	1.1	25.0	52.0
DD	1.5	1.5	1.7	13.5	13.8	14.8	30.0	30.0	60.0	23.8	28.8	50.0

for certain trips, indicating that rerouting decisions occasionally impose substantial local penalties.

Statistical model Across all experiments, we deploy `COaML` with a graph neural network (GNN) as a statistical model, selected through a two-stage hyperparameter-tuning procedure. In the first stage, we conduct a broad sweep over encoder architectures, learning rate schedules, and perturbation intensities using independently generated scenarios. In a second stage, we evaluate the robustness of the best-performing configuration across multiple random training seeds and varying training batch sizes. Model selection is conducted on a held-out validation set, using the mean performance gap to the offline benchmark as the evaluation criterion. Appendix A5.5 provides full details on the candidate architectures, training protocol, validation metric, and model-selection procedure.

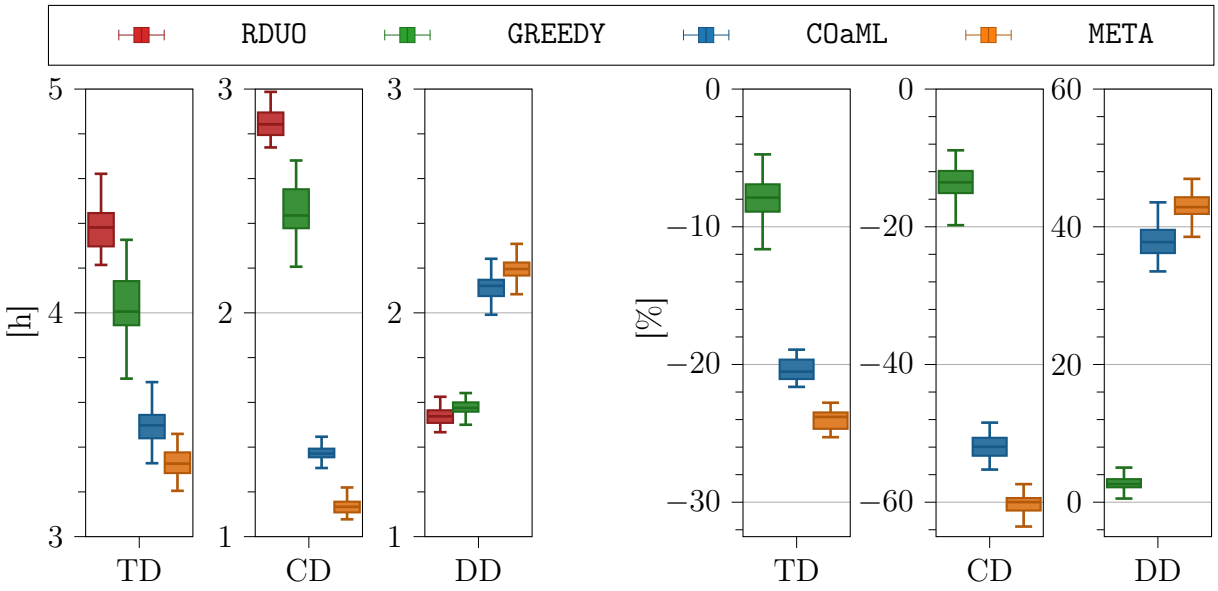
Computational setup The framework is implemented in `Julia` 1.10.9. All experiments were conducted on an `Intel(R) i9-9900` CPU at 3.1 GHz with 10 GB of RAM. The full implementation and experimental setup are available at https://github.com/coppolabs/online_bal_stag.

5.6 Results

Figure 5.6 reports test-set performance across the main KPIs for the considered algorithms. Specifically, we report both their absolute values and their percentage changes relative to `RDUO`, the selfish-routing baseline. Results reveal a clear performance hierarchy. The 4.4 h median total delay of the `RDUO` baseline decreases to 4.0 h under `GREEDY`, drops further to 3.5 h with `COaML`, and ultimately reaches 3.3 h for the `META` offline benchmark, corresponding to reductions of 8%, 21%, and 24%, respectively.

Result 12 *As the level of routing coordination increases, the system-wide total delay decreases in a clear, monotonic manner.*

A decomposition of total delay into congestion and detour components shows that the observed gains are primarily driven by reductions in congestion delay, amounting to approximately 14% for `GREEDY`, 54% for `COaML`, and 60% for `META`. These results highlight a key strength of `COaML`: the proposed pipeline achieves congestion reductions that approach those of the offline, full-foresight benchmark. In contrast, all coordination-based algorithms incur higher detour delays than the selfish baseline, albeit to varying extents: approximately 3% for `GREEDY`, 38% for `COaML`, and 42% for the `META` offline benchmark. Similar to `META`, `COaML` effectively leverages traffic balancing, converting detouring decisions into the substantial congestion relief observed above.



(a) Absolute KPI distributions.

(b) Relative variations to RDUO

Figure 5.6: Aggregate traffic performance metrics, presented in absolute values and as percentage changes relative to the RDUO baseline.

Result 13 *COaML achieves its performance gains primarily through pronounced congestion-delay reductions, while introducing detours in a disciplined manner that remains on par with the META offline benchmark.*

To quantify how much of the offline improvement is retained in real-time deployment, Figure 5.7 reports the share of the total delay reduction achieved by META that GREEDY and COaML recover online. COaML consistently captures a large share of the offline potential, achieving a median recovery of approximately 85%. Across all evaluated instances, performance remains above 80% with low dispersion, indicating strong robustness. In contrast, GREEDY captures only about 32% of the offline improvement and exhibits substantially higher variability, underscoring the performance loss associated with myopic coordination.

Result 14 *In online deployment, COaML preserves a substantial portion of the offline performance potential, delivering consistently high recovery rates, whereas GREEDY captures only a modest, and markedly more variable, share of the attainable gains.*

Figure 5.8 complements the aggregate KPIs analysis by illustrating how delays evolve over the simulation horizon. At first glance, RDUO and GREEDY display two pronounced delay peaks that mirror the bimodal demand pattern. In contrast, COaML and META exhibit a flatter delay profile, attenuating peak intensities through more effective traffic coordination. A closer look reveals that GREEDY exhibits a markedly state-dependent be-

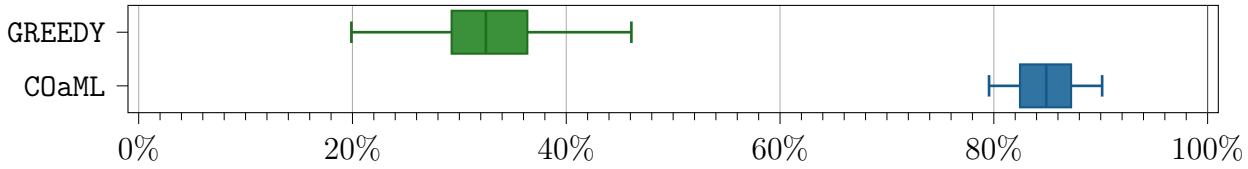


Figure 5.7: Recovery fractions of GREEDY and COaML solutions relative to META solutions.

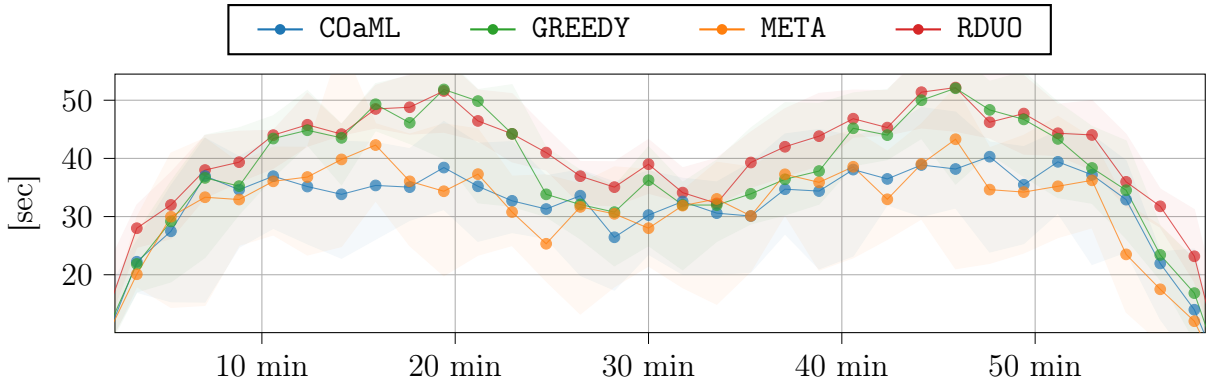
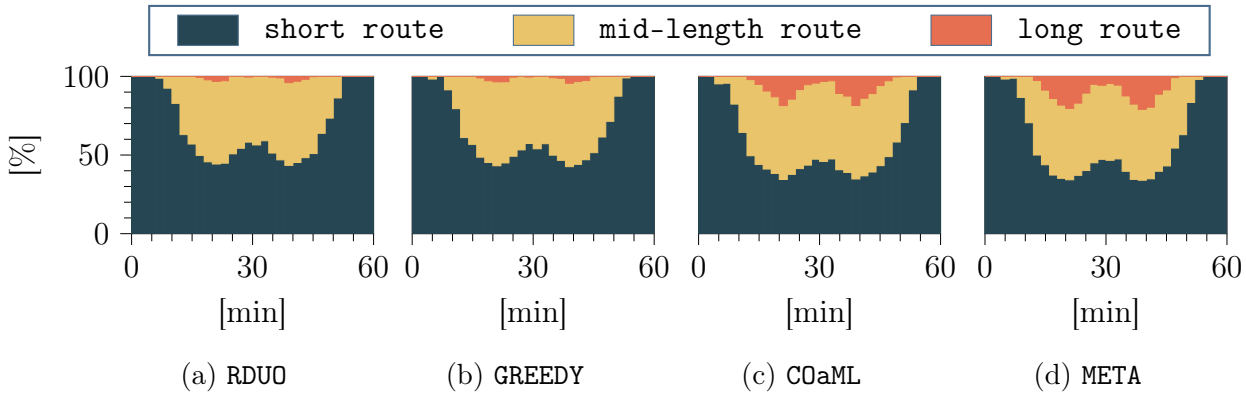


Figure 5.8: Aggregated trip delay progression over the simulation duration. Lines represent the median delay across all scenarios, while the shaded areas indicate the interquartile range.

havior. During low- and moderate-demand intervals, its median delays approach those of META, suggesting that in the presence of limited congestion externalities, myopic coordination can suffice for effective delay management. At demand peaks, however, GREEDY regresses toward RDUO-level performance, with the gap to META widening substantially. This pattern indicates that the benefits of coordination concentrate in highly congested regimes, where short-sighted routing decisions undermine system-level efficiency. COaML effectively mitigates peak-driven performance degradation by closely tracking the META benchmark across the entire horizon. Unlike GREEDY, it maintains coordination efficiency during high-congestion interval periods when performance gains are most impactful yet hardest to achieve.

Result 15 *Coordination is most critical at demand peaks.* ***COaML*** *preserves effective coordination in these high-congestion intervals, preventing the performance degradation observed under* ***GREEDY****.*

To understand the mechanisms underlying observed performances, we examine the balanced and staggered routing patterns induced by each strategy. Figure 5.9 reports, for each algorithm, the fraction of demand assigned to the short, mid-length, and long routes over time. Across all methods, the share of traffic on the short route follows a similar qualitative pattern. Away from demand peaks, nearly all trips are routed on the shortest



(a) RDUO (b) GREEDY (c) COaML (d) META

Figure 5.9: Temporal evolution of routing shares under each strategy. Stacked areas show, for each time bin, the fraction of demand assigned to the short, mid-length, and long routes.

path. During peak periods, this share drops to roughly 50% under the reactive strategies (RDUO and GREEDY) and to about 40% under the forward-looking strategies (COaML and META). Thus, differences in aggregate performance primarily stem from differences in the utilization of alternative routes. The reactive strategies allocate only a negligible share of traffic to the longest route – at most about 3% even at peak demand – thereby underutilizing available spatial capacity. In contrast, COaML closely mirrors the offline benchmark, routing up to 30% of trips via the long path during peaks and roughly 10% between peak periods. By diverting a larger share of traffic to the long route, COaML alleviates congestion on the short- and mid-length routes during peak demand, thereby enabling larger system-wide performance gains.

Finally, we examine the temporal staggering strategies induced by each algorithm. Figure 5.10 presents the interquartile range of trip-level departure shifts throughout the simulation for GREEDY, COaML, and META. At the boundaries of the horizon, all three methods exhibit comparable levels of staggering, reflecting the diminished need for anticipatory logic when demand is low. Distinct behaviors emerge during congestion surges: while GREEDY mirrors the META benchmark under moderate conditions, its temporal shift magnitude drops sharply during peak periods. Without foresight of the upcoming demand surge, GREEDY fails to smooth departures in advance and instead relies on reactive spatial rebalancing during heightened traffic. By contrast, COaML maintains consistent departure smoothing during the onset and duration of peak demand. Prior to the emergence of congestion waves, it adopts a more conservative staggering policy than META, preserving flexibility under demand uncertainty. At peak periods, however, it applies comparatively stronger temporal shifts, effectively leveraging the coordination capacity preserved in earlier phases. In doing so, COaML avoids the sharp collapse in temporal management observed under the GREEDY approach. Combined with the routing patterns discussed earlier, these

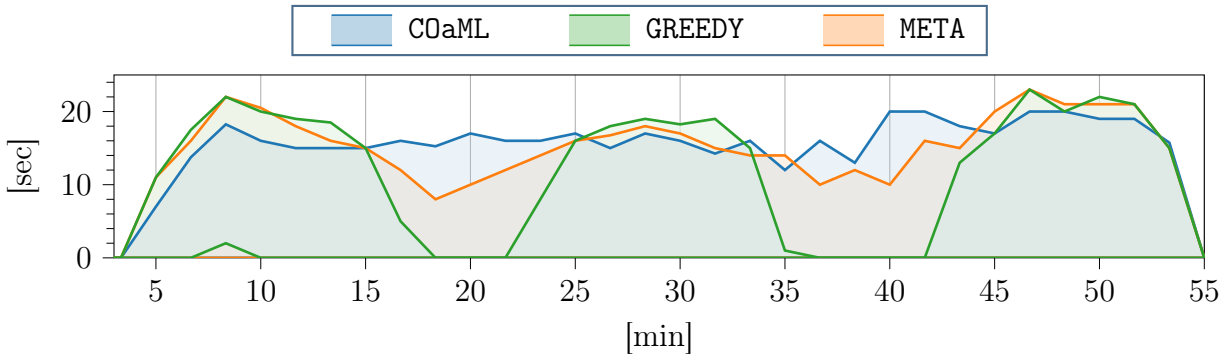


Figure 5.10: Temporal distribution of applied staggering. Shaded bands show the interquartile range of trip-level staggering within release-time bins.

results demonstrate how COaML adaptively integrates temporal and spatial mechanisms, sustaining departure shifts as congestion intensifies and complementing them with route reallocation when necessary.

Result 16 *Demand peaks expose the limits of reactive control: in these phases, alternative routes remain underexploited and departure smoothing collapses. In contrast, COaML anticipates congestion, preserves coordination capacity during low-traffic periods, and strategically deploys it as traffic pressure intensifies.*

5.7 Conclusion

In this work, we addressed the online balanced and staggered routing problem for AMoD systems, where real-time trip assignments must minimize expected system-wide travel time under demand uncertainty. We introduced a COaML pipeline that bridges the gap between myopic heuristics and anticipatory coordination by integrating combinatorial optimization with imitation learning. The core of this approach is a reformulation of the assignment task as a shortest-path problem on a time-expanded graph, combined with a ML predictor that parameterizes the graph's arc weights. In this way, the model encodes anticipated demand interactions into a differentiable control layer, effectively reproducing the forward-looking logic of an offline benchmark within a real-time decision framework.

Numerical experiments show that the proposed COaML pipeline substantially outperforms reactive heuristics, particularly in high-congestion regimes where coordination is most critical. By reducing median total delay by 21% relative to the selfish baseline, the pipeline more than doubles the 8% improvement achieved by greedy coordination. These gains are primarily driven by a 54% reduction in congestion-induced delay. Notably, the proposed method robustly recovers 85% of the total offline performance potential, consistently remaining above 80% across instances, whereas reactive approaches capture

only 32% and exhibit pronounced variability. Structural analysis indicates that these improvements arise from the coordinated interaction of spatial and temporal mechanisms. In contrast to myopic approaches, COaML preserves coordination capacity during moderate-demand phases and deploys it as congestion intensifies. By combining sustained departure staggering with balanced routing, the pipeline maintains high-efficiency coordination precisely when it matters most, narrowing the gap to full-foresight offline planning.

Future research should extend the methodology to larger networks and explicitly incorporate fleet-level constraints, such as limited fleet size, vehicle repositioning dynamics, and stochastic background traffic. Another important direction is to reduce reliance on offline benchmarks. Owing to the combinatorial complexity of joint routing and staggering decisions, obtaining optimal labels for realistic instances remains computationally intractable, requiring training to rely on high-quality heuristic approximations that may impose a performance ceiling. Transitioning to a structured reinforcement learning framework would decouple the approach from expert solutions and enable the model to discover novel coordination strategies that could exceed current performance limits.

A5.1 Shortest path computation on DAGs

Algorithm 13 presents the pseudocode for computing the shortest path on the time-expanded graphs constructed using Algorithm 12. These graphs are DAGs, which enables efficient shortest path computation in a single pass over a topological ordering of the nodes.

Algorithm 13 DAGSHORTESTPATH

Input: DAG $\mathcal{G} = (\mathcal{V}, \mathcal{A})$, arc weights θ, source node `src`, destination node `tgt`

Initialize distance vector $\texttt{dist}[v] \leftarrow \infty$ for all $v \in \mathcal{V}$; $\texttt{dist}[\texttt{src}] \leftarrow 0$
Initialize predecessor vector $\texttt{pred}[v] \leftarrow \texttt{null}$ for all $v \in \mathcal{V}$
for $u \in$ TOPOLOGICALSORT($\mathcal{G}$) **do**
    for $(u, v) \in$ OUTGOINGARCS(u) **do**
        $\texttt{alt} \leftarrow \texttt{dist}[u] + \theta[u, v]$
        if $\texttt{alt} < \texttt{dist}[v]$ **then**
            $\texttt{dist}[v] \leftarrow \texttt{alt}$; $\texttt{pred}[v] \leftarrow u$
Initialize empty list $\texttt{path} \leftarrow [\,]$
$v \leftarrow \texttt{tgt}$
while $v \neq \texttt{null}$ **do**
    Prepend v to `path`
    $v \leftarrow \texttt{pred}[v]$
if $\texttt{path}[0] \neq \texttt{src}$ **then**
    return `null` ▷ No path exists
return `path`

A5.2 Metaheuristic

We consider the offline integrated balanced and staggered routing problem, which constitutes the full-information counterpart of the online problem introduced in Section 5.3. In this setting, all trip requests over the planning horizon are known in advance, and decisions are made jointly rather than sequentially. A solution is therefore represented by a single global assignment $\pi = (p, s)$, where p and s denote the vectors of assigned routes and departure times, respectively, for all trips in the instance. Algorithm 14 summarizes the tailored metaheuristic proposed by Coppola et al. (2025b) for this offline problem. We briefly outline its main components and refer the reader to the original paper for full details. The algorithm combines constructive and improvement heuristics, all of which rely on the following lower-level operators:

(i) *Insert:* sequentially inserts an ordered list of trips into a partial solution.

(ii) *Remove:* perturbs a solution by removing a selected subset of trips.

(iii) *Local search:* re-optimizes assignments for a selected subset of trips.

The algorithm first generates two initial solutions using constructive heuristics:

(i) *RDUO:* a selfish policy that assigns each trip to its earliest feasible departure time and shortest-time route, thereby minimizing individual travel time.

(ii) *Greedy assignment:* assigns trips in order of increasing slack to the route–departure pair minimizing system-wide travel time, repairing any infeasibilities via local search.

A LNS procedure then refines these solutions. It is initialized with the better of the two, defined as the solution π minimizing the penalized objective:

$$\mathrm{C}(\pi) = \mathrm{TD}(\pi) + \alpha\, \mathrm{I}(\pi),$$

where $\mathrm{TD}(\pi)$ denotes total travel delay and $\alpha \geq 0$ adaptively weights the infeasibility term $\mathrm{I}(\pi)$, computed as the time trips travel beyond their deadlines. The LNS then iteratively applies destroy-and-repair cycles. At each iteration, the destroy operator targets a subset of trips – either the *most costly* or the *least perturbed* ones – to form a removal pool. The algorithm then performs a sequence of destruction–repair cycles, each time selecting a subset of trips from this pool and reinserting them in randomized order. If the solution

Algorithm 14 Metaheuristic overview

Input: Set of trips $\mathcal{R}$

$\pi_{\mathrm{RDUO}} \leftarrow$ buildRDUO($\mathcal{R}$)
$\pi_{\mathrm{greedy}} \leftarrow$ greedyAssignment($\mathcal{R}$)
$\pi \leftarrow \arg\min\{C(\pi_{\mathrm{RDUO}}),\ C(\pi_{\mathrm{greedy}})\}$
$\pi \leftarrow \mathrm{LNS}(\pi)$
return π

improves, the search continues with the same targeting strategy; otherwise, a local search step is applied, and the algorithm switches to the next targeting strategy. If, after trying all targeting strategies within an iteration, local search yields an improvement, a new iteration starts from the first targeting strategy; otherwise, the algorithm terminates.

A5.3 Description of `constructState`

Algorithm 15 details the procedure for constructing the system state S associated with a solution π, which details a joint route and departure time assignment of a set of trips $\mathcal{R}$. We start by sorting the trip start times s^r into a priority queue Q and initializing an empty set $\mathcal{T}_a$ for each arc a to track when each trip completes the traversal of a. Then, we construct an initial solution iteratively:

i) We start by extracting the departure time $\underline{s}_a^r$ with the highest priority from Q.

ii) We then set $f_a^r = |\{\bar{s}_a^{r'} \in \mathcal{T}_a \,|\, \underline{s}_a^r < \bar{s}_a^{r'}\}|$, that is the number of trips that have not completed their traversal of arc a by time $\underline{s}_a^r$, and compute $d_a^r = d(f_a^r)$.

iii) Finally, we add the time in which the arc traversal is completed, which is $\underline{s}_a^r + d_a^r + \underline{\tau}_a$, to $\mathcal{T}_a$ by setting $\mathcal{T}_a = \mathcal{T}_a \cup (\underline{s}_a^r + d_a^r + \underline{\tau}_a)$. If a is not the last arc of route p^r, we insert $\underline{s}_a^r + d_a^r + \underline{\tau}_a$ into Q.

We repeat these steps until Q is empty.

A5.4 The feature mapping

In this section, we describe the feature mapping

$$\phi : \; x_t \;\longmapsto\; \left\{ \phi_a^s(x_t) \in \mathbb{R}^d \;\middle|\; (s,a) \in \mathcal{A}^t \right\},$$

Algorithm 15 constructState

Input: Solution π

Q $\leftarrow$ initializePriorityQueue(π)
$S \leftarrow$ initializeSchedule(π)
while Q is not empty **do**
  $(r, a) \leftarrow$ Q.pop()
  $f_a^r \leftarrow$ computeFlowOnArc($\underline{s}_a^r$)
  $d_a^r \leftarrow$ computeDelay(f_a^r)
  $a' \leftarrow$ getNextArc(r, a, π)
  $\underline{s}_{a'}^r \leftarrow \underline{s}_a^r + \underline{\tau}_a + d_a^r$
  $S \leftarrow$ addDeparture($\underline{s}_{a'}^r$)
  if $a' \neq \bar{a}_r$ **then**
    Q $\leftarrow$ pushTrip(r, a')
return S

which maps the system state x_t, constructed at the reveal time $t = \underline{e}^r$ of trip r, to a d-dimensional feature vector associated with each arc (s, a) of the time-expanded graph $\mathcal{G}^t$. Table 5.2 provides an overview of the selected features. The coordinates of $\phi_a^s(x_t)$ capture information at three abstraction levels: arc, route, and trip. Arc-level features encode local congestion and timing conditions on individual arcs. Route-level features aggregate arc-level information to represent the traffic conditions induced by a route and departure time choice, such as cumulated travel times and delays, and feasibility with respect to deadlines. Trip-level features provide global temporal context, describing request characteristics and their position within the demand stream. To compute features, we start from the system state x_t, in which the newly revealed trip r is unassigned at its origin and does not yet affect traffic. To evaluate a candidate route–departure choice, we

Feature label	**Description**
Arc-level features	
Arc start time	Time at which the trip enters the arc
Arc arrival time	Time at which the trip exits the arc
Actual arc travel time	Travel time experienced on the arc
Delay on arc	Congestion-induced delay on the arc
Delay ratio on arc	Congestion delay normalized by actual arc travel time
Flow on arc	Number of concurrent trips on the arc at entry
Nominal arc time	Free-flow travel time of the arc
Arc nominal fraction	Fraction of route free-flow time attributable to the arc
Route-level features	
Route start time	Departure time at which the route is initiated
Route nominal travel time	Total free-flow travel time of the route
Route actual travel time	Total congestion-aware travel time of the route
Route congestion delay	Total congestion delay accumulated along the route
Route arrival time	Arrival time at the destination when following the route
Route infeasibility	Positive lateness with respect to the deadline
Route slack	Remaining earliness before the deadline
Route detour delay	Additional free-flow time from the chosen route
Staggering time	Departure delay applied if the route is selected
Normalized staggering time	Departure delay normalized by max. allowed staggering
Trip-level features	
Release time	Trip release time
Normalized release time	Release time normalized by the release horizon
Latest departure time	Latest feasible departure time
Max staggering window	Maximum feasible departure delay
Deadline	Latest allowable arrival time
Deadline window	Time window between release and deadline
Previous release times (normalized)	Normalized release times of the ten most recent prior trips

Table 5.2: Arc-, route-, and trip-level input features.

construct a counterfactual state in which r is assigned to that route and departure time and simulate its interaction with existing traffic. We then compute the corresponding features for all arcs of the associated time-expanded route. Repeating this process for each candidate route in $\mathcal{G}^t$ yields feature vectors for all feasible assignments, which the predictor uses to select the assignment to apply.

We manually selected the feature set based on validation performance. We did not apply an explicit feature selection procedure, as perturbations of the parameter vector during training serve as an implicit regularizer, shrinking the weights of irrelevant features toward zero. We standardize all features using their empirical means and standard deviations prior to training.

A5.5 Hyperparameter tuning

To define the statistical model, we consider two families of differentiable encoders:

(i) *Multilayer perceptrons (MLP):* A feedforward network that independently maps arc-level features to arc weights, serving as a network-agnostic baseline.

(ii) *GNN:* A relational encoder that captures spatial dependencies via message passing, modeling arcs as nodes in an undirected learning graph to enable bidirectional information flow between adjacent components.

As summarized in Table 5.3, we evaluate small- and medium-scale variants within each family to assess the effect of model capacity. We evaluate all architectures over an initial, broad hyperparameter sweep defined by four perturbation intensities ε and four learning rate schedules, summarized in Figure 5.11. For all experiments, the number of perturbation samples is fixed at $z = 50$. Each architecture—hyperparameter combination is trained independently on a dataset of 40 instances, each containing 400 trips, for a total of 16000 data points constructed according to the procedure described in Section 5.4.2.1. To mitigate potential correlations induced by data-point ordering, we reshuffle the training data at the start of each epoch. At the end of each epoch, we evaluate the learned

Architecture	**Type (with number of layers)**	**Layer widths**
`MLP_S`	MLP (3)	$F \to 32 \to 16 \to 1$
`MLP_M`	MLP (3)	$F \to 64 \to 32 \to 1$
`GNN_S`	GNN (3 GCN)	$F \to 32 \to 16 \to 1$
`GNN_M`	GNN (3 GCN)	$F \to 64 \to 32 \to 1$

Table 5.3: Summary of considered encoder architectures with input dimension F. MLPs utilize dense transformations with batch normalization; GNNs employ relational graph convolutional (GCN) blocks. All configurations apply ReLU nonlinearities and a final flattening stage to produce scalar arc weights.

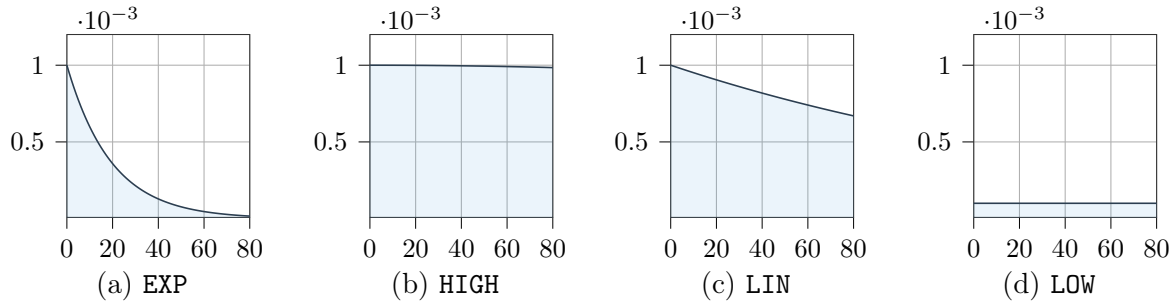


Figure 5.11: Learning rate schedules considered in the hyperparameter sweep.

policy on 10 validation instances, beginning from epoch 10 to avoid distortions caused by unstable early-training dynamics. Performance is quantified by the mean percentage gap relative to the offline benchmark:

$$\widetilde{\Delta} = \frac{1}{10} \sum_{i=1}^{10} \frac{\mathrm{C}(\pi_i^{\mathrm{ON}}) - \mathrm{C}(\pi_i^{\mathrm{OFF}})}{\mathrm{C}(\pi_i^{\mathrm{OFF}})}.$$

Here, $\mathrm{C}(\pi)$ denotes a penalized cost function defined as

$$\mathrm{C}(\pi) = \mathrm{TD}(\pi) + \alpha\, \mathrm{I}(\pi),$$

which sums the total delay of the solution π and the total time exceeding the specified deadlines $\mathrm{I}(\pi)$, scaled by the penalty coefficient $\alpha = 100$. For each configuration in this first-stage sweep, we retain the checkpoint achieving the minimum $\widetilde{\Delta}$ within a fixed 48-hour training window. Figure 5.12 visualizes the validation performance across the explored hyperparameter grid. The results indicate that GNN-based architectures consistently outperform MLP variants, suggesting that incorporating graph-structured information substantially improves predictive accuracy, with the best-performing configuration achieving a mean validation gap of approximately 8%.

Building on these findings, we fix the best-performing configuration from the global sweep and proceed to a second, more focused model-selection stage. We evaluate its robustness across 10 training seeds, each governing three sources of stochasticity: random weight initialization, stochastic input perturbations, and data reshuffling at the start of each epoch. This analysis is conducted under both stochastic gradient descent and mini-batch training within a 48-hour time window. The configuration achieving the strongest validation performance is retained for subsequent deployment. As illustrated in Figure 5.13, training dynamics vary substantially across batch sizes. Training with SGD yields comparatively low validation gaps and moderate yet controlled variability across seeds. In contrast, batch size = 8 exhibits pronounced instability: the mean validation gap increases markedly during mid-training and remains elevated, accompanied by a wide

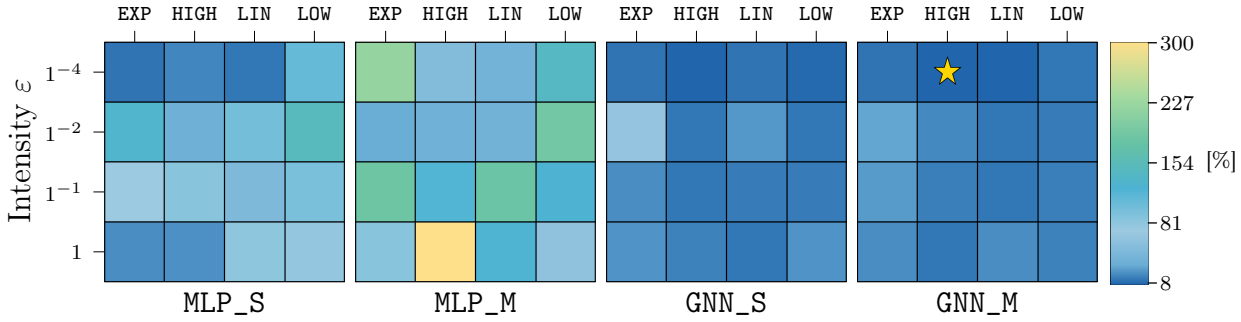
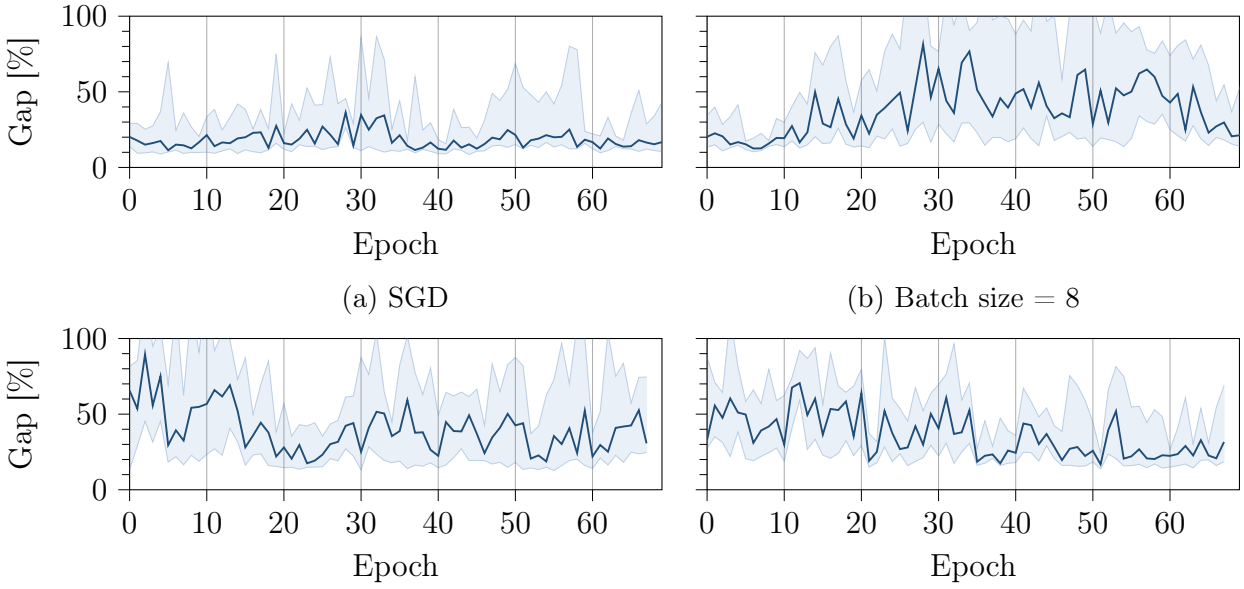


Figure 5.12: Validation performance across hyperparameter combinations. Each architecture sub-grid evaluates learning-rate schedules and perturbation intensities. The star marks the best-performing configuration.



(a) SGD

(b) Batch size = 8

(c) Batch size = 32

(d) Batch size = 64

Figure 5.13: Mean validation gap and interquartile range across 10 seeds for different batch sizes.

interquartile range, indicating limited robustness. Increasing the batch size to 32 mitigates some of this volatility, yet noticeable oscillations and dispersion persist, particularly in early epochs. Increasing the batch size to 64 restores stability: the mean validation gap exhibits a clearer downward trend, and the interquartile range tightens over time, indicating consistent convergence across seeds.

6 Conclusion

This thesis examined centralized spatiotemporal coordination as a proactive instrument for congestion mitigation in AMoD systems. Motivated by the systemic inefficiencies of uncoordinated fleet behavior, it addressed the overarching question: *how can balanced routing and departure time staggering be embedded within real-time AMoD control to align fleet-level objectives with network-wide efficiency?*

The thesis progressed through three stages, each building on the previous. It first quantified the congestion-mitigation potential of temporal control in isolation, then established the complementary value of integrating spatial balancing with departure staggering as traffic intensity increased, and finally investigated how these coordination principles can be deployed in real time under stochastic demand. Collectively, the findings demonstrate that spatiotemporal coordination is not only theoretically compelling but practically attainable. Integrated control delivers substantial delay reductions under full information through robust spatial–temporal synergies in realistic congestion regimes. Crucially, a large share of these gains remains achievable under demand uncertainty and stringent computational constraints. By translating system-optimal traffic principles into implementable fleet-level control policies, this thesis establishes an actionable framework through which AMoD systems can transform spatial and temporal flexibility into sustained congestion mitigation in real-world settings.

The remainder of this chapter synthesizes the principal insights, discusses limitations, and outlines directions for future research.

6.1 Key findings

A first central conclusion of this thesis concerns the effectiveness of *temporal control* in AMoD operations. Allowing limited shifts in departure times, while preserving arrival-time commitments, turns temporal flexibility into an operational resource. Experiments on the Manhattan network show that departure coordination yields substantial congestion relief. In low-congestion regimes, where congestion is primarily induced by the AMoD fleet, staggered routing mitigates nearly all fleet-induced congestion under full information, reducing average congestion by approximately 98%. While this represents an upper

bound, a simple rolling-horizon implementation still recovers about 82% of these gains. In traffic regimes where interactions with exogenous traffic are more pronounced, temporal control alone reduces total delay by approximately 60% under full information and by about 30% in online settings relative to reactive baselines.

Key finding 1 (Staggering is effective with minimal disruption) *Temporal staggering is a powerful and practically viable congestion-management lever in AMoD systems. Under full information, it mitigates nearly all fleet-induced congestion in low-congestion regimes and reduces total delay by about 60% under moderate exogenous traffic. Even in rolling-horizon online settings, roughly 80% of the low-congestion gains and 30% of the moderate-congestion gains are recovered, requiring only small departure shifts while fully preserving arrival-time constraints.*

A second central conclusion of this thesis is that, as traffic volumes increase, temporal control alone becomes insufficient. Under heavy load, congestion arises not only from peak timing but also from the spatial concentration of flows along a limited set of critical corridors, rendering explicit route balancing, alongside temporal staggering, essential. Large-scale experiments on the Manhattan network under realistic peak-hour demand show that joint spatial–temporal coordination delivers substantially larger and more robust system-wide improvements than either mechanism alone. Under full control, the integrated approach reduces total system delay by up to 25% relative to selfish routing, with congestion-induced delay reduced by up to approximately 35%. These gains arise from a clear division of roles: route balancing relieves pressure on overloaded corridors, while temporal staggering smooths residual peaks without creating new bottlenecks. Benefits of centralized routing persist under partial control: even when only a fraction of trips is centrally managed, coordinated spatial–temporal control improves outcomes for both controlled and exogenous traffic under welfare- and profit-oriented objectives.

Key finding 2 (Balancing and staggering are complementary) *Spatial route balancing and temporal departure staggering are complementary control levers. Under strong congestion, integrated coordination reduces total system delay by up to 25% and congestion-induced delay by up to approximately 35% relative to selfish routing, with benefits persisting even when control is limited to a fraction of traffic.*

A third central conclusion is that the performance gains identified under full information remain largely attainable in real-time settings. Building on the demonstrated benefits of spatial and temporal coordination, this thesis shows that their anticipatory structure can be embedded within scalable online controllers. In real-time deployment, the proposed framework recovers 85% of the offline performance potential, compared to only 32% un-

der reactive strategies, demonstrating that effective spatiotemporal coordination remains achievable despite demand uncertainty and computational constraints.

Key finding 3 (Offline performance can be largely recovered online) *Embedding combinatorial structure within a learning-augmented control framework substantially reduces the gap between the potential for offline coordination and real-time performance. In deployment, the framework recovers over 80% of the offline gains and lowers the median total delay by 21% relative to selfish baselines—more than doubling the gains of reactive coordination.*

6.2 Limitations and future research

This thesis opens several directions for future research, including operational extensions, integration within broader urban and regulatory environments, stronger algorithmic guarantees, learning–based policy improvement, and expanded empirical validation.

A first line of future research concerns operational completeness. The current framework abstracts from explicit fleet-size constraints, detailed dispatching and vehicle–request matching decisions, and their interaction with routing and departure-time control. Integrating these layers within a unified model would provide a more comprehensive representation of AMoD operations and enable the study of coordination across the full fleet-management stack.

Beyond the operational modeling limitations discussed above, the framework abstracts from the broader urban and institutional environment in which AMoD systems operate, including interactions with other modes, competing operators, and regulatory mechanisms. Future research should therefore examine balanced and staggered routing in conjunction with public transit coordination, dynamic congestion pricing, and platform-level regulatory frameworks. Analyzing these interactions would clarify how spatiotemporal fleet control shapes long-term mode choice, infrastructure utilization, and welfare outcomes. Such extensions would elevate these coordination strategies from isolated fleet-management instruments to systemic components of integrated urban transportation planning.

On the algorithmic side, the NP-hardness of the joint routing and scheduling problem raises two distinct challenges for future research. First, the optimality gap of the proposed methods remains uncharacterized, as the distance to the true global optimum is unknown. Developing tighter bounds, improved relaxation schemes, or exact decomposition techniques for restricted subproblems would allow a more rigorous quantification of the remaining coordination potential. Second, the quality of offline solutions may impose an intrinsic performance ceiling on real-time control. Because the COaML pipeline is trained via imitation learning, its performance is fundamentally bounded by the quality

of the expert solutions used for supervision. Future research could relax this dependency by transitioning toward structured RL formulations. By eliminating reliance on expert labels, a reinforcement learning framework could explore a broader coordination policy space and potentially outperform existing heuristics.

Finally, the empirical evaluation performed reflects a trade-off between network realism and methodological depth. While the offline analysis employs a high-fidelity model of Manhattan, the real-time deployment experiments rely on simplified parallel networks to isolate and validate the learning mechanism. Extending learning-augmented control to large-scale urban grids with complex topologies and richer congestion dynamics represents an interesting future direction. Beyond scaling within a single city, future work should also examine diverse network structures and demand regimes to evaluate the robustness and transferability of spatiotemporal coordination across heterogeneous urban environments.

Bibliography

Aalipour, A., & Khani, A. (2024). Modeling, analysis, and control of autonomous mobility-on-demand systems: A discrete-time linear dynamical system and a model predictive control approach. *IEEE Transactions on Intelligent Transportation Systems*, *25*, 8615–8628.

Abraham, I., Delling, D., Goldberg, A. V., & Werneck, R. F. (2013). Alternative routes in road networks. *Journal of Experimental Algorithmics (JEA)*, *18*, 1–1.

Adacher, L., & Tiriolo, M. (2018). A macroscopic model with the advantages of microscopic model: A review of cell transmission model's extensions for urban traffic networks. *Simulation Modelling Practice and Theory*, *86*, 102–119.

Adnan, M., Pereira, F. C., Azevedo, C. M. L., Basak, K., Lovric, M., Raveau, S., Zhu, Y., Ferreira, J., Zegras, C., & Ben-Akiva, M. (2016). Simmobility: A multi-scale integrated agent-based simulation platform. *95th Annual Meeting of the Transportation Research Board Forthcoming in Transportation Research Record*, *2*.

Agarwal, S., Mani, D., & Telang, R. (2023). The impact of ride-hailing services on congestion: Evidence from indian cities. *Manufacturing & Service Operations Management*, *25*(3), 862–883.

Aghamohammadi, R., & Laval, J. A. (2020). Dynamic traffic assignment using the macroscopic fundamental diagram: A review of vehicular and pedestrian flow models. *Transportation Research Part B: Methodological*, *137*, 99–118.

Al-Abbasi, A. O., Ghosh, A., & Aggarwal, V. (2019). Deeppool: Distributed model-free algorithm for ride-sharing using deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, *20*(12), 4714–4727.

Alizadeh, M., Wai, H.-T., Chowdhury, M., Goldsmith, A., Scaglione, A., & Javidi, T. (2017). Optimal pricing to manage electric vehicles in coupled power and transportation networks. *IEEE Transactions on Control of Network Systems*, *4*(4), 863–875.

Alonso-Mora, J., Wallar, A., & Rus, D. (2017). Predictive routing for autonomous mobility-on-demand systems with ride-sharing. *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3583–3590.

Alotaibi, E. T., Alhuzaymi, T. M., & Herrmann, J. M. (2023). Autonomous mobility on demand: From case studies to standardized evaluation. *Frontiers in future transportation*, *4*, 1224322.

Arnott, R., de Palma, A., & Lindsey, R. (1990a). Departure time and route choice for the morning commute. *Transportation Research Part B: Methodological*, *24*(3), 209–228.

Arnott, R., de Palma, A., & Lindsey, R. (1990b). Economics of a bottleneck. *Journal of Urban Economics*, *27*(1), 111–130.

Ashkrof, P., de Almeida Correia, G. H., Cats, O., & van Arem, B. (2024). On the relocation behavior of ride-sourcing drivers. *Transportation Letters*, *16*(4), 330–337.

Aubin-Frankowski, P.-C., Castro, Y. D., Parmentier, A., & Rudi, A. (2025). Generalization bounds of surrogate policies for combinatorial optimization problems. *arXiv:2407.17200 [stat.ML]*. https://doi.org/10.48550/arXiv.2407.17200

Axhausen, K. W., Horni, A., & Nagel, K. (2016). *The multi-agent transport simulation matsim*. Ubiquity Press.

Azevedo, C. L., Marczuk, K., Raveau, S., Soh, H., Adnan, M., Basak, K., Loganathan, H., Deshmunkh, N., Lee, D.-H., Frazzoli, E., et al. (2016). Microsimulation of demand and supply of autonomous mobility on demand. *Transportation Research Record*, *2564*(1), 21–30.

Bahrami, S., & Roorda, M. J. (2020). Optimal traffic management policies for mixed human and automated traffic flows. *Transportation Research Part A: Policy and Practice*, *135*, 130–143.

Bang, H., & Malikopoulos, A. A. (2022). Congestion-aware routing, rebalancing, and charging scheduling for electric autonomous mobility-on-demand system. *2022 American Control Conference (ACC)*, 3152–3157.

Baptiste, P., Le Pape, C., & Nuijten, W. (2001). *Constraint-based scheduling: Applying constraint programming to scheduling problems* (Vol. 39). Springer Science & Business Media.

Bartusch, M., Möhring, R. H., & Radermacher, F. J. (1988). Scheduling project networks with resource constraints and time windows. *Annals of Operations Research*, *16*, 199–240.

Basu, R., Araldo, A., Akkinepally, A. P., Nahmias Biran, B. H., Basak, K., Seshadri, R., Deshmukh, N., Kumar, N., Azevedo, C. L., & Ben-Akiva, M. (2018). Automated mobility-on-demand vs. mass transit: A multi-modal activity-driven agent-based simulation approach. *Transportation Research Record*, *2672*(8), 608–618.

Battifarano, M., & Qian, S. (2023). The impact of optimized fleets in transportation networks. *Transportation Science*, *57*(4), 1047–1068.

Baty, L., Jungel, K., Klein, P. S., Parmentier, A., & Schiffer, M. (2024). Combinatorial optimization-enriched machine learning to solve the dynamic vehicle routing problem with time windows. *Transportation Science*, *58*(4), 708–725.

Belakaria, S., Ammous, M., Smith, L., Sorour, S., & Abdel-Rahim, A. (2019). Multi-class management with sub-class service for autonomous electric mobility on-demand systems. *IEEE Transactions on Vehicular Technology*, *68*(7), 7155–7159.

Bennett, A. (2006). *Lagrangian fluid dynamics*. Cambridge University Press.

Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J.-P., & Bach, F. (2020). Learning with differentiable pertubed optimizers. *Advances in Neural Information Processing Systems*, *33*, 9508–9519.

Bertsimas, D., Jaillet, P., & Martin, S. (2019). Online vehicle routing: The edge of optimization in large-scale applications. *Operations Research*, *67*(1), 143–162.

Bertsimas, D., & Kallus, N. (2020). From predictive to prescriptive analytics. *Management Science*, *66*(3), 1025–1044.

Bhaskar, U., Ligett, K., Schulman, L. J., & Swamy, C. (2014). Achieving target equilibria in network routing games without knowing the latency functions. *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, 31–40.

Bischoff, J., & Maciejewski, M. (2016). Simulation of city-wide replacement of private cars with autonomous taxis in berlin. *Procedia Computer Science*, *83*, 237–244.

Blondel, M., Martins, A. F., & Niculae, V. (2020). Learning with fenchel-young losses. *Journal of Machine Learning Research*, *21*(35), 1–69.

Boesch, P. M., Ciari, F., & Axhausen, K. W. (2016). Autonomous vehicle fleet sizes required to serve different levels of demand. *Transportation Research Record*, *2542*(1), 111–119.

Boewing, F., Schiffer, M., Salazar, M., & Pavone, M. (2020). A vehicle coordination and charge scheduling algorithm for electric autonomous mobility-on-demand systems. *2020 American Control Conference (ACC)*, 248–255.

Bonifaci, V., Harks, T., & Schäfer, G. (2010). Stackelberg routing in arbitrary networks. *Mathematics of Operations Research*, *35*(2), 330–346.

Bouvier, L., Prunet, T., Leclère, V., & Parmentier, A. (2025). Primal-dual algorithm for contextual stochastic combinatorial optimization. *arXiv:2505.04757 [cs.LG]*. https://doi.org/10.48550/arXiv.2505.04757

Boyd, S., & Vandenberghe, L. (2004). *Convex optimization*. Cambridge University Press.

Braverman, A., Dai, J. G., Liu, X., & Ying, L. (2019). Empty-car routing in ridesharing systems. *Operations Research*, *67*(5), 1437–1452.

Brown, A., & LaValle, W. (2021). Hailing a change: Comparing taxi and ridehail service quality in los angeles. *Transportation*, *48*(2), 1007–1031.

Cáp, M., & Alonso-Mora, J. (2018). Multi-objective analysis of ridesharing in automated mobility-on-demand. In H. Kress-Gazi, S. Srinivasa, T. Howard, & N. Atanasov (Eds.), *Proceedings of RSS 2018: Robotics - Science and Systems XIV*.

Carey, M., & Watling, D. (2012). Dynamic traffic assignment approximating the kinematic wave model: System optimum, marginal costs, externalities and tolls. *Transportation Research Part B: Methodological*, *46*(5), 634–648.

Carron, A., Seccamonte, F., Ruch, C., Frazzoli, E., & Zeilinger, M. N. (2021). Scalable model predictive control for autonomous mobility-on-demand systems. *IEEE Transactions on Control Systems Technology*, *29*(2), 635–644.

Chabini, I. (1998). Discrete dynamic shortest path problems in transportation applications: Complexity and algorithms with optimal run time. *Transportation Research Record*, *1645*(1), 170–175.

Chen, T. D., & Kockelman, K. M. (2016). Management of a shared autonomous electric vehicle fleet: Implications of pricing schemes. *Transportation Research Record*, *2572*(1), 37–46.

Chen, Z., He, F., Zhang, L., & Yin, Y. (2016). Optimal deployment of autonomous vehicle lanes with endogenous market penetration. *Transportation Research Part C: Emerging Technologies*, *72*, 143–156.

Chondrogiannis, T., Bouros, P., Gamper, J., Leser, U., & Blumenthal, D. B. (2020). Finding k-shortest paths with limited overlap. *The VLDB Journal*, *29*(5), 1023–1047.

Clewlow, R. R., & Mishra, G. S. (2017). *Disruptive transportation: The adoption, utilization, and impacts of ride-hailing in the united states* (Research Report). Institute of Transportation Studies, University of California, Davis.

Coppola, A., Hiermann, G., Paccagnan, D., Gendreau, M., & Schiffer, M. (2025a). Integrated balanced and staggered routing in autonomous mobility-on-demand systems. *arXiv:2506.19722 [math.OC]*. https://doi.org/10.48550/arXiv.2506.19722

Coppola, A., Hiermann, G., Paccagnan, D., & Schiffer, M. (2025b). Staggered routing in autonomous mobility-on-demand systems. *European Journal of Operational Research*, *327*(3), 875–891.

Daganzo, C. F. (1994). The cell transmission model: A dynamic representation of highway traffic consistent with the hydrodynamic theory. *Transportation Research Part B: Methodological*, *28*(4), 269–287.

Daganzo, C. F. (2007). Urban gridlock: Macroscopic modeling and mitigation approaches. *Transportation Research Part B: Methodological*, *41*(1), 49–62.

Dalle, G., Baty, L., Bouvier, L., & Parmentier, A. (2022). Learning with combinatorial optimization layers: A probabilistic approach [Version 2]. *HAL preprint*. https://hal.science/hal-03739485v2

De Palma, A., Kilani, M., & Lindsey, R. (2005). Comparison of second-best and third-best tolling schemes on a road network. *Transportation Research Record*, *1932*(1), 89–96.

De Palma, A., & Lindsey, R. (2011). Traffic congestion pricing methodologies and technologies. *Transportation Research Part C: Emerging Technologies*, *19*(6), 1377–1399.

Diao, M., Kong, H., & Zhao, J. (2021). Impacts of transportation network companies on urban mobility. *Nature Sustainability*, *4*(6), 494–500.

Djavadian, S., Hoogendoorn, R. G., Van Arerm, B., & Chow, J. Y. (2014). Empirical evaluation of drivers' route choice behavioral responses to social navigation. *Transportation Research Record*, *2423*(1), 52–60.

Dong, T., Luo, Q., Xu, Z., Yin, Y., & Wang, J. (2024). Strategic driver repositioning in ride-hailing networks with dual sourcing. *Transportation Research Part C: Emerging Technologies*, *158*, 104450.

Elmachtoub, A. N., & Grigas, P. (2022). Smart "predict, then optimize". *Management Science*, *68*(1), 9–26.

Enders, T., Harrison, J., Pavone, M., & Schiffer, M. (2023). Hybrid multi-agent deep reinforcement learning for autonomous mobility on demand systems. *Learning for Dynamics and Control Conference*, 1284–1296.

Erhardt, G. D., Roy, S., Cooper, D., Sana, B., Chen, M., & Castiglione, J. (2019). Do transportation network companies decrease or increase congestion? *Science Advances*, *5*(5), eaau2670.

Estandia, A., Schiffer, M., Rossi, F., Luke, J., Kara, E. C., Rajagopal, R., & Pavone, M. (2021). On the interaction between autonomous mobility on demand systems and power distribution networks - an optimal power flow approach. *IEEE Transactions on Control of Network Systems*, *8*(3), 1163–1176.

Fagnant, D. J., & Kockelman, K. M. (2018). Dynamic ride-sharing and fleet sizing for a system of shared autonomous vehicles in austin, texas. *Transportation*, *45*(1), 143–158.

Fiedler, D., Čáp, M., & Čertický, M. (2017). Impact of mobility-on-demand on traffic congestion: Simulation-based study. *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, 1–6.

Friesz, T. L., Kim, T., Kwon, C., & Rigdon, M. A. (2011). Approximate network loading and dual-time-scale dynamic user equilibrium. *Transportation Research Part B: Methodological*, *45*(1), 176–207.

Gammelli, D., Yang, K., Harrison, J., Rodrigues, F., Pereira, F. C., & Pavone, M. (2021). Graph neural network reinforcement learning for autonomous mobility-on-demand systems. *2021 60th IEEE Conference on Decision and Control (CDC)*, 2996–3003.

Geisberger, R., Sanders, P., Schultes, D., & Delling, D. (2008). Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In C. C. McGeoch (Ed.), *Experimental Algorithms* (pp. 319–333). Springer Berlin Heidelberg.

George, D. K. (2012). *Stochastic modeling and decentralized control policies for large-scale vehicle sharing systems via closed queueing networks* [Doctoral dissertation, The Ohio State University].

Greif, T., Bouvier, L., Flath, C. M., Parmentier, A., Rohmer, S. U. K., & Vidal, T. (2024). Combinatorial optimization and machine learning for dynamic inventory routing. *arXiv:2402.04463 [math.OC]*. https://doi.org/10.48550/arXiv.2402.04463

Guériau, M., Cugurullo, F., Acheampong, R., & Dusparic, I. (2020). Shared autonomous mobility on demand: A learning-based approach and its performance in the presence of traffic congestion. *IEEE Intelligent Transportation Systems Magazine*, *12*, 208–218.

Guériau, M., & Dusparic, I. (2018). Samod: Shared autonomous mobility-on-demand using decentralized reinforcement learning. *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 1558–1563.

Han, K., Friesz, T. L., & Yao, T. (2013). Existence of simultaneous route and departure choice dynamic user equilibrium. *Transportation Research Part B: Methodological*, *53*, 17–30.

Han, M., Senellart, P., Bressan, S., & Wu, H. (2016). Routing an autonomous taxi with reinforcement learning. *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, 2421–2424.

He, S., Wang, Y., Han, S., Zou, S., & Miao, F. (2023a). A robust and constrained multi-agent reinforcement learning electric vehicle rebalancing method in amod systems. *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5637–5644.

He, S., Zhang, Z., Han, S., Pepin, L., Wang, G., Zhang, D., Stankovic, J. A., & Miao, F. (2023b). Data-driven distributionally robust electric vehicle balancing for autonomous

mobility-on-demand systems under demand and supply uncertainties. *IEEE Transactions on Intelligent Transportation Systems*, *24*(5), 5199–5215.

Heineke, A., Kloss, B., Scurtu, D., & Weig, F. (2021). The future of mobility: The ride-sharing revolution [Industry report].

Heller, C., Johnen, J., Schmitz, S., et al. (2019). Congestion pricing: A mechanism design approach. *Journal of Transport Economics and Policy (JTEP)*, *53*(1), 74–98.

Henao, A., & Marshall, W. E. (2019). An analysis of the individual economics of ride-hailing drivers. *Transportation Research Part A: Policy and Practice*, *130*, 440–451.

Hendrickson, C., & Kocur, G. (1981). Schedule delay and departure time decisions in a deterministic model. *Transportation Science*, *15*(1), 62–77.

Hildebrandt, F. D., Thomas, B. W., & Ulmer, M. W. (2023). Opportunities for reinforcement learning in stochastic dynamic vehicle routing. *Computers & Operations Research*, *150*, 106071.

Hoppe, H., Baty, L., Bouvier, L., Parmentier, A., & Schiffer, M. (2025). Structured reinforcement learning for combinatorial decision-making [Accepted at NeurIPS 2025]. *arXiv:2505.19053 [cs.LG]*. https://doi.org/10.48550/arXiv.2505.19053

Hoppe, H., Enders, T., Cappart, Q., & Schiffer, M. (2024). Global rewards in multi-agent deep reinforcement learning for autonomous mobility on demand systems. *6th Annual Learning for Dynamics & Control Conference*, 260–272.

Hörl, S. (2020). *Dynamic demand simulation for automated mobility-on-demand* [Doctoral dissertation, ETH Zurich].

Hörl, S., Ruch, C., Becker, F., Frazzoli, E., & Axhausen, K. W. (2019). Fleet operational policies for automated mobility: A simulation assessment for Zurich. *Transportation Research Part C: Emerging Technologies*, *102*, 20–31.

Hu, L., & Dong, J. (2022). An artificial-neural-network-based model for real-time dispatching of electric autonomous taxis. *IEEE Transactions on Intelligent Transportation Systems*, *23*(2), 1519–1528.

Hu, T.-Y., & Mahmassani, H. S. (1997). Day-to-day evolution of network flows under real-time information and reactive signal control. *Transportation Research Part C: Emerging Technologies*, *5*(1), 51–69.

Huang, H.-J., & Lam, W. H. (2002). Modeling and solving the dynamic user equilibrium route and departure time choice problem in network with queues. *Transportation Research Part B: Methodological*, *36*(3), 253–273.

Hyland, M., & Mahmassani, H. S. (2018). Dynamic autonomous vehicle fleet operations: Optimization-based strategies to assign avs to immediate traveler demand requests. *Transportation Research Part C: Emerging Technologies*, *92*, 278–297.

Hyland, M., & Mahmassani, H. S. (2020). Operational benefits and challenges of shared-ride automated mobility-on-demand services. *Transportation Research Part A: Policy and Practice*, *134*, 251–270.

Iacobucci, R., McLellan, B., & Tezuka, T. (2018). Modeling shared autonomous electric vehicles: Potential for transport and power grid integration. *Energy*, *158*, 148–163.

Iacobucci, R., McLellan, B., & Tezuka, T. (2019). Optimization of shared autonomous electric vehicles operations with charge scheduling and vehicle-to-grid. *Transportation Research Part C: Emerging Technologies*, *100*, 34–52.

Iglesias, R., Rossi, F., Wang, K., Hallac, D., Leskovec, J., & Pavone, M. (2018). Data-driven model predictive control of autonomous mobility-on-demand systems. *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 6019–6025.

Iglesias, R., Rossi, F., Zhang, R., & Pavone, M. (2019). A BCMP network approach to modeling and controlling autonomous mobility-on-demand systems. *The International Journal of Robotics Research*, *38*(2-3), 357–374.

Jahn, O., Möhring, R. H., Schulz, A. S., & Stier-Moses, N. E. (2005). System-optimal routing of traffic flows with user constraints in networks with congestion. *Operations research*, *53*(4), 600–616.

Jakob, W., Rhinelander, J., & Moldovan, D. (2017). Pybind11 – seamless operability between c++11 and python. https://github.com/pybind/pybind11

Jalota, D., Paccagnan, D., Schiffer, M., & Pavone, M. (2023). Online routing over parallel networks: Deterministic limits and data-driven enhancements. *INFORMS Journal on Computing*, *35*(3), 560–577.

James, J., Yu, W., & Gu, J. (2019). Online vehicle routing with neural combinatorial optimization and deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, *20*(10), 3806–3817.

Jauffred, F. J., & Bernstein, D. (1996). An alternative formulation of the simultaneous route and departure-time choice equilibrium problem. *Transportation Research Part C: Emerging Technologies*, *4*(6), 339–357.

Jiao, Y., Tang, X., Qin, Z. (, Li, S., Zhang, F., Zhu, H., & Ye, J. (2021). Real-world ride-hailing vehicle repositioning using deep reinforcement learning. *Transportation Research Part C: Emerging Technologies*, *130*, 103289.

Jindal, I., Qin, Z. T., Chen, X., Nokleby, M., & Ye, J. (2018). Optimizing taxi carpool policies via reinforcement learning and spatio-temporal mining. *2018 IEEE International Conference on Big Data*, 1417–1426.

Joe, W., & Lau, H. C. (2020). Deep reinforcement learning approach to solve dynamic vehicle routing problem with stochastic customers. *Proceedings of the International Conference on Automated Planning and Scheduling*, *30*, 394–402.

Jungel, K., Paccagnan, D., Parmentier, A., & Schiffer, M. (2024). Wardropnet: Traffic flow predictions via equilibrium-augmented learning. *arXiv:2410.06656 [cs.LG]*. https://doi.org/10.48550/arXiv.2410.06656

Jungel, K., Parmentier, A., Schiffer, M., & Vidal, T. (2025). Learning-based online optimization for autonomous mobility-on-demand fleet control. *INFORMS Journal on Computing*.

Kamel, J., Vosooghi, R., Puchinger, J., Ksontini, F., & Sirin, G. (2019). Exploring the impact of user preferences on shared autonomous vehicle modal split: A multi-agent simulation approach. *Transportation Research Procedia*, *37*, 115–122.

Kang, D., & Levin, M. W. (2021). Maximum-stability dispatch policy for shared autonomous vehicles. *Transportation Research Part B: Methodological*, *148*, 132–151.

Ke, Z., & Qian, S. (2023). Leveraging ride-hailing services for social good: Fleet optimal routing and system optimal pricing. *Transportation Research Part C: Emerging Technologies*, *155*, 104284.

Klein, I., Levy, N., & Ben-Elia, E. (2018). An agent-based model of the emergence of cooperation and a fair and stable system optimum using atis on a simple road network. *Transportation Research Part C: Emerging Technologies*, *86*, 183–201.

Köhler, E., Langkau, K., & Skutella, M. (2002). Time-expanded graphs for flow-dependent transit times. *Algorithms - ESA 2002: 10th Annual European Symposium Rome, Italy, September 17–21, 2002 Proceedings 10*, 599–611.

Kondor, D., Bojic, I., Resta, G., Duarte, F., Santi, P., & Ratti, C. (2022). The cost of non-coordination in urban on-demand mobility. *Scientific Reports*, *12*(1), 4669.

Kong, L., Cui, J., Zhuang, Y., Feng, R., Prakash, B. A., & Zhang, C. (2022). End-to-end stochastic optimization with energy-based models. *Advances in Neural Information Processing Systems*, *35*, 11341–11354.

Korilis, Y. A., Lazar, A. A., & Orda, A. (2002). Achieving network optima using stackelberg routing strategies. *IEEE/ACM Transactions on Networking*, *5*(1), 161–173.

Kotary, J., Fioretto, F., Hentenryck, P. V., & Wilder, B. (2021). End-to-end constrained optimization learning: A survey. *arXiv:2103.16378 [cs.LG]*. https://doi.org/10.48550/arXiv.2103.16378

Krajzewicz, D. (2010). Traffic simulation with SUMO – simulation of urban mobility. In *Fundamentals of Traffic Simulation* (pp. 269–293). Springer.

Krichene, W., Reilly, J. D., Amin, S., & Bayen, A. M. (2014). Stackelberg routing on parallel networks with horizontal queues. *IEEE Transactions on Automatic Control*, *59*(3), 714–727.

Kumarage, S., Yildirimoglu, M., Ramezani, M., & Zheng, Z. (2021). Schedule-constrained demand management in two-region urban networks. *Transportation Science*, *55*(4), 857–882.

Lam, C. T., Liu, M., & Hui, X. (2021). The geography of ridesharing: A case study on New York City. *Information Economics and Policy*, *57*, 100941.

Lanzetti, N., Schiffer, M., Ostrovsky, M., & Pavone, M. (2023). On the interplay between self-driving cars and public transportation. *IEEE Transactions on Control of Network Systems*, *11*(3), 1478–1490.

Lazar, D. A., Bıyık, E., Sadigh, D., & Pedarsani, R. (2021). Learning how to dynamically route autonomous vehicles on shared roads. *Transportation Research Part C: Emerging Technologies*, *130*, 103258.

Levin, M. W. (2017). Congestion-aware system optimal route choice for shared autonomous vehicles. *Transportation Research Part C: Emerging Technologies*, *82*, 229–247.

Levin, M. W., Kockelman, K. M., Boyles, S. D., & Li, T. (2017). A general framework for modeling shared autonomous vehicles with dynamic network-loading and dynamic ride-sharing application. *Computers, Environment and Urban Systems*, *64*, 373–383.

Levy, J. I., Buonocore, J. J., & Von Stackelberg, K. (2010). Evaluation of the public health impacts of traffic congestion: A health risk assessment. *Environmental Health*, *9*, 1–12.

Li, C., Parker, D., & Hao, Q. (2021a). Optimal online dispatch for high-capacity shared autonomous mobility-on-demand systems. *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 779–785.

Li, L., Pantelidis, T., Chow, J. Y., & Jabari, S. E. (2021b). A real-time dispatching strategy for shared automated electric vehicles with performance guarantees. *Transportation Research Part E: Logistics and Transportation Review*, *152*, 102392.

Li, M., Qin, Z., Jiao, Y., Yang, Y., Wang, J., Wang, C., Wu, G., & Ye, J. (2019a). Efficient ridesharing order dispatching with mean field multi-agent reinforcement learning. *The World Wide Web Conference*, 983–994.

Li, Y., & Vignon, D. (2024). Do ride-hailing congestion fees in nyc work? *Transportation Research Part A: Policy and Practice*, *190*, 104274.

Li, Z., Wu, Q., Yu, H., Chen, C., Zhang, G., Tian, Z., & Prevedouros, P. (2019b). Temporal-spatial dimension extension-based intersection control formulation for connected and autonomous vehicle systems. *Transportation Research Part C: Emerging Technologies*.

Li, Z.-C., Huang, H.-J., & Yang, H. (2020). Fifty years of the bottleneck model: A bibliometric review and future research directions. *Transportation Research Part B: Methodological*, *139*, 311–342.

Liang, E., Wen, K., Lam, W. H. K., Sumalee, A., & Zhong, R. (2022). An integrated reinforcement learning and centralized programming approach for online taxi dispatching. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(9), 4742–4756.

Lindsey, C. R., Van den Berg, V. A., & Verhoef, E. T. (2012). Step tolling with bottleneck queuing congestion. *Journal of Urban Economics*, *72*(1), 46–59.

Lindsey, R., & Verhoef, E. (2001). Traffic congestion and congestion pricing. in: Handbook of transport systems and traffic control. *Publication of: Elsevier Scientific Publishing Company*.

Liu, J., Kockelman, K. M., Boesch, P. M., & Ciari, F. (2017). Tracking a system of shared autonomous vehicles across the austin, texas network using agent-based simulation. *Transportation*, *44*(6), 1261–1278.

Liu, Y., & Nie, Y. M. (2011). Morning commute problem considering route choice, user heterogeneity and alternative system optima. *Transportation Research Part B: Methodological*, *45*(4), 619–642.

Liu, Y., & Samaranayake, S. (2022). Proactive rebalancing and speed-up techniques for on-demand high capacity ridesourcing services. *IEEE Transactions on Intelligent Transportation Systems*, *23*(2), 819–826.

Liu, Z., Miwa, T., Zeng, W., Bell, M. G., & Morikawa, T. (2019). Dynamic shared autonomous taxi system considering on-time arrival reliability. *Transportation Research Part C: Emerging Technologies*, *103*, 281–297.

Lo, H. K., & Szeto, W. (2002). A cell-based variational inequality formulation of the dynamic user optimal assignment problem. *Transportation Research Part B: Methodological*, *36*(5), 421–443.

Long, J., Huang, H.-J., Gao, Z., & Szeto, W. Y. (2013). An intersection-movement-based dynamic user optimal route choice problem. *Operations Research*, *61*(5), 1134–1147.

Long, J., Wang, C., & Szeto, W. (2018). Dynamic system optimum simultaneous route and departure time choice problems: Intersection-movement-based formulations and comparisons. *Transportation Research Part B: Methodological*, *115*, 166–206.

Lujak, M., Giordani, S., & Ossowski, S. (2015). Route guidance: Bridging system and user optimization in traffic assignment. *Neurocomputing*, *151*, 449–460.

Luke, J., Salazar, M., Rajagopal, R., & Pavone, M. (2021). Joint optimization of autonomous electric vehicle fleet operations and charging station siting. *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, 3340–3347.

Maciejewski, M., & Bischoff, J. (2018). Congestion effects of autonomous taxi fleets. *Transport*, *33*(4), 971–980.

Makridis, C., Menelaou, C., Timotheou, S., & Panayiotou, C. (2024). A real-time demand management and route-guidance system for eliminating congestion. *IEEE Intelligent Transportation Systems Magazine*, *16*, 86–104.

Mandi, J., Kotary, J., Berden, S., Mulamba, M., Bucarey, V., Guns, T., & Fioretto, F. (2024). Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. *Journal of Artificial Intelligence Research*, *80*, 1623–1701.

Menelaou, C., Timotheou, S., Kolios, P., & Panayiotou, C. (2024). Efficient demand management for the on-time arrival problem: A convexified multi-objective approach assuming macroscopic traffic dynamics. *European Journal of Control*, 101057.

Menelaou, C., Timotheou, S., Kolios, P., & Panayiotou, C. G. (2021). Joint route guidance and demand management for real-time control of multi-regional traffic networks. *IEEE Transactions on Intelligent Transportation Systems*, *23*(7), 8302–8315.

Miralinaghi, M., & Peeta, S. (2016). Multi-period equilibrium modeling planning framework for tradable credit schemes. *Transportation Research Part E: Logistics and Transportation Review*, *93*, 177–198.

Miralinaghi, M., & Peeta, S. (2019). Promoting zero-emissions vehicles using robust multi-period tradable credit scheme. *Transportation Research Part D: Transport and Environment*, *75*, 265–285.

Mišić, V. V., & Perakis, G. (2020). Data analytics in operations management: A review. *Manufacturing & Service Operations Management*, *22*(1), 158–169.

Morandi, V. (2024). Bridging the user equilibrium and the system optimum in static traffic assignment: A review. *4or*, *22*(1), 89–119.

Mounce, R., & Carey, M. (2011). Route swapping in dynamic traffic networks. *Transportation Research Part B: Methodological*, *45*(1), 102–111.

Nahmias-Biran, B.-h., Oke, J. B., Kumar, N., Lima Azevedo, C., & Ben-Akiva, M. (2021). Evaluating the impacts of shared automated mobility on-demand services: An activity-based accessibility approach. *Transportation*, *48*(4), 1613–1638.

Narayanan, S., Chaniotakis, E., & Antoniou, C. (2020). Shared autonomous vehicle services: A comprehensive review. *Transportation Research Part C: Emerging Technologies*, *111*, 255–293.

Neumann, K., Schwindt, C., & Zimmermann, J. (2002). *Project scheduling with time windows and scarce resources: Temporal and resource-constrained project scheduling with regular and nonregular objective functions* (Vol. 508). Springer.

Nguyen-Phuoc, D. Q., Zhou, M., Chua, M. H., Alho, A. R., Oh, S., Seshadri, R., & Le, D.-T. (2023). Examining the effects of automated mobility-on-demand services on public transport systems using an agent-based simulation approach. *Transportation Research Part A: Policy and Practice*, *169*, 103583.

Nie, Y. (2010). Solving the dynamic user optimal assignment problem considering queue spillback. *Networks and Spatial Economics*, *10*, 49–71.

Nowozin, S., & Lampert, C. H. (2011). Structured learning and prediction in computer vision. *Foundations and Trends® in Computer Graphics and Vision*, *6*(3–4), 185–365.

Nowozin, S., Lampert, C. H., et al. (2011). Structured learning and prediction in computer vision. *Foundations and Trends® in Computer Graphics and Vision*, *6*(3–4), 185–365.

NYCTLC. (2009). TLC Trip Record Data. https://tinyurl.com/tlc-trip-data

NYCTLC. (2015). TLC Trip Record Data. https://tinyurl.com/tlc-trip-data

NYMTC. (2023). Hub bound travel data 2023 [New York Metropolitan Transportation Council]. https://tinyurl.com/3bdsvhsd

Oh, S., Lentzakis, A. F., Seshadri, R., & Ben-Akiva, M. (2021). Impacts of automated mobility-on-demand on traffic dynamics, energy and emissions: A case study of Singapore. *Simulation Modelling Practice and Theory*, *110*, 102327.

Oh, S., Seshadri, R., Azevedo, C. L., Kumar, N., Basak, K., & Ben-Akiva, M. (2020). Assessing the impacts of automated mobility-on-demand through agent-based simulation: A study of singapore. *Transportation Research Part A: Policy and Practice*, *138*, 367–388.

Oke, J. B., Akkinepally, A. P., Chen, S., Xie, Y., Aboutaleb, Y. M., Azevedo, C. L., Zegras, P. C., Ferreira, J., & Ben-Akiva, M. (2020). Evaluating the systemic effects of automated mobility-on-demand services via large-scale agent-based simulation of auto-dependent prototype cities. *Transportation Research Part A: Policy and Practice*, *140*, 98–126.

OpenStreetMap. (2024). https://www.openstreetmap.org

Otsubo, H., & Rapoport, A. (2008). Vickrey's model of traffic congestion discretized. *Transportation Research Part B: Methodological*, *42*(10), 873–889.

Papageorgiou, M., & Kotsialos, A. (2003). Freeway ramp metering: An overview. *IEEE Transactions on Intelligent Transportation Systems*, *3*(4), 271–281.

Parmentier, A. (2021). Learning structured approximations of combinatorial optimization problems. *arXiv:2107.04323 [cs.RO]*. https://doi.org/10.48550/arXiv.2107.04323

Patriksson, M. (2015). *The traffic assignment problem: Models and methods*. Courier Dover Publications.

Pavone, M. (2015). Autonomous mobility-on-demand systems for future urban mobility. In *Autonomes fahren: Technische, rechtliche und gesellschaftliche aspekte* (pp. 399–416). Springer Berlin Heidelberg.

Pavone, M., Smith, S. L., Frazzoli, E., & Rus, D. (2012). Robotic load balancing for mobility-on-demand systems. *The International Journal of Robotics Research*, *31*(7), 839–854.

Peeta, S., & Ziliaskopoulos, A. K. (2001). Foundations of dynamic traffic assignment: The past, the present and the future. *Networks and Spatial Economics*, *1*, 233–265.

Pigou, A. (1920). *The economics of welfare*. London, Macmillan.

Pishue, B. (2023). 2022 INRIX global traffic scorecard.

Qian, G., Guo, M., Zhang, L., Wang, Y., Hu, S., & Wang, D. (2021). Traffic scheduling and control in fully connected and automated networks. *Transportation Research Part C: Emerging Technologies*, *126*, 103011.

Ramezani, M., Haddad, J., & Geroliminis, N. (2015). Dynamics of heterogeneity in urban networks: Aggregated traffic modeling and hierarchical control. *Transportation Research Part B: Methodological*, *74*, 1–19.

Ran, B., Hall, R. W., & Boyce, D. E. (1996). A link-based variational inequality model for dynamic departure time/route choice. *Transportation Research Part B: Methodological*, *30*(1), 31–46.

Rautenstrauss, M., & Schiffer, M. (2025). Optimization-augmented machine learning for vehicle operations in emergency medical services. *arXiv:2503.11848 [cs.LG]*. https://doi.org/10.48550/arXiv.2503.11848

Rayle, L., Dai, D., Chan, N., Cervero, R., & Shaheen, S. (2016). Just a better taxi? a survey-based comparison of taxis, transit, and ridesourcing services in san francisco. *Transport policy*, *45*, 168–178.

Rossi, F., Iglesias, R., Alizadeh, M., & Pavone, M. (2020). On the interaction between autonomous mobility-on-demand systems and the power network: Models and coordination algorithms. *IEEE Transactions on Control of Network Systems*, *7*(1), 384–397.

Rossi, F., Zhang, R., Hindy, Y., & Pavone, M. (2018). Routing autonomous vehicles in congested transportation networks: Structural properties and coordination algorithms. *Autonomous Robots*, *42*, 1427–1442.

Roughgarden, T. (2005). *Selfish routing and the price of anarchy* (Vol. 74). MIT press.

Ruch, C., Ehrler, R., Hörl, S., Balac, M., & Frazzoli, E. (2021). Simulation-based assessment of parking constraints for automated mobility on demand: A case study of Zurich. *Vehicles*, *3*(2), 272–286.

Ruch, C., Hörl, S., & Frazzoli, E. (2018). Amodeus, a simulation-based testbed for autonomous mobility-on-demand systems. *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 3639–3644.

Ruch, C., Lu, C., Sieber, L., & Frazzoli, E. (2020). Quantifying the efficiency of ride sharing. *IEEE Transactions on Intelligent Transportation Systems*, *22*(9), 5811–5816.

Sadana, U., Chenreddy, A., Delage, E., Forel, A., Frejinger, E., & Vidal, T. (2025). A survey of contextual optimization methods for decision-making under uncertainty. *European Journal of Operational Research*, *320*(2), 271–289.

Sadeghi Eshkevari, S., Tang, X., Qin, Z., Mei, J., Zhang, C., Meng, Q., & Xu, J. (2022). Reinforcement learning in the wild: Scalable RL dispatching algorithm deployed in ride-hailing marketplace. *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 3838–3848.

Salazar, M., Lanzetti, N., Rossi, F., Schiffer, M., & Pavone, M. (2020). Intermodal autonomous mobility-on-demand. *IEEE Transactions on Intelligent Transportation Systems*, *21*, 3946–3960.

Salazar, M., Tsao, M., Aguiar, I., Schiffer, M., & Pavone, M. (2019). A congestion-aware routing scheme for autonomous mobility-on-demand systems. *2019 18th European Control Conference (ECC)*, 3040–3046.

Schiffer, M., Hoppe, H., Su, Y., Bouvier, L., & Parmentier, A. (2026). Combinatorial optimization augmented machine learning. *arXiv:2601.10583 [cs.LG]*. https://doi.org/10.48550/arXiv.2601.10583

Schmidt, C., Gammelli, D., Pereira, F. C., & Rodrigues, F. (2024). Learning to control autonomous fleets from observation via offline reinforcement learning. *2024 European Control Conference (ECC)*, 1399–1406.

Seilabi, S. E., Tabesh, M. T., Davatgari, A., Miralinaghi, M., & Labi, S. (2020). Promoting autonomous vehicles using travel demand and lane management strategies. *Frontiers in Built Environment*, *6*, 560116.

Sheffi, Y. (1985). *Urban transportation networks* (Vol. 6). Prentice-Hall, Englewood Cliffs, NJ.

Shirmohammadi, N., & Yin, Y. (2016). Tradable credit scheme to control bottleneck queue length. *Transportation Research Record*, *2561*(1), 53–63.

Shirmohammadi, N., Zangui, M., Yin, Y., & Nie, Y. (2013). Analysis and design of tradable credit schemes under uncertainty. *Transportation Research Record*, *2333*(1), 27–36.

Sichitiu, M. L., & Kihl, M. (2008). Inter-vehicle communication systems: A survey. *IEEE Communications Surveys & Tutorials*, *10*(2), 88–105.

Singhal, A., Gammelli, D., Luke, J., Gopalakrishnan, K., Helmreich, D., & Pavone, M. (2024). Real-time control of electric autonomous mobility-on-demand systems via graph reinforcement learning. *2024 European Control Conference (ECC)*, 1407–1414.

Skordilis, E., Hou, Y., Tripp, C., Moniot, M., Graf, P., & Biagioni, D. (2021). A modular and transferable reinforcement learning framework for the fleet rebalancing problem. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), 11903–11916.

Solovey, K., Salazar, M., & Pavone, M. (2019). Scalable and congestion-aware routing for autonomous mobility-on-demand via frank–wolfe optimization. *arXiv:1903.03697 [math.OC]*. https://doi.org/10.48550/arXiv.1903.03697

Song, Z., Yin, Y., & Lawphongpanich, S. (2015). Optimal deployment of managed lanes in general networks. *International Journal of Sustainable Transportation*, *9*(6), 431–441.

Speranza, M. G. (2018). Trends in transportation and logistics. *European Journal of Operational Research*, *264*(3), 830–836.

Spieckermann, C., Minner, S., & Schiffer, M. (2025). Reduce-then-optimize for the fixed-charge transportation problem. *Transportation Science*, *59*(3), 540–564.

Spieser, K., Treleaven, K., Zhang, R., Frazzoli, E., Morton, D., & Pavone, M. (2014). Toward a systematic approach to the design and evaluation of automated mobility-on-demand systems: A case study in Singapore. In G. Meyer & S. Beiker (Eds.), *Road Vehicle Automation* (pp. 229–245). Springer International Publishing.

Sriskandarajah, C., & Ladet, P. (1986). Some no-wait shops scheduling problems: Complexity aspect [Flexible Manufacturing Systems]. *European Journal of Operational Research*, *24*(3), 424–438.

Statista. (2024). Ride-hailing - United States. https://tinyurl.com/ride-hailing-us

Statista. (2025). Uber technologies – statistics & facts. https://tinyurl.com/statista-uber

Stopher, P. R. (2004). Reducing road congestion: A reality check. *Transport Policy*, *11*(2), 117–131.

Swamy, C. (2012). The effectiveness of stackelberg strategies and tolls for network congestion games. *ACM Transactions on Algorithms (TALG)*, *8*(4), 1–19.

Tang, X., Qin, Z. (, Zhang, F., Wang, Z., Xu, Z., Ma, Y., Zhu, H., & Ye, J. (2019). A deep value-network based approach for multi-driver order dispatching. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1780–1790.

Thomopoulos, N. T. (2012). *Fundamentals of queuing systems: Statistical methods for analyzing queuing models*. Springer Science & Business Media.

Tirachini, A., & Gomez-Lobo, A. (2020). Does ride-hailing increase or decrease vehicle kilometers traveled (VKT)? a simulation approach for santiago de chile. *International Journal of Sustainable Transportation*, *14*(3), 187–204.

Tresca, L., Schmidt, C., Harrison, J., Rodrigues, F., Zardini, G., Gammelli, D., & Pavone, M. (2025). Robo-taxi fleet coordination at scale via reinforcement learning. *arXiv:2504.06125 [cs.LG]*. https://doi.org/10.48550/arXiv.2504.06125

Tsao, M., Iglesias, R., & Pavone, M. (2018). Stochastic model predictive control for autonomous mobility on demand. *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 3941–3948.

Tsao, M., Milojevic, D., Ruch, C., Salazar, M., Frazzoli, E., & Pavone, M. (2019). Model predictive control of ride-sharing autonomous mobility-on-demand systems. *2019 International Conference on Robotics and Automation (ICRA)*, 6665–6671.

Turan, B., Pedarsani, R., & Alizadeh, M. (2020). Dynamic pricing and fleet management for electric autonomous mobility on demand systems. *Transportation Research Part C: Emerging Technologies*, *121*, 102829.

Ukkusuri, S. V., Han, L., & Doan, K. (2012). Dynamic user equilibrium with a path based cell transmission model for general traffic networks. *Transportation Research Part B: Methodological*, *46*(10), 1657–1684.

US Bur. Public Roads. (1964). *Traffic assignment manual for application with a large, high speed computer* (Vol. 2). US Gov. Print. Off.

van Essen, M., Eikenbroek, O., Thomas, T., & van Berkum, E. (2019). Travelers' compliance with social routing advice: Impacts on road network performance and equity. *IEEE Transactions on Intelligent Transportation Systems*, *21*(3), 1180–1190.

Vazifeh, M. M., Santi, P., Resta, G., Strogatz, S. H., & Ratti, C. (2018). Addressing the minimum fleet problem in on-demand urban mobility. *Nature*, *557*(7706), 534–538.

Vickrey, W. S. (1969). Congestion theory and transport investment. *The American Economic Review*, *59*(2), 251–260.

Vlastelica, M., Paulus, A., Musil, V., Martius, G., & Rolínek, M. (2019). Differentiation of blackbox combinatorial solvers. *arXiv:1912.02175 [cs.LG]*. https://doi.org/10.48550/arXiv.1912.02175

Wallar, A., Van Der Zee, M., Alonso-Mora, J., & Rus, D. (2018). Vehicle rebalancing for mobility-on-demand systems with ride-sharing. *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 4539–4546.

Wang, H., & Yang, H. (2019). Ridesourcing systems: A framework and review. *Transportation Research Part B: Methodological*, *129*, 122–155.

Wang, X., Yang, H., Zhu, D., & Li, C. (2012). Tradable travel credits for congestion management with heterogeneous users. *Transportation Research Part E: Logistics and Transportation Review*, *48*(2), 426–437.

Wang, Y., Szeto, W. Y., Han, K., & Friesz, T. L. (2018). Dynamic traffic assignment: A review of the methodological advances for environmentally sustainable road transportation applications. *Transportation Research Part B: Methodological*, *111*, 370–394.

Ward, J. W., Michalek, J. J., Samaras, C., Azevedo, I. L., Henao, A., Rames, C., & Wenzel, T. (2021). The impact of Uber and Lyft on vehicle ownership, fuel economy, and transit across u.s. cities. *iScience*, *24*(1), 101933.

Wardrop, J. G. (1952). Road paper. some theoretical aspects of road traffic research. *Proceedings of the Institution of Civil Engineers*, *1*(3), 325–362.

Wen, J., Chen, Y. X., Nassir, N., & Zhao, J. (2018). Transit-oriented autonomous vehicle operation with integrated demand-supply interaction. *Transportation Research Part C: Emerging Technologies*, *97*, 216–234.

Wen, J., Nassir, N., & Zhao, J. (2019). Value of demand information in autonomous mobility-on-demand systems. *Transportation Research Part A: Policy and Practice*, *121*, 346–359.

Wollenstein-Betech, S., Houshmand, A., Salazar, M., Pavone, M., Cassandras, C., & Paschalidis, I. (2020). Congestion-aware routing and rebalancing of autonomous mobility-on-demand systems in mixed traffic. *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, 1–7.

Wollenstein-Betech, S., Salazar, M., Houshmand, A., Pavone, M., Paschalidis, I. C., & Cassandras, C. G. (2022). Routing and rebalancing intermodal autonomous mobility-on-demand systems in mixed traffic. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), 12263–12275.

Woywood, Z., Wiltfang, J. I., Luy, J., Enders, T., & Schiffer, M. (2024). Multi-agent soft actor-critic with coordinated loss for autonomous mobility-on-demand fleet control. *arXiv:2404.06975 [eess.SY]*. https://doi.org/10.48550/arXiv.2404.06975

Xu, Z., Li, Z., Guan, Q., Zhang, D., Li, Q., Nan, J., Liu, C., Bian, W., & Ye, J. (2018). Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach. *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 905–913.

Yang, H., & Huang, H.-J. (2005). *Mathematical and economic theory of road pricing*. Emerald Group Publishing Limited.

Yang, H., & Meng, Q. (1998). Departure time, route choice and congestion toll in a queuing network with elastic demand. *Transportation Research Part B: Methodological*, *32*(4), 247–260.

Yang, H., & Wang, X. (2011). Managing network mobility with tradable credits. *Transportation Research Part B: Methodological*, *45*(3), 580–594.

Yazawa, Y., Fujita, Y., Muramatsu, T., Ishiko, Y., Hochi, D., & Yamaguchi, M. (2025, November). *Autonomous mobility facts 2025: A global shift from pilots to implementation* (tech. rep.) (Report). PwC. https://www.pwc.com/jp/en/knowledge/thoughtleadership/autonomous-mobility-facts-2025.html

Yildirimoglu, M., & Ramezani, M. (2020). Demand management with limited cooperation among travellers: A doubly dynamic approach [23rd International Symposium on Transportation and Traffic Theory (ISTTT 23)]. *Transportation Research Part B: Methodological*, *132*, 267–284.

Yu, X., & Shen, S. (2020). An integrated decomposition and approximate dynamic programming approach for on-demand ride pooling. *IEEE Transactions on Intelligent Transportation Systems*, *21*(9), 3811–3820.

Zanardi, A., Sessa, P. G., Käslin, N., Bolognani, S., Censi, A., & Frazzoli, E. (2023). How bad is selfish driving? bounding the inefficiency of equilibria in urban driving games. *IEEE Robotics and Automation Letters*, *8*(4), 2293–2300.

Zardini, G., Lanzetti, N., Belgioioso, G., Hartnik, C., Bolognani, S., Dörfler, F., & Frazzoli, E. (2023). Strategic interactions in multi-modal mobility systems: A game-theoretic perspective. *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, 5452–5459.

Zardini, G., Lanzetti, N., Pavone, M., & Frazzoli, E. (2022). Analysis and control of autonomous mobility-on-demand systems. *Annual Review of Control, Robotics, and Autonomous Systems*, *5*(1), 633–658.

Zgraggen, J., Tsao, M., Salazar, M., Schiffer, M., & Pavone, M. (2019). A model predictive control scheme for intermodal autonomous mobility-on-demand. *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, 1953–1960.

Zhang, L., Hu, T., Min, Y., Wu, G., Zhang, J., Feng, P., Gong, P., & Ye, J. (2017). A taxi order dispatch model based on combinatorial optimization. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2151–2159.

Zhang, W., & Guhathakurta, S. (2017). Parking spaces in the age of shared autonomous vehicles: How much parking will we need and where? *Transportation Research Record*, *2651*(1), 80–91.

Zhou, M., Le, D.-T., Nguyen-Phuoc, D. Q., Zegras, P. C., & Ferreira Jr, J. (2021). Simulating impacts of automated mobility-on-demand on accessibility and residential relocation. *Cities*, *118*, 103345.

Zhou, M., Jin, J., Zhang, W., Qin, Z., Jiao, Y., Wang, C., Wu, G., Yu, Y., & Ye, J. (2019). Multi-agent reinforcement learning for order-dispatching via order-vehicle distribution matching. *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2645–2653.

Ziliaskopoulos, A. K., Waller, S. T., Li, Y., & Byram, M. (2004). Large-scale dynamic traffic assignment: Implementation issues and computational analysis. *Journal of Transportation Engineering*, *130*(5), 585–593.

Ziliaskopoulos, A. K., & Rao, L. (1999). A simultaneous route and departure time choice equilibrium model on dynamic networks. *International Transactions in Operational Research*, *6*(1), 21–37.